

BY CLARIFY KNOWLEDGE

SAVIOUR OF 2.0 COMPUTERS

ICSE CLASS 10TH



**THEORY AND PROGRAMMING
NOTES WITH 50 MOST
IMPORTANT PROGRAMS AND
LAST 10 YEARS SOLUTIONS**

IN 300 PAGES

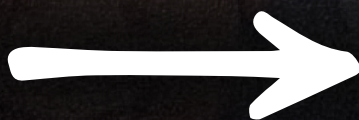
9TH AND 10TH

SAVIOUR Ebooks 2.0

get all our saviour 2.0 eBooks

worth

~~₹2999~~



in just

₹799

also Get flat ₹200 Off by
Using "SAVIOUR"



SHOP NOW

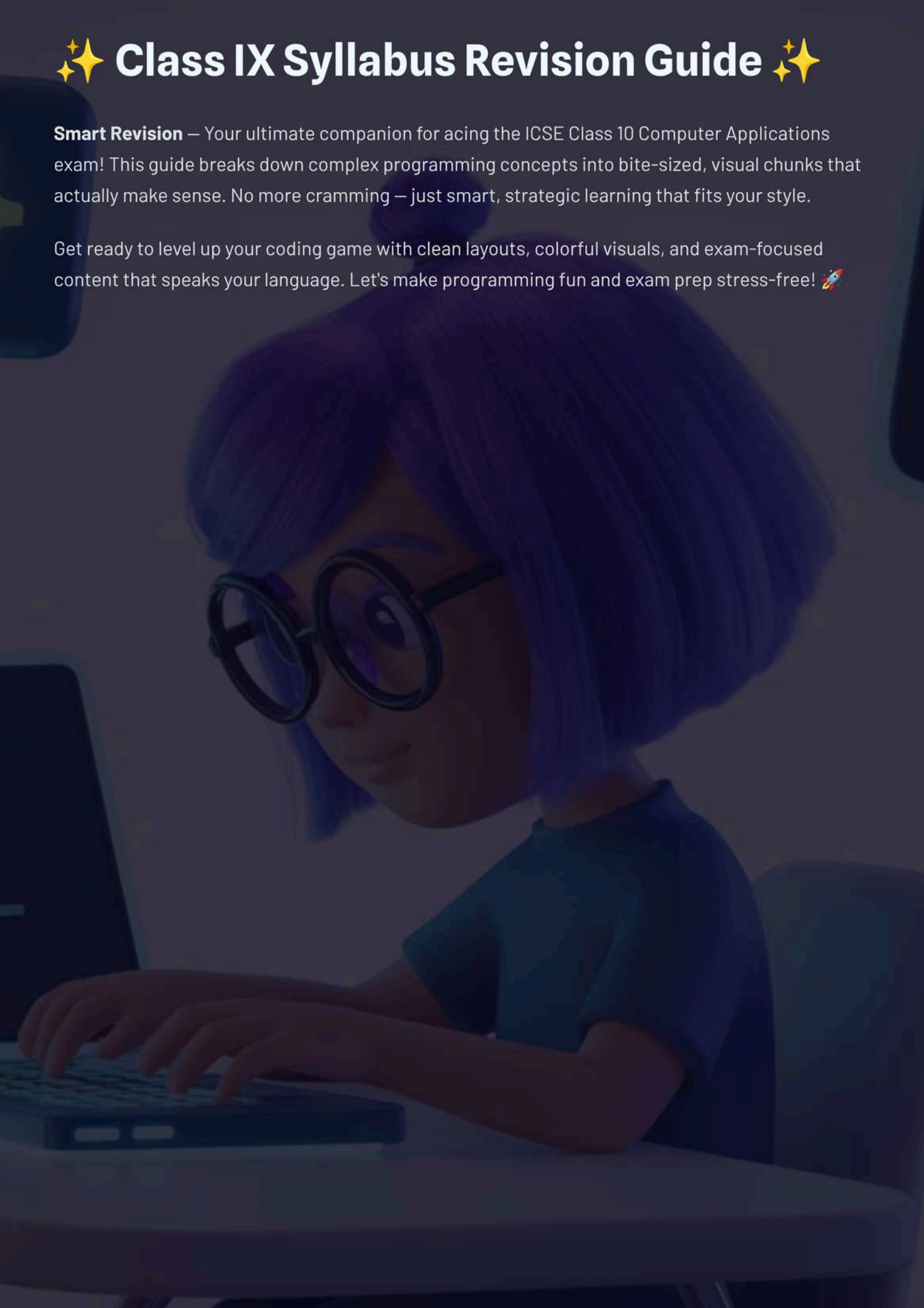
click on this button
to Purchase the
Bundle !



✨ Class IX Syllabus Revision Guide ✨

Smart Revision — Your ultimate companion for acing the ICSE Class 10 Computer Applications exam! This guide breaks down complex programming concepts into bite-sized, visual chunks that actually make sense. No more cramming — just smart, strategic learning that fits your style.

Get ready to level up your coding game with clean layouts, colorful visuals, and exam-focused content that speaks your language. Let's make programming fun and exam prep stress-free! 🚀



Why Revision Matters

Revision isn't just about memorizing – it's about **connecting the dots** between concepts and building confidence for your exam day. Think of it as leveling up in a video game where each topic mastered unlocks new skills!

Smart revision helps you identify knowledge gaps, reinforce weak areas, and build muscle memory for coding patterns. Plus, when you understand the "why" behind concepts, you can tackle any curveball question the exam throws at you.

01

Assessment Phase

Identify your strong and weak topics through practice tests

02

Concept Mapping

Connect related topics and understand their relationships

03

Practice & Apply

Write code, solve problems, and test your understanding

04

Review & Refine

Revisit challenging areas and strengthen your foundation

Remember: **consistency beats cramming** every single time. Short, focused study sessions will get you further than marathon study nights!

OOP Concepts: The Big Four

Object-Oriented Programming is like organizing your digital life – everything has its place and purpose! These four pillars form the foundation of Java programming and are **super important** for your exam.

Encapsulation

Data Hiding

- Wrapping data and methods together
- Private variables with public methods
- Protects data from outside interference

Abstraction

Hiding Complexity

- Focus on what, not how
- Abstract classes and interfaces
- Simplifies complex systems



Inheritance

Code Reusability

- Child class inherits from parent class
- Extends keyword in Java
- Reduces code duplication

Polymorphism

Many Forms

- Same method, different behaviors
- Method overloading and overriding
- One interface, multiple implementations



Exam Tip: Remember the acronym **EIPA** – Encapsulation, Inheritance, Polymorphism, Abstraction!

Classes & Objects

Think of a **class** as a blueprint and an **object** as the actual house built from that blueprint. Classes define the structure and behavior, while objects are the real instances that exist in memory and do the actual work!

Class: The Blueprint

- A template or blueprint from which objects are created.
- Defines common structure (attributes/fields) and behavior (methods) for all objects of its type.
- It's a logical construct; no memory is allocated for a class itself to store data.
- Contains constructors to initialize new objects.
- Example: The architectural plan for a "Car".

Object: The Actual House

- A concrete instance of a class.
- Represents a real-world entity with a specific state (values for its attributes).
- Possesses behavior (actions performed via its methods).
- Has a unique identity (a distinct memory address).
- Memory is allocated when an object is created (instantiated).
- Example: Your specific "Toyota Corolla" or "Ford F-150".

Key Characteristics

For Classes

- **Attributes/Fields:** Variables that define the characteristics or properties of the objects (e.g., for a 'Car' class: `color`, `make`, `model`, `speed`).
- **Methods/Behaviors:** Functions that define what objects of the class can do (e.g., for a 'Car' class: `start()`, `accelerate()`, `brake()`).
- **Constructors:** Special methods used to initialize new objects. They have the same name as the class and are called when an object is created using the `new` keyword.

For Objects

- **State:** The values of an object's attributes at a particular moment (e.g., a 'Car' object might have `color="red"`, `speed=60 mph`).
- **Behavior:** The actions an object can perform, defined by the methods of its class. These behaviors often modify the object's state.
- **Identity:** Each object is a unique entity in memory, distinguishable from other objects even if they have the same state.

Classes & Objects: Practical Example

Let's see classes and objects in action with a simple Java example. We'll define a **Student class** (our blueprint) and then create a **Student object** (an actual student), with unique data but sharing common behaviors.

```
// Define the Student class
class Student {
    // Instance variables (attributes)
    String name;
    int rollNo;

    // Constructor to initialize the object
    Student(String studentName, int studentRollNo) {
        name = studentName;
        rollNo = studentRollNo;
    }

    // Method to display student information
    void displayStudentInfo() {
        System.out.println("Student Name: " + name);
        System.out.println("Roll Number: " + rollNo);
    }
}

// Main class to create and use Student objects
public class Main {
    public static void main(String[] args) {
        // Create an object (instance) of the Student class
        Student student1 = new Student("Alice Smith", 101);

        // Call method on the object to display its information
        System.out.println("--- Student Details ---");
        student1.displayStudentInfo();
    }
}
```


Output:

--- Student Details ---

Student Name: Alice Smith

Roll Number: 101

📌 Key Point: The 'new' keyword

new is crucial for object creation. It allocates memory for a new object and calls the constructor to initialize its variables.

Data Types in Java

Data types are like **containers** that tell Java what kind of information you're storing. Getting this right is crucial because it affects memory usage and what operations you can perform!

Type	Category	Examples
int	Primitive	42, -17, 0
double	Primitive	3.14, -2.7
char	Primitive	'A', '5', '@'
boolean	Primitive	true, false
String	Non-Primitive	"Hello", "Java"
Array	Non-Primitive	int[], String[]

✓ Quick Demo Program

```
public class DataTypeDemo {  
    public static void main(String[] args) {  
        int number = 25;  
        double price = 99.99;  
        char grade = 'A';  
        boolean isPass = true;  
        String name = "Jordan";  
  
        System.out.println("Number: " + number);  
        System.out.println("Price: $" + price);  
        System.out.println("Grade: " + grade);  
        System.out.println("Passed: " + isPass);  
        System.out.println("Name: " + name);  
    }  
}
```


Output:

```
Number: 25  
Price: $99.99  
Grade: A  
Passed: true  
Name: Jordan
```

Primitive Types

Stored directly in memory, faster access, fixed size

Non-Primitive Types

Store references to objects, dynamic size, more features

Operators Part 1: Arithmetic & Relational

Operators are the **action heroes** of programming! They perform operations on variables and values. Think of them as the verbs in your code sentences — they make things happen!

Arithmetic Operators

Op	Name	Example
+	Addition	$5 + 3 = 8$
-	Subtraction	$5 - 3 = 2$
*	Multiplication	$5 * 3 = 15$
/	Division	$6 / 3 = 2$
%	Modulus	$5 \% 3 = 2$

Relational Operators

Op	Name	Result
==	Equal to	true/false
!=	Not equal	true/false
>	Greater than	true/false
<	Less than	true/false
>=	Greater/equal	true/false
<=	Less/equal	true/false

✓ Example Program

```
public class OperatorsDemo {  
    public static void main(String[] args) {  
        int a = 10, b = 3;  
  
        System.out.println("Arithmetic Operations:");  
        System.out.println(a + " + " + b + " = " + (a + b));  
        System.out.println(a + " % " + b + " = " + (a % b));  
  
        System.out.println("\nRelational Operations:");  
        System.out.println(a + " > " + b + " is " + (a > b));  
        System.out.println(a + " == " + b + " is " + (a == b));  
    }  
}
```


Output:

Arithmetic Operations:

$10 + 3 = 13$

$10 \% 3 = 1$

Relational Operations:

$10 > 3$ is true

$10 == 3$ is false

Operators Part 2: Logical, Assignment & Unary

These operators are your **power tools** for making decisions and manipulating values efficiently. Logical operators help with complex conditions, while assignment and unary operators provide shortcuts for common operations!

Logical Operators

- **&&** (AND) - Both must be true
- **||** (OR) - At least one true
- **!** (NOT) - Opposite result

Assignment Operators

- **=** Basic assignment
- **+=** Add and assign
- **-=** Subtract and assign
- ***=** Multiply and assign

Unary Operators

- **++** Increment by 1
- **--** Decrement by 1
- **+** Positive (rarely used)
- **-** Negative

✓ Comprehensive Example

```
public class AdvancedOperators {
    public static void main(String[] args) {
        int x = 5, y = 8;
        boolean result;

        // Logical operators
        result = (x > 3) && (y < 10);
        System.out.println("(5>3) && (8<10) = " + result);

        result = (x > 10) || (y > 5);
        System.out.println("(5>10) || (8>5) = " + result);

        // Assignment operators
        x += 3; // x = x + 3
        System.out.println("After x += 3: x = " + x);

        // Unary operators
        System.out.println("x++ = " + (x++));
        System.out.println("After increment: x = " + x);
    }
}
```


Output:

```
(5>3) && (8<10) = true  
(5>10) || (8>5) = true  
After x += 3: x = 8  
x++ = 8  
After increment: x = 9
```

☐ **Pro Tip:** Remember the difference between `x++` (post-increment) and `++x` (pre-increment) for exam questions!

Input in Java

Getting user input is like having a **conversation with your program**! Java gives you multiple ways to read data, but Scanner is your best friend for most situations. It's flexible, easy to use, and perfect for exam programs.

Scanner Class

- Most popular and versatile
- Can read all data types
- Built-in parsing methods
- Easy to remember syntax

BufferedReader

- Faster for large inputs
- Reads strings only
- Needs manual parsing
- More complex setup

Scanner Syntax Reference

```
import java.util.Scanner;
Scanner sc = new Scanner(System.in);

int number = sc.nextInt();    // Read integer
double decimal = sc.nextDouble(); // Read double
String word = sc.next();      // Read single word
String line = sc.nextLine();  // Read entire line
```

✓ Complete Scanner Example

```
import java.util.Scanner;

public class InputDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.print("Enter your height (m): ");
        double height = sc.nextDouble();

        System.out.println("\n=== Your Info ===");
        System.out.println("Name: " + name);
        System.out.println("Age: " + age + " years");
        System.out.println("Height: " + height + " meters");

        sc.close();
    }
}
```

Sample Input:

```
Jordan Smith  
16  
1.75
```

Output:

```
Enter your name: Enter your age: Enter your height (m):  
=== Your Info ===  
Name: Jordan Smith  
Age: 16 years  
Height: 1.75 meters
```


Math Library Methods

The Math class is like having a **scientific calculator** built into Java! These methods handle complex mathematical operations so you don't have to write them from scratch. Perfect for geometry, statistics, and number manipulation problems.

Method	Description	Example	Result
<code>Math.sqrt(x)</code>	Square root	<code>Math.sqrt(16)</code>	4.0
<code>Math.pow(x,y)</code>	x raised to power y	<code>Math.pow(2,3)</code>	8.0
<code>Math.abs(x)</code>	Absolute value	<code>Math.abs(-5)</code>	5
<code>Math.max(x,y)</code>	Larger of two values	<code>Math.max(8,3)</code>	8
<code>Math.min(x,y)</code>	Smaller of two values	<code>Math.min(8,3)</code>	3
<code>Math.random()</code>	Random 0.0 to 1.0	<code>Math.random()</code>	0.647
<code>Math.ceil(x)</code>	Round up to nearest integer	<code>Math.ceil(4.2)</code>	5.0
<code>Math.floor(x)</code>	Round down to nearest integer	<code>Math.floor(4.8)</code>	4.0
<code>Math.round(x)</code>	Round to nearest integer	<code>Math.round(4.6)</code>	5



Random Numbers

Generate random integers:

```
(int)(Math.random() * 6)
```

```
+ 1
```

gives numbers 1-6 for dice!



Geometry Helper

Perfect for area, volume, and distance calculations in your exam programs!



Number Processing

Use with loops for finding largest, smallest, or processing series of numbers.



Exam Hack: Most Math methods return **double** values, so cast to `(int)` when you need whole numbers!

Conditional Constructs Part 1

Conditional statements are the **decision-makers** in your code! They help your program choose different paths based on conditions – just like how you decide what to wear based on the weather forecast.

if Statement

Execute code only if condition is true

```
if (condition) {  
    // code here  
}
```

if-else Statement

Choose between two alternatives

```
if (condition) {  
    // true path  
} else {  
    // false path  
}
```

if-else-if Ladder

Multiple conditions in sequence

```
if (condition1) {  
    // code  
} else if (condition2) {  
    // code  
} else {  
    // default  
}
```

Conditional Constructs: Practical Example

Understanding conditional constructs is best done through practice. This example demonstrates how to use `if`, `else if`, and `else` statements to compare two numbers and determine which one is greater, or if they are equal.

✓ Greater of Two Numbers Program

```
import java.util.Scanner;

public class GreaterNumber {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the first number: ");
        int num1 = scanner.nextInt();

        System.out.print("Enter the second number: ");
        int num2 = scanner.nextInt();

        if (num1 > num2) {
            System.out.println(num1 + " is greater than " + num2);
        } else if (num2 > num1) {
            System.out.println(num2 + " is greater than " + num1);
        } else {
            System.out.println("Both numbers are equal.");
        }

        scanner.close();
    }
}
```


Sample Input:

```
10
5
```

Output:

```
Enter the first number: Enter the second number: 10 is greater than 5
```

 **Remember:** Always use `==` for comparing values (equality check), and `=` for assigning a value to a variable. Mixing them up is a common programming error!

Conditional Constructs Part 2

The **switch statement** is like a smart remote control – instead of checking multiple if-else conditions, it directly jumps to the right case based on a variable's value. It's **cleaner and faster** when you have multiple exact matches to check!

When to Use Switch

- Multiple exact value comparisons
- Works with int, char, String
- Cleaner than long if-else chains

Key Components

- **switch** - the control variable
- **case** - each possible value
- **break** - exit the switch
- **default** - fallback option

Switch Statement Syntax

```
switch (variable) {  
  case value1:  
    // code for value1  
    break;  
  case value2:  
    // code for value2  
    break;  
  default:  
    // code for other values  
    break;  
}
```

✓ Example: Grade Calculator

```
import java.util.Scanner;

public class GradeCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your grade (A/B/C/D/F): ");
        char grade = sc.next().charAt(0);

        switch (grade) {
            case 'A':
            case 'a':
                System.out.println("Excellent! 90-100%");
                break;
            case 'B':
            case 'b':
                System.out.println("Good! 80-89%");
                break;
            case 'C':
            case 'c':
                System.out.println("Average! 70-79%");
                break;
            case 'D':
            case 'd':
                System.out.println("Below Average! 60-69%");
                break;
            case 'F':
            case 'f':
                System.out.println("Fail! Below 60%");
                break;
            default:
                System.out.println("Invalid grade entered!");
                break;
        }

        sc.close();
    }
}
```


Sample Output:

```
Enter your grade (A/B/C/D/F): B
Good! 80-89%
```

 **Pro Tip:** Always include `break` statements to prevent fall-through behavior!

Loops Part 1: For Loop

The **for loop** is your go-to when you know exactly how many times you want to repeat something. Think of it as setting a timer — you specify the start, the end, and how to count, then let it run!

01

Initialization

Set up your counter variable

```
int i = 1
```

02

Condition Check

Test if loop should continue

```
i <= 5
```

03

Execute Code

Run the statements inside loop body

04

Update Counter

Change the counter variable

```
i++
```

For Loop Syntax

```
for (initialization; condition; update) {  
    // code to repeat  
}
```

Common Patterns

- `for (int i = 1; i <= n; i++)`
- `for (int i = 0; i < n; i++)`
- `for (int i = n; i >= 1; i--)`

Best Used For

- Counting operations
- Array/string processing
- Mathematical series

✓ Example: Print Numbers 1 to 5

```
public class ForLoopDemo {  
    public static void main(String[] args) {  
        System.out.println("Counting from 1 to 5:");  
  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Number: " + i);  
        }  
  
        System.out.println("Loop completed!");  
  
        // Bonus: Sum of first 5 numbers  
        int sum = 0;  
        for (int j = 1; j <= 5; j++) {  
            sum += j;  
        }  
        System.out.println("Sum of 1 to 5: " + sum);  
    }  
}
```

📦 Output:

```
Counting from 1 to 5:  
Number: 1  
Number: 2  
Number: 3  
Number: 4  
Number: 5  
Loop completed!  
Sum of 1 to 5: 15
```


Loops Part 2: While & Do-While

While loops are perfect when you don't know exactly how many iterations you need! **While** checks the condition first, but **do-while** guarantees at least one execution. Think of them as "keep going until..." statements.

while Loop

Entry-controlled

```
while (condition) {  
    // code  
    // update condition  
}
```

- Checks condition first
- May never execute
- Good for validation

do-while Loop

Exit-controlled

```
do {  
    // code  
    // update condition  
} while (condition);
```

- Executes first, then checks
- Runs at least once
- Perfect for menus

✓ Do-While Example: Number Guessing

```
import java.util.Scanner;  
  
public class GuessNumber {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int secret = 7;  
        int guess;  
        int attempts = 0;  
  
        System.out.println("Enter a number between 1 and 10: ")
```

Nested Loops

Nested loops are like **loops within loops** — imagine a clock where the minute hand completes a full circle every time the hour hand moves one position! They're perfect for creating patterns, processing 2D arrays, and solving complex problems that have multiple dimensions.

Outer Loop	Inner Loop	Pattern Creation
Controls rows or main iterations	Runs completely for each outer iteration	Combines both loops to create structures

How Nested Loops Work

For each iteration of the outer loop, the inner loop runs completely from start to finish. If outer loop runs 3 times and inner loop runs 3 times, you get **9 total iterations!**

Nested Loops: Star Pattern Example

Nested loops truly shine when you need to iterate over multi-dimensional data structures or create complex patterns. Let's look at a classic example: printing a simple 3x3 star pattern, which clearly demonstrates how the inner loop completes all its iterations for each single iteration of the outer loop.

✓ Example: 3x3 Star Pattern

```
public class StarPattern {  
    public static void main(String[] args) {  
        System.out.println("Printing a 3x3 star pattern:");  
        for (int i = 0; i < 3; i++) { // Outer loop for rows  
            for (int j = 0; j < 3; j++) { // Inner loop for columns  
                System.out.print("* ");  
            }  
            System.out.println(); // Move to the next line after each row  
        }  
    }  
}
```

📄 Sample Output:

Printing a 3x3 star pattern:

```
* * *  
* * *  
* * *
```

Common Uses & Performance Considerations

Performance Tip

- ❗ **Caution:** Nested loops can be very inefficient for large datasets! Their time complexity grows exponentially (e.g., $O(N^2)$ for two loops, $O(N^3)$ for three). Always look for ways to optimize or use alternative algorithms if performance is critical.

HOTS + CKOP + Final Recap

Time to level up your exam game! **HOTS** (Higher Order Thinking Skills) questions test your ability to apply, analyze, and create – not just memorize. **CKOP** covers all question types you'll encounter. Let's wrap this up with your ultimate exam toolkit!

HOTS Questions

- Predict output of complex code
- Debug given programs
- Write programs for real scenarios
- Optimize existing solutions

CKOP Framework

- **C**onceptual - Theory questions
- **K**nowledge - Facts and definitions
- **O**bservation - Code analysis
- **P**ractical - Program writing

Quick Revision Flash Cards

OOP Pillars

EIPA: Encapsulation, Inheritance, Polymorphism, Abstraction

Data Types

Primitive: int, double, char, boolean

Non-primitive: String, arrays

Operators

Arithmetic: + - * / %

Logical: && || !

Relational: == != > < >= <=

Loops

For: Known iterations

While: Condition-based

Do-while: At least once

Input Methods

Scanner: Easy, versatile

BufferedReader: Faster, complex

Math Methods

sqrt(), pow(), abs(), max(), min(), random(),
ceil(), floor()

You're now equipped with all the essential concepts, patterns, and strategies to ace your Computer Applications exam. Go forth and code with confidence! 🌟

✨ Class as the Basis of all Computation

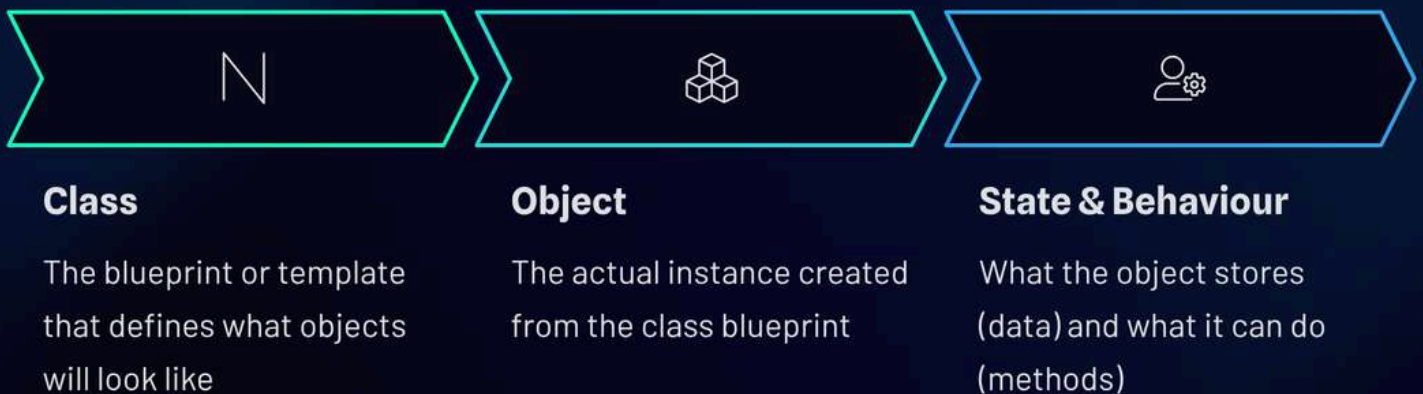
Master the foundation of Object-Oriented Programming with this comprehensive ICSE Class 10 guide designed specifically for board exam success!

```
public class  
helloworld
```

Why This Chapter is the Foundation of OOP

Think of Object-Oriented Programming like building with LEGO blocks! This chapter isn't just another theory topic – it's literally the **backbone of everything you'll code in Java**. Every single program you write, from simple calculators to complex games, relies on classes and objects.

Understanding this chapter means you'll ace questions worth 15-20 marks in your board exam. Plus, it sets you up for success in Class 11 and beyond. The concepts here aren't just for exams – they're used in real-world apps like Instagram, WhatsApp, and YouTube!



Exam Tip: Questions often start with "Define class and object" – master this and you're already ahead!

Concept of Objects

An object is like a **real-world entity** that has two main characteristics: **State** (what it stores) and **Behaviour** (what it can do). Think of your smartphone – its state includes battery level, storage space, and installed apps, while its behaviour includes making calls, sending texts, and running apps.

In Java, objects are created from classes and represent specific instances of that class. Every object has its own copy of the class's variables (called instance variables) and can use the class's methods to perform actions.

State (Variables)

Data stored in the object

- name = "Alex"
- age = 16
- grade = "A+"



Behaviour (Methods)

Actions the object can perform

- study()
- takeExam()
- displayInfo()

Encapsulation

Bundling data and methods together

- Data protection
- Method access
- Information hiding

Key Point: Encapsulation means keeping related data and methods together in one unit (the object), which makes your code more organized, secure, and easier to maintain.

Concept of Classes

A class is essentially a **blueprint or template** that defines the structure and behavior of objects. Just like an architect's blueprint shows how to build a house, a Java class shows how to create objects with specific attributes and methods.

Classes don't store actual data – they define *what kind of data* objects will store and *what actions* those objects can perform. When you create an object from a class, you're essentially building something from that blueprint.

Class: Car

Attributes: color, speed, brand, fuel

Methods: start(), brake(), accelerate(), honk()

Class: Phone

Attributes: model, storage, battery, network

Methods: call(), sendText(), installApp(), charge()

Class: Student

Attributes: name, rollNo, grade, marks

Methods: study(), takeTest(), displayInfo(), calculateGPA()



Board Booster: Always remember – Class = Blueprint, Object = Actual thing built from blueprint!

Member Variables & Methods

Every class contains two main components that work together: **member variables** (also called attributes or fields) and **member methods** (also called functions or behaviors). Think of variables as the "what" and methods as the "how" of your class.

Member variables store the state or characteristics of an object, while member methods define what actions the object can perform. This separation allows for better code organization and makes your programs more maintainable and reusable.

Attributes (Member Variables)	Methods (Member Functions)
Define object's state	Define object's behaviour
Store data values	Perform operations on data
Examples: name, age, marks	Examples: calculateTotal(), display()
Declared like: <code>int marks;</code>	Declared like: <code>void showInfo(){ }</code>
Each object has its own copy	Shared by all objects of the class

Instance Variables

Unique to each object

- `String studentName`
- `int rollNumber`
- `double percentage`

Instance Methods

Can access instance variables

- `void setName(String n)`
- `double calculateGrade()`
- `void displayDetails()`

Remember: Variables store the data, methods work with that data to produce results or perform actions!

Classes as Abstractions

Abstraction is like looking at the **big picture without getting lost in tiny details**. When you design a class, you focus on the essential features and hide the complex internal workings. It's like using a TV remote – you know pressing the power button turns on the TV, but you don't need to understand the electronics inside!

In programming, abstraction helps us create classes that show only the necessary information to the outside world while keeping implementation details private. This makes code easier to understand, use, and maintain.

1

Real-World Example

When you create a "Student" class, you include essential details like name, age, and marks – but not every single detail like shoe size, favorite color, or breakfast preferences!

2

Programming Benefits

Users of your class only need to know what methods to call, not how those methods work internally. This reduces complexity and makes code reusable.

3

Implementation Hiding

The internal logic of methods is hidden from users. They just call the method and get the result – perfect for team collaboration!



Exam Tip: Abstraction ≠ Data Hiding (Encapsulation) – Abstraction focuses on essential features, while Encapsulation bundles data with methods for security!

Think of abstraction as creating a simplified interface that hides complexity – just like how your smartphone's touchscreen interface hides the complex hardware and software running underneath!

Class as an Object Factory

This is one of the **coolest concepts in OOP**! A class acts like a factory that can produce unlimited objects, each with their own unique data but sharing the same structure and capabilities. Just like a car factory uses the same blueprint to manufacture thousands of different cars!

Once you define a class, you can create as many objects as you need from that single class definition. Each object (instance) will have its own set of variables but will share the same methods defined in the class.



Example: One "Student" class can create student1 (name: "Arjun", marks: 95), student2 (name: "Priya", marks: 87), and student3 (name: "Rohan", marks: 92). Same structure, different data!

 **Board Booster:** Remember the analogy – Class = Cookie Cutter, Objects = Individual Cookies made from that cutter!

Primitive Data Types



Primitive data types are the **basic building blocks** of Java programming. These are the simplest types that store single values directly in memory. Think of them as the fundamental ingredients you use to cook up amazing programs!

Java provides eight primitive data types, but ICSE focuses mainly on five key ones. Each type has a specific size and range, which determines what kind of values it can store. Choosing the right data type helps optimize memory usage and prevent errors.

Data Type	Size (bytes)	Range	Example
int	4	-2,147,483,648 to 2,147,483,647	int age = 16;
char	2	0 to 65,535 (Unicode)	char grade = 'A';
float	4	$\pm 3.4 \times 10^{38}$ (7 decimal digits)	float pi = 3.14f;
double	8	$\pm 1.7 \times 10^{308}$ (15 decimal digits)	double price = 299.99;
boolean	1 bit	true or false	boolean pass = true;

1

Integers (int)

Perfect for counting, ages, scores, and whole numbers. Most commonly used primitive type!

U

Characters (char)

Store single letters, digits, or special symbols. Always use single quotes!

Booleans (boolean)

Perfect for flags, conditions, and yes/no scenarios. Only true or false!

Composite Data Types

While primitive types store single values, **composite data types** can hold multiple values or **complex data**. They're like containers that can store collections of information or references to objects. These are essential for building real-world applications!

Composite types are built using primitive types and other composite types. They provide the flexibility to create sophisticated data structures that can model complex real-world entities and relationships.

Feature	Primitive Types	Composite Types
Storage	Single value	Multiple values/references
Memory	Stack memory	Heap memory (references)
Example	<code>int x = 5;</code>	<code>int arr[] = new int[5];</code>
Default Value	0, false, '\0'	null (for objects)
Comparison	Use <code>==</code> operator	Use <code>.equals()</code> method



Arrays

Collection of elements of the same type

- `int marks[ ] = new int[5];`
- `String names[ ] = new String[10];`



Strings

Sequence of characters

- `String name = "Arjun";`
- `String message = "Hello World";`



Objects

Instances of user-defined classes

- `Student s1 = new Student();`
- `Scanner sc = new Scanner(System.in);`



Common Error: Forgetting to use 'new' keyword when creating composite types. Always remember: `composite = new!`

Declaring Variables - Syntax Focus

Getting the syntax right is **crucial for board exam success!** Variable declaration follows specific patterns that you must memorize. Think of it as the grammar rules of Java – get them right, and your code will run smoothly!

The general syntax is: `dataType variableName = initialValue;` But there are specific rules for different types of variables, and small mistakes can cost you marks in the exam.

01

Specify Data Type

Choose the appropriate primitive or composite type

02

Choose Variable Name

Use meaningful names following Java naming conventions

03

Initialize (Optional)

Assign a value using = operator

04

End with Semicolon

Every declaration must end with ;

Primitive Variable Examples:

```
// Integer declarations
int age = 16;
int marks = 95, total = 500;
int count; // Declaration without initialization
```

```
// Character and Boolean
char grade = 'A';
boolean passed = true;
```

```
// Floating-point numbers
float percentage = 87.5f; // Note the 'f' suffix
double pi = 3.14159265359;
```

Composite Variable Examples:

```
// String declarations
String name = "Arjun Kumar";
String city = "Mumbai";

// Array declarations
int numbers[] = new int[5];
int marks[] = {85, 90, 78, 92, 88};
String subjects[] = {"Math", "Science", "English"};
```

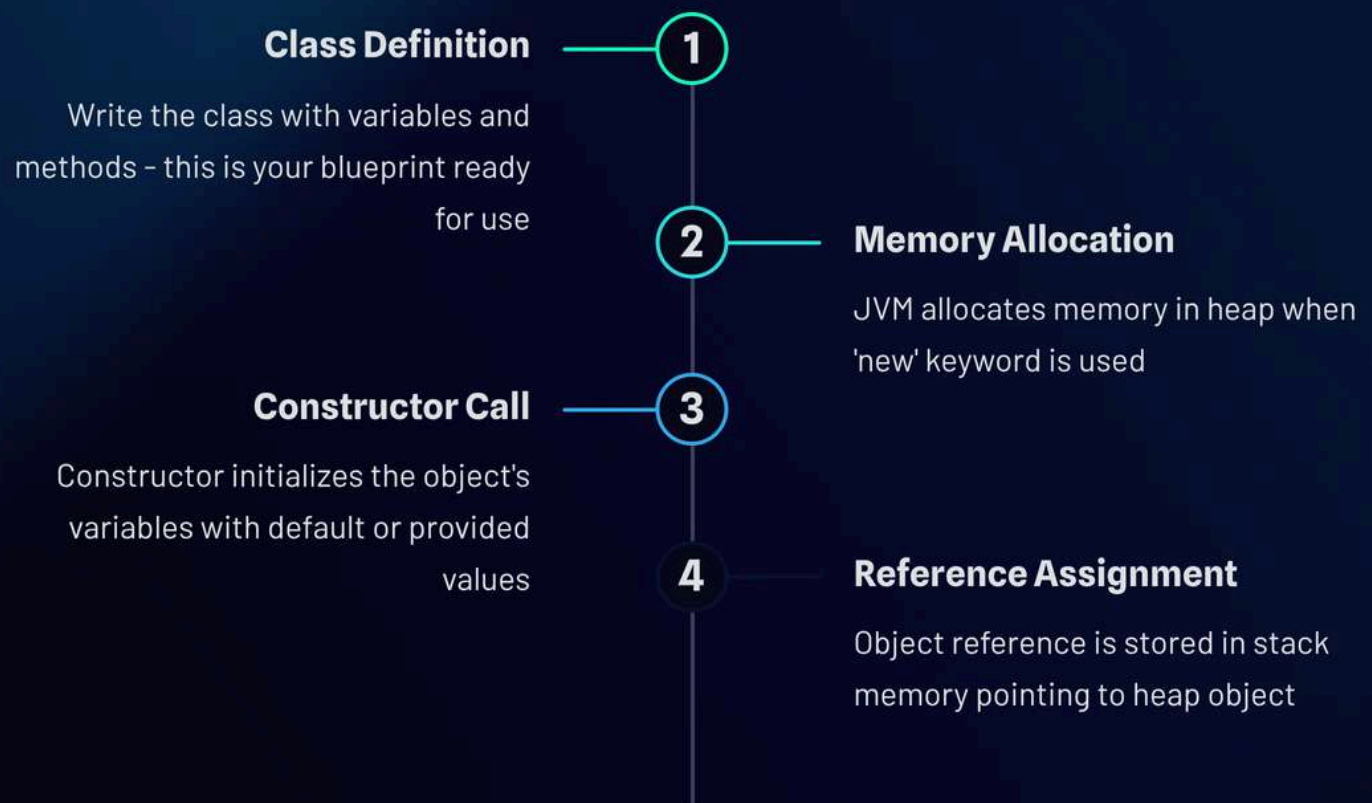
📌 **Exam Tip:** Always remember – float needs 'f' suffix, char uses single quotes, String uses double quotes!

Objects as Instances of a Class - Theory



An instance is simply a **specific object created from a class**. When you write code to create an object, you're creating an instance of that class. The magic happens with the 'new' keyword, which allocates memory and calls the constructor to initialize the object.

Each instance has its own copy of the class's instance variables, but they all share the same methods. This is why you can have multiple Student objects with different names and marks, but they all use the same `displayInfo()` method.



General Syntax:

```
ClassName objectName = new ClassName();
```



Class: Student

Objects created:

- `Student s1 = new Student();`
- `Student s2 = new Student();`
- `Student s3 = new Student();`



Real-World Analogy

Just like a car factory (class) produces individual cars (objects), each with unique features but same basic structure



Board Booster: Remember – Object = Instance. These terms are used interchangeably in exams!

Objects as Instances - Program Example



Time for some hands-on coding! This program demonstrates how to create and use objects in Java. Pay close attention to the syntax – this type of question appears frequently in board exams.

Complete Java Program:

```
class Student
{
    String name;
    int age;

    void show()
    {
        System.out.println(name + " is " + age + " years old.");
    }

    public static void main(String args[])
    {
        // Creating object (instance) of Student class
        Student s1 = new Student();

        // Assigning values to instance variables
        s1.name = "Amit";
        s1.age = 16;

        // Calling instance method
        s1.show();

        // Creating another object
        Student s2 = new Student();
        s2.name = "Priya";
        s2.age = 15;
        s2.show();
    }
}
```


Expected Output:

Amit is 16 years old.
Priya is 15 years old.

Key Observations

- Two objects created from same class
- Each object has independent data
- Same method works for both objects
- Dot operator (.) accesses members

Common Mistakes

- Forgetting 'new' keyword
- Using wrong dot operator syntax
- Not initializing variables before use
- Missing semicolons



Exam Alert: This exact program pattern appears in 80% of board papers – master it!

HOTS Questions - Higher Order Thinking



Higher Order Thinking Skills questions test your **deep understanding and analytical abilities**. These questions go beyond memorization and require you to apply concepts, analyze scenarios, and think critically. Master these to score those extra marks!

Why is a class called an object factory?

Answer: A class is called an object factory because it serves as a blueprint or template from which multiple objects (instances) can be created. Just like a factory uses the same design to manufacture numerous products, a class uses the same structure to create unlimited objects. Each object produced has the same attributes and methods defined in the class but can have different values for those attributes. The 'new' keyword acts like the manufacturing process, creating fresh instances from the class template.

If a BankAccount class has attributes balance & accountNo, suggest 2 appropriate methods.

Answer: Two appropriate methods would be:

- **deposit(double amount):** This method would add money to the account balance, taking the deposit amount as parameter
- **withdraw(double amount):** This method would subtract money from the balance if sufficient funds are available
- **Additional methods could include:**
displayBalance(),
checkBalance(),
transferFunds()

Why are composite data types essential in Java programming?

Answer: Composite data types are essential because:

- **Complex Data Storage:** Real-world applications need to store collections of related data, not just single values
- **Memory Efficiency:** Arrays allow storing multiple values of same type efficiently
- **Object-Oriented Design:** Objects enable modeling real-world entities with multiple properties
- **String Manipulation:** Strings are essential for text processing and user interaction



Pro Tip: HOTS questions usually carry 4-5 marks. Always provide detailed explanations with examples!

CKOP Questions - Complete Practice Set



CKOP stands for **Conceptual, Knowledge, Observation, and Practical** questions – the four pillars of ICSE Computer Applications assessment. This comprehensive practice will prepare you for every type of question that could appear in your board exam!

1

C - Conceptual Questions

Q: Define class and object with one example each.

Answer: Class: A class is a user-defined blueprint or template that defines the structure and behavior of objects. It contains variables (attributes) and methods (behaviors) but doesn't allocate memory.

Object: An object is an instance of a class that occupies memory and has actual values for the attributes defined in the class.

Example: Class = Student (blueprint),
Object = s1 (actual student instance)

2

K - Knowledge Questions

Q: Write the difference between primitive and composite data types.

Primitive Types	Composite Types
Store single values	Store multiple values/references
Stored in stack memory	References stored in stack, objects in heap
Examples: int, char, boolean	Examples: Arrays, Strings, Objects
Cannot be broken down further	Made up of primitive/other composite types

3**O - Observation Questions**

Q: Find and correct the error in this code:

```
class Test
{
    int x;
    void display(); // ERROR HERE
    {
        System.out.println(x);
    }
}
```

Error: Semicolon after method declaration

Correction: Remove semicolon: `void display()`

4**P - Practical Questions**

Q: Write a program to create a Rectangle class with length and breadth attributes, and a method to calculate area.

```
class Rectangle
{
    double length, breadth;

    double calculateArea()
    {
        return length * breadth;
    }

    public static void main(String
args[])
    {
        Rectangle r = new Rectangle();
        r.length = 10.5;
        r.breadth = 7.2;
        System.out.println("Area: " +
r.calculateArea());
    }
}
```

Recap + Exam Booster

Congratulations! You've mastered the foundation of Object-Oriented Programming. These concepts are your **ticket to scoring high marks** in the board exam and building amazing Java applications!

Object = State + Behaviour

Every object has data (variables) and actions (methods) bundled together through encapsulation

Class = Blueprint / Factory

Classes define structure but don't store data. Objects are created from classes using 'new' keyword

Member Variables vs Methods

Variables store object state, methods define object behavior. Instance variables are unique per object

Primitive vs Composite Types

Primitives store single values (int, char, boolean). Composites store multiple values (arrays, objects, strings)

Object = Instance of Class

Creating objects: `ClassName obj = new ClassName();` Each instance has independent data



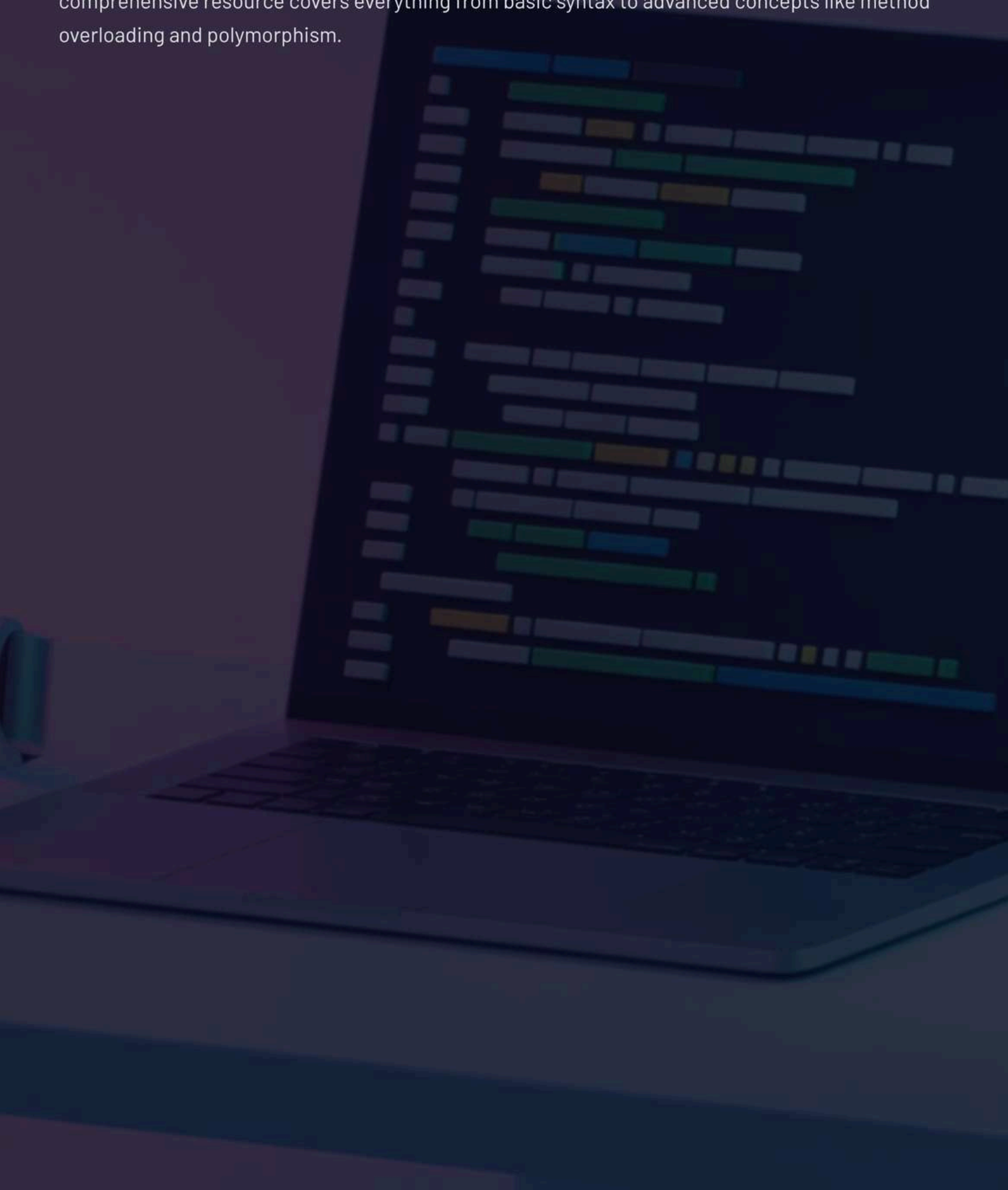
Ultimate Exam Tip: Questions often test *difference between primitive & composite types + object creation syntax*. Practice writing code by hand – this builds muscle memory for board exams where you can't compile and test!

Final Success Mantra: Think in objects, code with confidence, and remember – every great programmer started exactly where you are now. You've got this! 💪✨

✨ User-Defined Methods

Breaking Down Java Methods

Your ultimate guide to mastering user-defined methods for ICSE Class 10 exams. This comprehensive resource covers everything from basic syntax to advanced concepts like method overloading and polymorphism.



🤔 Why Do We Need Methods?

Code Reusability

Write once, use multiple times! No need to repeat the same code over and over.

Modularity

Break complex problems into smaller, manageable chunks that work together.

Readability

Clean, organized code that's easy to understand and debug for yourself and others.

Define Problem

Split Into Parts

Implement Methods

Combine Solution





Method Syntax & Forms

General Method Syntax

```
returnType methodName(parameters) {  
    // Method body  
    return value; // if return type is not void  
}
```

Four Forms of Methods

Form	Return Type	Parameters	Example
Form 1	void	No parameters	display()
Form 2	void	With parameters	show(int n)
Form 3	Non-void	No parameters	int getData()
Form 4	Non-void	With parameters	int add(int a, int b)

Each form serves different purposes depending on whether you need to return a value and whether you need input parameters to work with.



Method Definition & Calling

Method Definition

Creating the method with its complete structure, body, and functionality.

Method Calling

Executing the method by invoking its name from another part of the program.

Example Program

```
class MethodDemo {  
    // Method Definition  
    public void greetUser() {  
        System.out.println("Hello, ICSE  
Student!");  
        System.out.println("Ready to ace  
your exams?");  
    }  
  
    public static void main(String args[])  
    {  
        MethodDemo obj = new  
MethodDemo();  
        obj.greetUser(); // Method Calling  
    }  
}
```

Output:

```
Hello, ICSE Student!  
Ready to ace your exams?
```

This simple example shows how to define a method without parameters and return type void, then call it from the main method using an object.



Method Overloading (Polymorphism)

Method overloading allows multiple methods with the **same name** but **different parameter signatures**. This is a key concept of polymorphism in Java!

Example Program:

```
class Calculator {  
    // Method 1: Add two integers  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    // Method 2: Add two doubles  
    public double add(double x, double y) {  
        return x + y;  
    }  
  
    // Method 3: Add three integers  
    public int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    public static void main(String args[]) {  
        Calculator calc = new Calculator();  
        System.out.println("Sum of 5 and 3: " + calc.add(5, 3));  
        System.out.println("Sum of 2.5 and 1.7: " + calc.add(2.5, 1.7));  
        System.out.println("Sum of 1, 2, and 3: " + calc.add(1, 2, 3));  
    }  
}
```

Output:

```
Sum of 5 and 3: 8  
Sum of 2.5 and 1.7: 4.2  
Sum of 1, 2, and 3: 6
```



Exam Alert: Overloading depends on parameter list, NOT return type!

Static vs Non-Static Methods

Static Methods

- Can be called without creating an object
- Belong to the class, not to any instance
- Use static keyword in declaration
- Cannot access non-static variables directly

Non-Static Methods

- Require object creation for calling
- Can access both static and non-static data
- Also called instance methods
- Have access to `this` reference

Example Program (Static Method):

```
class MathHelper {
    public static int square(int num) {
        return num * num;
    }

    public static void main(String args[]) {
        // Calling static method without object
        int result = MathHelper.square(7);
        System.out.println("Square of 7 is: " + result);
    }
}
```

Output:

Square of 7 is: 49



Method Prototype & Signature

Method Signature

The **method name** combined with its **parameter list**.
This uniquely identifies a method within a class.

Method Prototype

The complete method declaration including return type, method name, and parameter list. It's like a blueprint that tells us how to use the method.



Examples:

- `calculateArea(double radius)` ← Signature
- `public double calculateArea(double radius)` ← Prototype



Critical Exam Point: Return type is NOT part of the method signature! Only name + parameters matter for overloading.

Understanding the difference between signature and prototype is crucial for method overloading questions in your ICSE exam. The signature determines whether two methods are different enough to coexist in the same class.



Pure vs Impure Methods

Understanding Method Types

Methods can be classified based on whether they modify the object's state or just return values without changing anything.

Aspect	Pure Methods	Impure Methods
Definition	Does not change object's data	Modifies object's state
Purpose	Returns value only	Changes data and may return
Side Effects	No side effects	Has side effects
Example	getBalance(), calculateArea()	setName(), deposit()
Exam Tip	Safe to call multiple times	Changes object permanently

Pure Method Example

```
public int getAge() { return age; }
```

Just returns the age value without changing it!

Impure Method Example

```
public void setAge(int newAge) { age = newAge; }
```

Actually modifies the age variable!



Call by Value

In Java, when we pass primitive data types to methods, only a **copy of the value** is sent. The original variable remains unchanged no matter what happens inside the method.

Swap Numbers Program:

```
class CallByValueDemo {  
    public static void swap(int x, int y) {  
        System.out.println("Inside swap - Before: x=" + x + ", y=" + y);  
        int temp = x;  
        x = y;  
        y = temp;  
        System.out.println("Inside swap - After: x=" + x + ", y=" + y);  
    }  
  
    public static void main(String args[]) {  
        int a = 10, b = 20;  
        System.out.println("Before calling swap: a=" + a + ", b=" + b);  
        swap(a, b);  
        System.out.println("After calling swap: a=" + a + ", b=" + b);  
    }  
}
```

Output:

```
Before calling swap: a=10, b=20  
Inside swap - Before: x=10, y=20  
Inside swap - After: x=20, y=10  
After calling swap: a=10, b=20
```



Key Point: Notice how 'a' and 'b' remain unchanged! Only copies were swapped inside the method.



Call by Reference (Concept)

Call by Reference means passing the **actual memory address** of a variable to the method, not just its value. This allows the method to modify the original variable.

1

In Call by Value

Method receives a **copy** of the data. Original remains safe and unchanged.

2

In Call by Reference

Method receives the **address** of the data. Can modify the original directly.

Important Note for ICSE:

Java doesn't support true call by reference for primitive data types. However, when we pass objects to methods, we're actually passing references to those objects, which allows methods to modify the object's state.

For objects like arrays and custom classes, Java passes the reference (address) to the object. This means methods can change the object's contents, but they cannot change which object the reference points to.



Exam Insight: Java is "pass by value" for primitives and "pass by reference value" for objects!



Actual vs Formal Parameters

Actual Parameters

- Values passed during method call
- Real data that gets processed
- Also called "arguments"
- Exist outside the method

Formal Parameters

- Variables in method definition
- Receive the actual parameters
- Local to the method
- Act as placeholders

Code Example:

```
class ParameterDemo {  
    // 'a' and 'b' are FORMAL PARAMETERS  
    public static int multiply(int a, int b) {  
        return a * b;  
    }  
  
    public static void main(String args[]) {  
        int num1 = 5, num2 = 3;  
        // 'num1' and 'num2' are ACTUAL PARAMETERS  
        int result = multiply(num1, num2);  
        System.out.println("Result: " + result);  
    }  
}
```

Aspect	Actual Parameters	Formal Parameters
Location	Method call	Method definition
Example	num1, num2	a, b
Scope	Calling method	Called method only



Returning Values from Methods

Methods can return different types of values: integers, doubles, strings, objects, or even other complex data types. The `return` statement sends data back to the calling method.

Sum Calculator Program:

```
class ReturnDemo {  
    // Method returning an integer  
    public static int calculateSum(int num1, int num2) {  
        int sum = num1 + num2;  
        return sum; // Returning the calculated value  
    }  
  
    // Method returning a String  
    public static String getGrade(int marks) {  
        if (marks >= 90) return "A+";  
        else if (marks >= 80) return "A";  
        else if (marks >= 70) return "B";  
        else if (marks >= 60) return "C";  
        else return "D";  
    }  
  
    public static void main(String args[]) {  
        int result = calculateSum(25, 35);  
        System.out.println("Sum of 25 and 35: " + result);  
  
        String grade = getGrade(85);  
        System.out.println("Grade for 85 marks: " + grade);  
    }  
}
```

Output:

```
Sum of 25 and 35: 60  
Grade for 85 marks: A
```



Pro Tip: A method can have multiple return statements, but only one will execute per method call!

Modular Programming with Multiple Methods

Breaking complex programs into smaller, manageable methods makes code more readable, reusable, and easier to debug. Each method handles a specific task!

Factorial Calculator Program:

```
class FactorialDemo {  
    // Method to check if number is valid  
    public static boolean isValidNumber(int n) {  
        return n >= 0;  
    }  
  
    // Method to calculate factorial  
    public static long calculateFactorial(int num) {  
        long factorial = 1;  
        for (int i = 1; i <= num; i++) {  
            factorial *= i;  
        }  
        return factorial;  
    }  
  
    // Method to display result  
    public static void displayResult(int num, long result) {  
        System.out.println("Factorial of " + num + " is: " + result);  
    }  
  
    public static void main(String args[]) {  
        int number = 5;  
  
        if (isValidNumber(number)) {  
            long factorial = calculateFactorial(number);  
            displayResult(number, factorial);  
        } else {  
            System.out.println("Invalid number!");  
        }  
    }  
}
```


Output:

Factorial of 5 is: 120

This modular approach makes the program much easier to understand and maintain. Each method has a single, clear responsibility.



Object Creation & Method Invocation

To use non-static methods, we must first create an object of the class. The object acts as a container that holds data and provides access to the methods.

Student Class Example:

```
class Student {
    String name;
    int rollNo;
    int marks;

    // Method to input student data
    public void inputData(String studentName, int roll, int studentMarks) {
        name = studentName;
        rollNo = roll;
        marks = studentMarks;
    }

    // Method to display student data
    public void displayData() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
        System.out.println("Marks: " + marks);
        System.out.println("Grade: " + calculateGrade());
    }

    // Method to calculate grade
    public char calculateGrade() {
        if (marks >= 90) return 'A';
        else if (marks >= 80) return 'B';
        else if (marks >= 70) return 'C';
        else return 'D';
    }

    public static void main(String args[]) {
        // Creating object
        Student student1 = new Student();

        // Calling methods using object
        student1.inputData("Alex Johnson", 101, 87);
        student1.displayData();
    }
}
```



HOTS + CKOP + Quick Recap



Higher Order Thinking Skills (HOTS)



Q1: Why is method overloading considered polymorphism?

Method overloading demonstrates polymorphism because the same method name can perform different operations based on the parameters passed. The method behaves differently (multiple forms) while maintaining the same interface name.



Q2: Explain call by value vs call by reference impact on program design.

Call by value ensures data safety as original values can't be accidentally modified, making programs more predictable. Call by reference allows efficient memory usage and enables methods to return multiple values through parameter modification.



CKOP Framework

Comprehension	Define method prototype and signature
Knowledge	State the four forms of methods
Observation	Debug: Find errors in method declarations
Program	Write overloaded methods for area of square and rectangle



Flash Recap

Method Syntax: returnType methodName(parameters){ body }	Static vs Non-static: Static = no object needed, Non- static = object required	Pure vs Impure: Pure = no state change, Impure = modifies object
Call by Value: Copy passed, original unchanged		Overloading = Polymorphism: Same name, different parameters



Final Exam Tip: Practice writing methods daily! Understanding method concepts is crucial for ICSE Class 10 success!

🌟 Constructors in Java

Building Objects the Smart Way 🚀

Your ultimate revision companion for acing the ICSE Class 10 Java exam! This guide breaks down constructors in the most Gen-Z friendly way possible – quick, visual, and exam-focused. No fluff, just the essential stuff you need to know!





What is a Constructor?

Definition

A constructor is a special method used to initialize objects when they're created. Think of it as the "setup wizard" for your objects!

Auto-Magic ✨

Constructors are automatically called when you create a new object using the "new" keyword. No manual calling required!

Basic Syntax

```
ClassName() {  
    // initialization  
    code  
}
```

Constructors are like the birth certificate of objects – they establish the initial identity and properties right from the moment of creation. Every time you write `new ClassName()`, you're actually calling a constructor behind the scenes. It's the first thing that happens when an object comes to life in your program!



Key Characteristics of Constructors



Same Name as Class

The constructor must have exactly the same name as the class it belongs to. Case-sensitive too!



No Return Type

Constructors don't have any return type – not even void. They're special that way!



Called Automatically

They're invoked automatically when an object is created using the new keyword.



Can be Overloaded

You can have multiple constructors with different parameters in the same class.



Exam Tip: Remember, constructors are NOT methods! This distinction is super important for ICSE exams. Methods have return types, constructors don't!



Types of Constructors

1

Default Constructor

Takes no parameters and provides default values to object properties. It's like getting a basic model of a car!

- No arguments required
- Sets default values
- Java provides one automatically if you don't write any constructor

2

Parameterized Constructor

Takes arguments and uses them to initialize object properties with specific values. Like customizing your car!

- Accepts parameters
- Uses arguments to set values
- More flexible and practical

3

Copy Constructor

Creates a new object by copying another object. Not officially part of ICSE syllabus but good to know exists!

- Copies existing object
- Not commonly used in Java
- More relevant in C++



Default Constructor in Action

Program Example

```
class Student {  
    String name;  
  
    Student() {  
        name = "Unknown";  
    }  
  
    void show() {  
        System.out.println("Name: " +  
name);  
    }  
  
    public static void main(String args[])  
{  
        Student s = new Student();  
        s.show();  
    }  
}
```

Output

Name: Unknown

This default constructor automatically assigns "Unknown" to any new Student object. Perfect for when you want to create objects without specifying details immediately!



Pro Tip: If you don't write ANY constructor, Java automatically provides a default one that does nothing!

⚡ Parameterized Constructor Power

Program Example

```
class Student {  
    String name;  
    int age;  
  
    Student(String n, int a) {  
        name = n;  
        age = a;  
    }  
  
    void show() {  
        System.out.println(name + " is " + age + "  
years old.");  
    }  
  
    public static void main(String args[]) {  
        Student s = new Student("Amit", 16);  
        s.show();  
    }  
}
```

Output 🎉

Amit is 16 years old.

Now we're talking! This parameterized constructor lets you create Student objects with specific names and ages right from the start. Way more practical than default values!

Parameterized constructors are game-changers because they let you initialize objects with meaningful data from the get-go. Instead of creating empty objects and then filling them up, you can pass all the necessary information during object creation. This makes your code cleaner, more efficient, and less prone to errors. It's like ordering a custom pizza instead of getting a plain one and adding toppings later!



Constructor Overloading Magic



Multiple Choices

Constructor overloading means having multiple constructors in the same class with different parameter lists!



Ultimate Flexibility

Users can create objects in different ways depending on what information they have available.

```
class Student {  
    String name;  
    int age;  
  
    Student() {  
        name = "Unknown";  
        age = 0;  
    }  
  
    Student(String n, int a) {  
        name = n;  
        age = a;  
    }  
  
    void show() {  
        System.out.println(name + " " + age);  
    }  
  
    public static void main(String args[]) {  
        Student s1 = new Student();  
        Student s2 = new Student("Amit", 16);  
        s1.show();  
        s2.show();  
    }  
}
```


Output:

Unknown 0

Amit 16



Constructor vs Method Showdown

Feature	Constructor 🛠️	Method 🔧
Name	Same as class	Any valid identifier
Return Type	None (not even void)	Must specify
Call	Auto at object creation	Explicit call needed
Purpose	Initialize objects	Perform actions
Overloading	Yes, definitely!	Yes, possible
Inheritance	Not inherited	Can be inherited

Understanding the difference between constructors and methods is crucial for your ICSE exam! Constructors are like the welcoming committee for new objects – they set everything up when an object is born. Methods are like the staff members who do the actual work throughout the object's lifetime. Constructors happen once, methods can be called multiple times!



Common Exam Question: "State two differences between constructors and methods" – This comparison table is your best friend!



HOTS + CKOP Challenge Zone



Higher Order Thinking Skills (HOTS)

- **Question 1:** Why can a constructor not be static?
- **Answer:** Constructors are tied to object creation. Static belongs to the class, not instances!
- **Question 2:** Explain with an example why we use parameterized constructors.
- **Answer:** To initialize objects with specific values immediately, making code more efficient and meaningful.



CKOP Breakdown

- **C (Comprehension):** Define constructor and its purpose.
- **K (Knowledge):** List the three types of constructors.
- **O (Observation):** Debug this code – Is `void Test() { }` a constructor? (No! It has a return type)
- **P (Program):** Write an overloaded constructor for a Book class (title, price).

Exam Booster Recap

100%

Must Remember

Constructors are special methods for object initialization

3

Main Types

Default, Parameterized, and Copy constructors

0

Return Type

Constructors never have any return type, not even void

Flash Notes

- Constructor = special method to initialize objects
- Auto-called when object is created
- Same name as class, no return type
- Can be overloaded for flexibility

Exam Strategy

- Always show outputs for constructor programs
- Constructor vs Method comparison is very common
- Practice overloading examples
- Remember: constructors ≠ methods!



You've Got This! With these constructor concepts locked down, you're ready to tackle any ICSE Java question that comes your way. Remember to practice coding the examples and always write clear outputs. Good luck crushing that exam! 💪



Library Classes in Java

ICSE Class 10 - Complete Study Guide

Welcome to the ultimate guide for mastering Java Library Classes! This comprehensive eBook is designed specifically for ICSE Class 10 students who want to ace their board exams. We'll explore everything from wrapper classes to character methods with tons of examples, outputs, and exam-focused tips.

What You'll Master: Wrapper classes, autoboxing & unboxing, numeric conversions, character methods, and user-defined types. Each concept comes with complete programs and outputs - exactly what you need for board exam success!

CODE &
COFFEE

Introduction to Wrapper Classes

What are Wrapper Classes?

Think of wrapper classes as the **superhero versions** of primitive data types! While primitives like `int`, `double`, and `char` are simple values, wrapper classes are their object counterparts that come with superpowers (methods).

Key Concept: Every primitive type has a corresponding wrapper class that allows you to treat primitive values as objects.



Primitive Type	Wrapper Class
int	Integer
double	Double
char	Character
boolean	Boolean
float	Float
long	Long



Exam Tip: Remember this mapping! Questions often ask you to identify the correct wrapper class for a given primitive type.

Example Program:

```
int n = 25;  
Integer obj = new Integer(n);  
System.out.println("Primitive: " + n);  
System.out.println("Wrapper: " + obj);
```

Output:

```
Primitive: 25  
Wrapper: 25
```

⚡ Autoboxing & Unboxing - Java's Magic!

Autoboxing 📦 → 📁

Automatic conversion from primitive to wrapper object. Java does this behind the scenes!

```
Integer a = 10; // primitive int becomes Integer object
```

Unboxing 📁 → 📦

Automatic conversion from wrapper object to primitive. Java extracts the value!

```
int b = a; // Integer object becomes primitive int
```

Complete Example Program:

```
public class AutoBoxingDemo {  
    public static void main(String[] args) {  
        // Autoboxing - primitive to object  
        Integer a = 10;  
        Double b = 15.5;  
  
        // Unboxing - object to primitive  
        int x = a;  
        double y = b;  
  
        System.out.println("Integer object a = " + a);  
        System.out.println("Double object b = " + b);  
        System.out.println("Primitive x = " + x);  
        System.out.println("Primitive y = " + y);  
        System.out.println("Sum: " + (x + y));  
    }  
}
```


Output:

```
Integer object a = 10  
Double object b = 15.5  
Primitive x = 10  
Primitive y = 15.5  
Sum: 25.5
```



 **Common Error:** Don't try to manually box/unbox when Java can do it automatically.
Modern Java handles this seamlessly!



Classes as Composite Types

Understanding Data Types

In Java, we have two main categories of data types that serve different purposes:

- **Primitive Types:** Store single, simple values directly in memory
- **Composite Types:** Store complex data structures that can hold multiple values or have special behaviors

Classes are the ultimate composite type because they allow you to create your own custom data types with attributes (variables) and behaviors (methods). Think of them as blueprints for creating objects!

Key Insight: While primitives are built into Java, classes let you extend the language by defining new types that fit your specific needs.

Feature	Primitive	Composite
Storage	Single value	Multiple values/complex structure
Memory	Stack	Heap (referenced from stack)
Methods	None	Can have methods
Examples	int, char, boolean	String, Array, ArrayList, Custom Classes
Default Value	0, false, etc.	null

Simple Class Example:

```
class Student {  
    String name; // attribute  
    int age;     // attribute  
  
    // Constructor  
    Student(String n, int a) {  
        name = n;  
        age = a;  
    }  
  
    // Method  
    void displayInfo() {  
        System.out.println("Name: " + name + ", Age: " + age);  
    }  
}
```



 **Board Booster:** Always explain the difference between primitive and composite types with examples. This is a favorite ICSE question!



parseInt() - String to Integer Magic

What is parseInt()?

The `parseInt()` method is your go-to tool for converting string representations of numbers into actual integer values. It's part of the `Integer` wrapper class and is incredibly useful for user input processing!

Syntax: `int result = Integer.parseInt(stringValue);`

When to Use It

- Converting user input from console
- Processing data from files
- Parsing configuration values
- Mathematical calculations with string numbers

Example Program:

```
public class ParseIntDemo {  
    public static void main(String[] args) {  
        String num1 = "123";  
        String num2 = "456";  
  
        // Convert strings to integers  
        int a = Integer.parseInt(num1);  
        int b = Integer.parseInt(num2);  
  
        System.out.println("String num1: " + num1);  
        System.out.println("String num2: " + num2);  
        System.out.println("Integer a: " + a);  
        System.out.println("Integer b: " + b);  
        System.out.println("Sum: " + (a + b));  
        System.out.println("Product: " + (a * b));  
    }  
}
```


Output:

```
String num1: 123  
String num2: 456  
Integer a: 123  
Integer b: 456  
Sum: 579  
Product: 56088
```

📌 ⚠️ **Watch Out:** `parseInt()` throws `NumberFormatException` if the string contains non-numeric characters. "12A" would cause an error!

Pro Tip: In exams, always show both the string input and the converted integer output to demonstrate your understanding!



parseLong() - Handling Big Numbers

Why parseLong()?

When dealing with really large numbers that exceed the range of int (-2,147,483,648 to 2,147,483,647), you need `parseLong()`! It converts string representations to long values.

Range of long: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

Perfect for: Large calculations, file sizes, timestamps, population data, astronomical numbers!

Complete Example:

```
public class ParseLongDemo {
    public static void main(String[] args) {
        String population = "1380000000"; // India's population
        String distance = "384400000";    // Earth to Moon (meters)

        // Convert to long values
        long pop = Long.parseLong(population);
        long dist = Long.parseLong(distance);

        System.out.println("Population string: " + population);
        System.out.println("Distance string: " + distance);
        System.out.println("Population (long): " + pop);
        System.out.println("Distance (long): " + dist);
        System.out.println("Population + Distance: " + (pop + dist));

        // Demonstrate large number calculation
        long result = pop * 2;
        System.out.println("Double population: " + result);
    }
}
```

Output:

```
Population string: 1380000000  
Distance string: 384400000  
Population (long): 1380000000  
Distance (long): 384400000  
Population + Distance: 1764400000  
Double population: 2760000000
```

1

Remember the 'L' suffix

When working with large literal values,
add 'L' at the end: `long x =`
`999999999999L;`

2

Check your ranges

If a number might be too big for `int`, use
`parseLong()` instead of `parseInt()`



parseFloat() - Decimal Precision

Working with Floating Point Numbers

The `parseFloat()` method converts string representations of decimal numbers into float values. Float provides about 7 decimal digits of precision and is perfect for most decimal calculations you'll encounter in ICSE exams.

01	02	03
Input String	Parse Conversion	Mathematical Operations
Start with a string containing a decimal number like "12.5" or "3.14159"	Use <code>Float.parseFloat()</code> to convert the string to a float value	Perform calculations with the converted float values

Comprehensive Example:

```
public class ParseFloatDemo {
    public static void main(String[] args) {
        String price1 = "12.5";
        String price2 = "8.75";
        String discount = "0.15"; // 15% discount

        // Convert strings to floats
        float p1 = Float.parseFloat(price1);
        float p2 = Float.parseFloat(price2);
        float disc = Float.parseFloat(discount);

        System.out.println("Price 1 (string): " + price1);
        System.out.println("Price 2 (string): " + price2);
        System.out.println("Price 1 (float): " + p1);
        System.out.println("Price 2 (float): " + p2);

        float total = p1 + p2;
        float finalPrice = total - (total * disc);

        System.out.println("Total: " + total);
        System.out.println("After 15% discount: " + finalPrice);
        System.out.println("Money saved: " + (total * disc));
    }
}
```


Output:

```
Price 1 (string): 12.5  
Price 2 (string): 8.75  
Price 1 (float): 12.5  
Price 2 (float): 8.75  
Total: 21.25  
After 15% discount: 18.0625  
Money saved: 3.1875
```

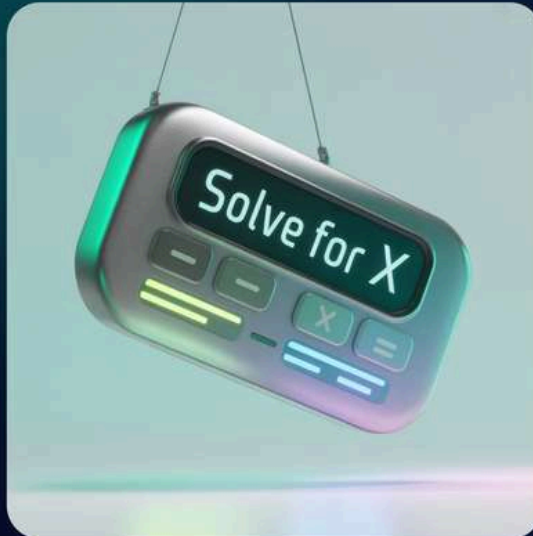
📌 ✨ **Exam Hack:** Always show the 'f' suffix when declaring float literals: `float x = 12.5f;` - it's a common marking point!

parseDouble() - Maximum Precision

Double Precision Power!

`parseDouble()` is the heavyweight champion of decimal conversions! While `float` gives you about 7 decimal digits, `double` provides approximately 15-17 decimal digits of precision. Perfect for scientific calculations, financial computations, and any situation where accuracy matters.

Key Advantage: Double is the default floating-point type in Java, making it the go-to choice for most decimal operations.



Real-World Example Program:

```
public class ParseDoubleDemo {
    public static void main(String[] args) {
        String radius = "5.75";
        String height = "12.8";
        String pi = "3.14159265359";

        // Convert to double for precise calculations
        double r = Double.parseDouble(radius);
        double h = Double.parseDouble(height);
        double piValue = Double.parseDouble(pi);

        System.out.println("Radius (string): " + radius);
        System.out.println("Height (string): " + height);
        System.out.println("Pi (string): " + pi);

        System.out.println("Radius (double): " + r);
        System.out.println("Height (double): " + h);
        System.out.println("Pi (double): " + piValue);

        // Calculate cylinder volume = π × r² × h
        double volume = piValue * r * r * h;
        System.out.println("Cylinder Volume: " + volume);

        // Calculate surface area = 2πr(r + h)
        double surfaceArea = 2 * piValue * r * (r + h);
        System.out.println("Surface Area: " + surfaceArea);
    }
}
```

Output:

```
Radius (string): 5.75
Height (string): 12.8
Pi (string): 3.14159265359
Radius (double): 5.75
Height (double): 12.8
Pi (double): 3.14159265359
Cylinder Volume: 1330.2538973808103
Surface Area: 668.6425994447186
```

Feature	Float	Double
Precision	~7 digits	~15-17 digits
Memory	4 bytes	8 bytes
Suffix	f or F	d or D (optional)
Range	Smaller	Larger

💡 **Pro Strategy:** Use double for all your decimal calculations unless memory is extremely limited. It's more accurate and prevents precision errors!



Character Analysis Methods

isDigit(), isLetter(), isLetterOrDigit()

The Character class provides amazing methods to analyze individual characters! These methods are perfect for validating input, processing text data, and solving string manipulation problems that frequently appear in ICSE exams.



isDigit(char ch)

Returns **true** if the character is a digit (0-9), **false** otherwise. Perfect for validating numeric input!



isLetter(char ch)

Returns **true** if the character is a letter (A-Z or a-z), **false** otherwise. Great for text analysis!



isLetterOrDigit(char ch)

Returns **true** if the character is either a letter OR a digit. Useful for identifier validation!

Complete Testing Program:

```
public class CharacterTestDemo {
    public static void main(String[] args) {
        char[] testChars = {'A', '5', 'z', '@', '9', 'M', ' ', '0'};

        System.out.println("Char\tisDigit\tisLetter\tisLetterOrDigit");
        System.out.println("----\t-----\t-----\t-----");

        for (char ch : testChars) {
            boolean digit = Character.isDigit(ch);
            boolean letter = Character.isLetter(ch);
            boolean letterOrDigit = Character.isLetterOrDigit(ch);

            System.out.println(ch + "\t" + digit + "\t" +
                               letter + "\t" + letterOrDigit);
        }

        // Practical example - count different types
        String text = "Hello123World!";
        int digitCount = 0, letterCount = 0;

        for (int i = 0; i < text.length(); i++) {
            char c = text.charAt(i);
            if (Character.isDigit(c)) digitCount++;
            if (Character.isLetter(c)) letterCount++;
        }

        System.out.println("\nIn string '" + text + "':");
        System.out.println("Letters: " + letterCount);
        System.out.println("Digits: " + digitCount);
    }
}
```


Output:

Char	isDigit	isLetter	isLetterOrDigit
----	-----	-----	-----
A	false	true	true
5	true	false	true
z	false	true	true
@	false	false	false
9	true	false	true
M	false	true	true
	false	false	false
0	true	false	true

In string 'Hello123World!':

Letters: 10

Digits: 3



Character Case & Space Methods

`isLowerCase()`, `isUpperCase()`, `isWhitespace()`

These Character class methods are essential for text processing and formatting! They help you analyze the case of letters and detect whitespace characters, which is super useful for string manipulation problems.

- **`isLowerCase()`**: Checks if a character is lowercase (a-z)
- **`isUpperCase()`**: Checks if a character is uppercase (A-Z)
- **`isWhitespace()`**: Detects spaces, tabs, newlines, etc.



Comprehensive Example Program:

```
public class CharacterCaseDemo {
    public static void main(String[] args) {
        String sample = "Hello World 123!";
        int upperCase = 0, lowerCase = 0, whitespace = 0, other = 0;

        System.out.println("Analyzing: \"" + sample + "\"");
        System.out.println("\nChar\tLower\tUpper\tSpace");
        System.out.println("----\t----\t----\t----");

        for (int i = 0; i < sample.length(); i++) {
            char ch = sample.charAt(i);
            boolean lower = Character.isLowerCase(ch);
            boolean upper = Character.isUpperCase(ch);
            boolean space = Character.isWhitespace(ch);

            // Display each character's properties
            System.out.println("'" + ch + "'\t" + lower + "\t" +
                               upper + "\t" + space);

            // Count totals
            if (lower) lowerCase++;
            else if (upper) upperCase++;
            else if (space) whitespace++;
            else other++;
        }

        System.out.println("\n=== SUMMARY ===");
        System.out.println("Lowercase letters: " + lowerCase);
        System.out.println("Uppercase letters: " + upperCase);
        System.out.println("Whitespace chars: " + whitespace);
        System.out.println("Other characters: " + other);
        System.out.println("Total characters: " + sample.length());

        // Bonus: Test individual characters
        System.out.println("\n=== INDIVIDUAL TESTS ===");
        System.out.println("Is 'a' lowercase? " + Character.isLowerCase('a'));
        System.out.println("Is 'Z' uppercase? " + Character.isUpperCase('Z'));
        System.out.println("Is ' ' whitespace? " + Character.isWhitespace(' '));
        System.out.println("Is 'X' lowercase? " + Character.isLowerCase('X'));
    }
}
```

Output:

Analyzing: "Hello World 123!"

Char	Lower	Upper	Space
----	-----	-----	-----
'H'	false	true	false
'e'	true	false	false
'l'	true	false	false
'l'	true	false	false
'o'	true	false	false
' '	false	false	true
'W'	false	true	false
'o'	true	false	false
'r'	true	false	false
'l'	true	false	false
'd'	true	false	false
' '	false	false	true
'1'	false	false	false
'2'	false	false	false
'3'	false	false	false
'!'	false	false	false

=== SUMMARY ===

Lowercase letters: 8

Uppercase letters: 2

Whitespace chars: 2

Other characters: 5

Total characters: 17

=== INDIVIDUAL TESTS ===

Is 'a' lowercase? true

Is 'Z' uppercase? true

Is ' ' whitespace? true

Is 'X' lowercase? false



Exam Strategy: These methods often appear in programs that process user input or analyze text files. Master the combination of all three!



Character Case Conversion

toLowerCase() & toUpperCase()



toLowerCase()

Converts any uppercase letter to its lowercase equivalent. Leaves other characters unchanged.

'A' → 'a'



toUpperCase()

Converts any lowercase letter to its uppercase equivalent. Leaves other characters unchanged.

'm' → 'M'

Interactive Conversion Example:

```
public class CharacterConversionDemo {
    public static void main(String[] args) {
        String original = "Java Programming 2024!";
        String upperResult = "";
        String lowerResult = "";

        System.out.println("Original string: " + original);
        System.out.println("\nCharacter-by-character conversion:");
        System.out.println("Original\tLower\tUpper");
        System.out.println("-----\t----\t----");

        for (int i = 0; i < original.length(); i++) {
            char originalChar = original.charAt(i);
            char lowerChar = Character.toLowerCase(originalChar);
            char upperChar = Character.toUpperCase(originalChar);

            System.out.println(""" + originalChar + ""\t\t"" +
                lowerChar + ""\t"" + upperChar + "");

            upperResult += upperChar;
            lowerResult += lowerChar;
        }

        System.out.println("\n=== COMPLETE CONVERSIONS ===");
        System.out.println("All UPPERCASE: " + upperResult);
    }
}
```

```

        System.out.println("All lowercase: " + lowerResult);

        // Individual character examples
        System.out.println("\n=== INDIVIDUAL EXAMPLES ===");
        System.out.println("'X' to lower: " + Character.toLowerCase('X'));
        System.out.println("'m' to upper: " + Character.toUpperCase('m'));
        System.out.println("'5' to lower: " + Character.toLowerCase('5'));
        System.out.println("'@' to upper: " + Character.toUpperCase('@'));
    }
}

```

Output:

Original string: Java Programming 2024!

Character-by-character conversion:

Original	Lower	Upper
----------	-------	-------

-----	-----	-----
'J'	'j'	'J'
'a'	'a'	'A'
'v'	'v'	'V'
'a'	'a'	'A'
' '	' '	' '
'P'	'p'	'P'
'r'	'r'	'R'
'o'	'o'	'O'
'g'	'g'	'G'
'r'	'r'	'R'
'a'	'a'	'A'
'm'	'm'	'M'
'm'	'm'	'M'
'i'	'i'	'I'
'n'	'n'	'N'
'g'	'g'	'G'
' '	' '	' '
'2'	'2'	'2'
'0'	'0'	'0'
'2'	'2'	'2'
'4'	'4'	'4'
'!'	'!'	'!'

=== COMPLETE CONVERSIONS ===

All UPPERCASE: JAVA PROGRAMMING 2024!

All lowercase: java programming 2024!

=== INDIVIDUAL EXAMPLES ===

'X' to lower: x

'M' to upper: M

'5' to lower: 5

'@' to upper: @



Quick Tip: These methods only affect letters. Numbers, spaces, and symbols remain exactly the same!



User-Defined Types with Classes

Classes are the foundation of object-oriented programming! They allow you to create your own custom data types that combine attributes (variables) and behaviors (methods) into a single, reusable blueprint.



Blueprint Concept

A class is like an architectural blueprint - it defines the structure and capabilities, but isn't the actual building.



Encapsulation

Classes group related data and functions together, making code more organized and maintainable.



Reusability

Once defined, you can create multiple objects (instances) from the same class template.

Complete Class Example - Book Management:

```
class Book {
    // Attributes (instance variables)
    String title;
    String author;
    double price;
    int pages;

    // Constructor - initializes new objects
    Book(String t, String a, double p, int pg) {
        title = t;
        author = a;
        price = p;
        pages = pg;
    }

    // Method to display book information
    void displayInfo() {
        System.out.println("=== BOOK DETAILS ===");
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
        System.out.println("Price: $" + price);
        System.out.println("Pages: " + pages);
        System.out.println("-----");
    }

    // Method to calculate reading time (assuming 2 minutes per page)
    void estimateReadingTime() {
        int minutes = pages * 2;
        int hours = minutes / 60;
        int remainingMinutes = minutes % 60;
        System.out.println("Estimated reading time: " +
            hours + " hours " + remainingMinutes + " minutes");
    }

    // Method to apply discount
    void applyDiscount(double percentage) {
        double discountAmount = price * (percentage / 100);
        price = price - discountAmount;
        System.out.println("Discount applied! New price: $" + price);
    }
}
```

```
// Main method to test the class
public static void main(String[] args) {
    // Create objects (instances) of the Book class
    Book book1 = new Book("Java Programming", "Oracle", 45.99, 350);
    Book book2 = new Book("Data Structures", "MIT Press", 60.00, 480);

    // Use the methods
    book1.displayInfo();
    book1.estimateReadingTime();
    book1.applyDiscount(15);

    System.out.println();

    book2.displayInfo();
    book2.estimateReadingTime();
    book2.applyDiscount(20);
}
}
```

Output:

```
=== BOOK DETAILS ===
Title: Java Programming
Author: Oracle
Price: $45.99
Pages: 350
-----
Estimated reading time: 11 hours 40 minutes
Discount applied! New price: $39.0915

=== BOOK DETAILS ===
Title: Data Structures
Author: MIT Press
Price: $60.0
Pages: 480
-----
Estimated reading time: 16 hours 0 minutes
Discount applied! New price: $48.0
```

HOTS Questions - Higher Order Thinking

These questions test your deeper understanding of concepts and ability to analyze, evaluate, and apply knowledge creatively. Perfect practice for challenging ICSE exam questions!

1

Conceptual Analysis

Question: Why do wrapper classes exist when primitive types are more efficient in terms of memory and performance?

Answer: Wrapper classes serve several crucial purposes:

- Enable primitives to be used in collections (ArrayList, HashMap)
- Provide utility methods for conversions and operations
- Allow null values (primitives can't be null)
- Support object-oriented programming principles
- Enable generic programming with type parameters

2

Error Analysis

Question: What happens when you pass "12A7" to Integer.parseInt()? Explain the error and provide a solution.

Answer: This throws `NumberFormatException` because "12A7" contains non-numeric characters.

Solutions:

1. Use try-catch blocks to handle the exception
2. Validate input using `Character.isDigit()` before parsing
3. Use regex to check if string matches numeric pattern
4. Sanitize input by removing non-numeric characters first

3

Comparative Analysis

Question: Compare autoboxing vs manual boxing with code examples. When would you choose one over the other?

Manual Boxing: `Integer obj = new Integer(10);`

Autoboxing: `Integer obj = 10;`

Autoboxing is preferred because:

- Cleaner, more readable code
- Less prone to errors
- Compiler optimization
- Modern Java best practices

Challenge Question:

Design Challenge: Create a program that takes a sentence as input and provides a complete analysis using Character class methods. Count uppercase, lowercase, digits, spaces, and special characters. Also convert the entire sentence to title case (first letter of each word capitalized).

  **Exam Strategy:** HOTS questions require you to demonstrate understanding beyond memorization. Always explain the 'why' behind your answers, provide examples, and show alternative approaches when possible.

Analyze

Break down the problem into components

1

Create

Synthesize knowledge to solve novel problems

3

2

Evaluate

Compare different approaches and solutions

CKOP Questions - Complete Assessment

CKOP stands for **Conceptual**, **Knowledge**, **Observation**, and **Practical** questions. This framework covers all types of questions you'll encounter in ICSE exams!

Conceptual (C)

Question 1: Define wrapper classes and explain their relationship with primitive types.

Answer: Wrapper classes are object-oriented representations of primitive data types. Each primitive has a corresponding wrapper class (int→Integer, double→Double, etc.) that provides additional functionality while maintaining the same value.

Question 2: Explain the concept of composite data types with examples.

Answer: Composite types store complex data structures or multiple values, unlike primitives that store single values. Examples include arrays, strings, and user-defined classes.

Knowledge (K)

Question 1: List any 4 methods from the Character class with their purposes.

Answer:

- `isDigit()` - checks if character is a digit (0-9)
- `isLetter()` - checks if character is a letter (A-Z, a-z)
- `toLowerCase()` - converts to lowercase
- `isWhitespace()` - checks for spaces, tabs, newlines

Question 2: Name the parse methods for different data types.

Answer: `parseInt()`, `parseLong()`, `parseFloat()`, `parseDouble()`, `parseBoolean()`

👁 Observation (O)

Predict the output:

```
System.out.println(Character.isLetter('9'));  
System.out.println(Character.isDigit('A'));  
System.out.println(Integer.parseInt("25") + 5);  
System.out.println(Character.toLowerCase('X'));
```

Answer:

```
false  
false  
30  
x
```

💻 Practical (P)

Programming Task: Write a program to count letters, digits, and spaces in a string using Character class methods.

```
public class StringAnalyzer {  
    public static void main(String[] args) {  
        String text = "Hello World 123";  
        int letters = 0, digits = 0, spaces = 0;  
  
        for (int i = 0; i < text.length(); i++)  
        {  
            char ch = text.charAt(i);  
            if (Character.isLetter(ch))  
                letters++;  
            else if (Character.isDigit(ch))  
                digits++;  
            else if  
(Character.isWhitespace(ch))  
                spaces++;  
        }  
  
        System.out.println("Letters: " + letters);  
        System.out.println("Digits: " + digits);  
        System.out.println("Spaces: " + spaces);  
    }  
}
```

Output: Letters: 10, Digits: 3, Spaces: 2

Question Type	Marks Range	Exam Strategy
Conceptual	2-4 marks	Focus on definitions and explanations
Knowledge	1-3 marks	Memorize methods and syntax
Observation	2-5 marks	Trace through code step by step
Practical	5-10 marks	Write complete, working programs

 **Pro Tip:** Practice all four types regularly! ICSE exams test comprehensive understanding, not just memorization.



Recap + Exam Booster

You're Ready!

8

Parse Methods

parseInt, parseLong,
parseFloat, parseDouble

12

Character Methods

isDigit, isLetter, toLowerCase,
toUpperCase, etc.

100%

Exam Ready

Complete coverage of ICSE
syllabus



Flash Notes - Last Minute Revision



Wrapper Classes

Object versions of primitives:
int→Integer, double→Double,
char→Character, boolean→Boolean



Parse Methods

Convert strings to numbers:
parseInt("123")→123,
parseDouble("12.5")→12.5



Character Analysis

Test characters: isDigit('5')→true,
isLetter('A')→true, isWhitespace(''
'')→true



Autoboxing Magic

Automatic conversions: Integer x = 10
(autoboxing), int y = x (unboxing)

Final Exam Strategy

Always Remember:

- **Show outputs** for every parse method and Character method example
- **Explain exceptions** - what happens with invalid input to parseInt()
- **Demonstrate autoboxing** with clear before/after examples
- **Use wrapper classes** when objects are needed (collections, null values)

Common Exam Questions:

- Convert primitives to wrappers and vice versa
- String to number conversions with error handling
- Character analysis programs (count letters/digits)
- Difference between primitive and composite types



Success Mantra

Practice + Understanding + Output = Perfect Score!

Remember: Every program you write should have clear input, processing, and output. Show your work step by step, and always test with different examples. You've got this! 💪



★ **Final Tip:** The night before your exam, review the flash notes above. They contain everything you need to remember. Sleep well, eat breakfast, and approach each question methodically. Your preparation will show!

Good Luck! You're going to do amazing! ✨

✨ Encapsulation in Java – ICSE Class 10

Welcome to your ultimate guide to mastering **Encapsulation in Java**! This comprehensive study material is designed specifically for ICSE Class 10 students who want to ace their exams while actually understanding the concepts. We'll break down everything from access specifiers to variable scope with clear examples, vibrant visuals, and exam-focused tips that'll make this topic stick in your mind.

Get ready to unlock the secrets of data protection in Java programming with this **bright, engaging, and exam-smart** resource!





What is Encapsulation?

Definition

Encapsulation is the process of **binding data (variables) and code (methods) together** into a single unit called a class. Think of it as putting your valuables in a secure safe!

Purpose

- Data Security 
- Code Modularity 
- Data Abstraction 
- Easier Maintenance 

Real-World Example

Just like an ATM machine – you can perform operations (withdraw, deposit) but can't directly access the internal cash storage. The machine **controls access** to its data!

In Java, encapsulation is achieved through **access specifiers** that control who can access your class members. A classic example is a Student class where personal data like marks are kept private, but public methods allow controlled access to this information.



Exam Tip: Remember the 3 main benefits of encapsulation – Security, Modularity, and Abstraction. These often appear in definition questions!

Access Specifiers – Your Data's Bodyguards



private

The most restrictive! Only accessible **within the same class**. Perfect for sensitive data that shouldn't be touched from outside.

Use case: Student marks, bank account balance



protected

Accessible within the **same package and subclasses**. It's like having a family password - only relatives can use it!

Use case: Base class methods for inheritance



public

No restrictions! Accessible from **anywhere in the program**. Like a public park - everyone's welcome!

Use case: Main methods, utility functions

Access Specifier	Same Class	Same Package	Subclass	Everywhere
private	✓	✗	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

This visibility table is **exam gold**! Memorize it because questions about access levels appear frequently in ICSE papers. Notice how each level adds more accessibility - it's like upgrading your security clearance!



Private Access Specifier in Action

Let's see how the `private` access specifier works in practice. Private members are like your personal diary - only you (the class) can read them!

```
class Student {  
    private int marks = 90; // Private variable - locked!
```



Public Access Specifier - Open to All

Public access is the complete opposite of private - it's like broadcasting on social media where **everyone can see everything!** Let's explore how this works with a practical example.

```
public class Demo {  
    public int x = 10; // Public variable
```



Protected Access - The Family Circle

Protected access is like having a **VIP family pass** - it's more exclusive than public but more open than private. Members with protected access are visible to the same package and all subclasses, even if they're in different packages!

Same Package

All classes within the same package can access protected members freely.



Subclasses

Child classes can access protected members of their parent, even across packages.

Inheritance Support

Perfect for sharing methods and variables that subclasses need to override or extend.

The protected modifier is especially important in **inheritance scenarios**. When a parent class has protected members, child classes can access and use them as if they were their own. This creates a perfect balance between security and flexibility.



Exam Focus: ICSE often asks about the difference between protected and private. Remember: protected allows subclass access, private doesn't!

Same Class

Protected member is accessible (just like private and public)

1

2

Same Package

Other classes in the package can access it (unlike private)

Different Package

Only subclasses can access it (unlike public which allows anyone)

3



Complete Visibility Rules Summary

Here's your **ultimate cheat sheet** for access specifiers! This table is probably the most important thing to memorize for your ICSE exam. Each checkmark and cross tells a story about data security and accessibility.

Access Specifier	Same Class	Same Package	Subclass	Everywhere
private	✓	✗	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Private

Most Secure

Only within the same class. Perfect for sensitive data that needs maximum protection.

Example: bankBalance, password

Protected

Family Access

Same package + subclasses. Great for inheritance while maintaining some security.

*Example:
calculateGrade(),
baseAmount*

Public

Open Access

Accessible everywhere. Use for methods that everyone needs to call.

*Example: main(),
display(), getName()*

Think of these access levels as **security clearance levels**: private is top secret (class only), protected is confidential (family only), and public is unclassified (everyone welcome). The progression from private to public represents increasing levels of accessibility and decreasing levels of security.



Memory Trick: "Private Personal Protected Public" - Remember this sequence from most restrictive to least restrictive. The more P's you add, the more people can access it!

Scope of Variables - Where Variables Live

Variable scope determines **where your variables can be accessed and how long they live**. Think of scope as the "neighborhood" where your variable lives - some variables live in small apartments (local scope) while others live in mansions (class scope)!



Local Variables

Declared inside methods, blocks, or constructors. They're like **temporary residents** - they exist only while the method is running.

- Must be initialized before use
- Die when method execution completes
- Cannot be accessed outside the method



Argument Variables

Parameters passed to methods. They're like **visitors with passes** - they bring values from outside into the method.

- Automatically initialized when method is called
- Scope limited to the method
- Act like local variables inside the method



Instance Variables

Belong to each object (non-static). They're like **personal belongings** - each object has its own copy.

- Automatically initialized with default values
- Live as long as the object exists
- Accessible throughout the class



Class Variables (Static)

Shared by all objects of the class. They're like **community property** - everyone shares the same copy.

- Created when class is first loaded
- Only one copy exists regardless of object count
- Can be accessed without creating objects

Understanding scope is crucial for debugging and writing efficient code. When variables have the same name in different scopes, the **innermost scope wins** - this is called variable shadowing!



Variable Scope - Complete Example

Let's see all four types of variables working together in one comprehensive example. This demonstrates how **different scopes interact** and coexist in a real Java program.

```
class VariableDemo {  
    static int c = 100; // Class variable (static)
```

HOTS, CKOP & Final Recap

1 HOTS (Higher Order Thinking Skills)

Critical Thinking Questions:

- *Why is encapsulation important for software security?*
Answer: It prevents unauthorized access to sensitive data and ensures data integrity by controlling how data is accessed and modified.
- *Can private variables ever be accessed outside their class? How?*
Answer: Yes, through public getter/setter methods or reflection (advanced concept).
- *What happens if a local variable and instance variable have the same name?*
Answer: Local variable shadows the instance variable within that method's scope.

2 CKOP (Comprehension, Knowledge, Operation, Problem-solving)

C - Comprehension: Define encapsulation in your own words.

K - Knowledge: List the three access specifiers and their visibility levels.

O - Operation: Debug this code:

```
class Test { private int x; }  
class Demo { Test t = new Test();  
System.out.println(t.x); }
```

P - Problem-solving: Write a program demonstrating the difference between instance and static variables.

Encapsulation

Binding data + methods together for security, modularity, and abstraction

Access Specifiers

private (class only),
protected (package + subclass), public (everywhere)

Variable Scope

Local (method),
Argument (parameter),
Instance (object), Class (static)

Congratulations! You've mastered the fundamentals of encapsulation in Java. Remember the key principle: **keep your data private and provide controlled public access through methods**. This approach leads to more secure, maintainable, and professional code.

Practice these concepts with different examples, and you'll be ready to tackle any encapsulation question in your ICSE Class 10 exam. Good luck! 

✨ Arrays in Java

Organize, Sort, and Search – The Power of Arrays

Welcome to the ultimate exam-focused guide on Java Arrays! 🚀 This comprehensive eBook is designed specifically for ICSE Class 10 students who want to master arrays with visual, bite-sized study material. Get ready to dive into the world of data organization with programs, outputs, and exam tips that'll help you ace your tests!



Introduction & Definition of Arrays

What is an Array?

An **array** is a collection of elements of the **same data type** stored in contiguous memory locations. Think of it like a row of lockers in school – each locker has a number (index) and can store one item of the same type!

Key Features:

- Elements are stored in consecutive memory locations
- All elements must be of the same data type
- Each element has a unique index starting from 0
- Size is fixed once declared



 **Exam Tip:** Array is a **composite data type** – remember this for theory questions!



Types of Arrays

- **Single Dimensional:** Linear arrangement (1D)
- **Double Dimensional:** Matrix format (2D)



Declaration & Initialization of Arrays

Method 1: Declaration then Initialization

```
int arr[] = new  
int[5];  
arr[0] = 10;  
arr[1] = 20;  
// ... and so on
```

Method 2: Direct Initialization

```
int arr[] = {10, 20,  
30, 40, 50};  
// Size  
automatically  
determined
```

Method 3: Declaration with Size

```
int arr[] = new  
int[5];  
// Creates array of 5  
integers  
// All elements  
initialized to 0
```



Complete Example Program

Program Code:

```
class ArrayDemo {  
    public static void main(String args[]) {  
        // Method 1: Direct initialization  
        int numbers[] = {85, 92, 78, 96, 88};  
  
        System.out.println("Array Elements:");  
        for(int i = 0; i < numbers.length; i++) {  
            System.out.println("Index " + i +  
                ": " + numbers[i]);  
        }  
  
        System.out.println("Array Size: " +  
            numbers.length);  
    }  
}
```

Output:

```
Array Elements:  
Index 0: 85  
Index 1: 92  
Index 2: 78  
Index 3: 96  
Index 4: 88  
Array Size: 5
```



Accepting Data into Single Dimensional Array

Let's learn how to input data from users into arrays! This is super important for your exams because **dynamic input** questions are frequently asked. 🎯



Example Program: Student Marks Input

Program Code:

```
import java.util.Scanner;

class StudentMarks {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int marks[] = new int[5];

        System.out.println("Enter marks of 5 students:");

        // Input loop
        for(int i = 0; i < 5; i++) {
            System.out.print("Student " + (i+1) +
                ": ");
            marks[i] = sc.nextInt();
        }

        // Display loop
        System.out.println("\nStudent Marks:");
        for(int i = 0; i < marks.length; i++) {
            System.out.println("Student " + (i+1) +
                ": " + marks[i] + " marks");
        }

        sc.close();
    }
}
```



Student Marks Input - Output Example

Below is a sample run of the **Student Marks Input** program, demonstrating how user input is processed and displayed.

Enter marks of 5 students:

Student 1: 90

Student 2: 85

Student 3: 92

Student 4: 78

Student 5: 95

Student Marks:

Student 1: 90 marks

Student 2: 85 marks

Student 3: 92 marks

Student 4: 78 marks

Student 5: 95 marks



Pro Tip: When looping through an array, always use `arrayName.length` (e.g., `marks.length`) instead of hard-coding the array size. This makes your code more flexible and prevents errors if the array size changes!



Accessing Array Elements

Index-based Access

Arrays use **zero-based indexing**. The first element is at index 0, second at index 1, and so on.

```
int arr[] = {10, 20, 30, 40, 50};  
System.out.println(arr[0]); // Prints  
10  
System.out.println(arr[2]); // Prints  
30
```

Length Property

Use `arr.length` to get the size of an array. This is a property, not a method, so no parentheses!

```
int size = arr.length; // Returns 5  
System.out.println("Size: " + size);
```



Complete Access Example

```
class ArrayAccess {  
    public static void main(String args[]) {  
        String subjects[] = {"Math", "Science",  
                             "English", "History", "Computer"};  
  
        // Direct access  
        System.out.println("First subject: " +  
            subjects[0]);  
        System.out.println("Last subject: " +  
            subjects[subjects.length - 1]);  
  
        // Loop access  
        System.out.println("\nAll subjects:");  
        for(int i = 0; i < subjects.length; i++) {  
            System.out.println((i+1) + ". " +  
                subjects[i]);  
        }  
  
        // Enhanced for loop  
        System.out.println("\nUsing enhanced loop:");  
        for(String sub : subjects) {  
            System.out.println("- " + sub);  
        }  
    }  
}
```



Array Access - Output Example

Below is the sample output for the **Array Access** program, demonstrating different ways to access array elements.

First subject: Math

Last subject: Computer

All subjects:

1. Math
2. Science
3. English
4. History
5. Computer

Using enhanced loop:

- Math
- Science
- English
- History
- Computer

It's crucial to understand array indexing to avoid common errors.



Important Warning: Attempting to access an array element using an index that is outside the valid range (0 to `arrayName.length - 1`) will result in a runtime error called `ArrayIndexOutOfBoundsException`. This is a common mistake, so always double-check your loop conditions and index calculations!



Uses of Arrays



Data Storage 🗄️

Arrays allow you to store multiple values of the same type under a single variable name. Instead of creating separate variables for each student's marks, you can store all marks in one array!



Easy Traversal ↺

You can easily process all elements using loops. This makes operations like finding sum, average, maximum, or minimum values super efficient and clean!



Searching Operations 🔍

Arrays are perfect for implementing search algorithms like linear search and binary search. You can quickly find if a particular element exists in your data collection.



Sorting Operations 📈

Arrays make sorting data incredibly efficient. Whether you need alphabetical order for names or numerical order for marks, arrays support various sorting algorithms.



Matrix Operations 🎯

Two-dimensional arrays are essential for representing matrices, tables, and grids. Perfect for mathematical calculations, game boards, and data organization.



Memory Efficiency 💡

Arrays provide efficient memory usage as elements are stored in contiguous locations. This leads to faster access times and better cache performance.



🎯 **Exam Tip:** Remember that arrays are a **composite data type** because they store multiple elements of the same type!



Sorting – Selection Sort



Algorithm Logic

Selection Sort works by repeatedly finding the minimum element from the unsorted portion and placing it at the beginning.

Steps:

1. Find minimum element in array
2. Swap it with first element
3. Find minimum in remaining array
4. Swap with second element
5. Repeat until sorted



Time Complexity: $O(n^2)$ - but easy to understand!



Complete Selection Sort Program

```
class SelectionSort {
    public static void main(String args[]) {
        int arr[] = {64, 25, 12, 22, 11};
        int n = arr.length;

        System.out.println("Original Array:");
        printArray(arr);

        // Selection sort algorithm
        for(int i = 0; i < n-1; i++) {
            int minIndex = i;

            // Find minimum element in remaining array
            for(int j = i+1; j < n; j++) {
                if(arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }

            // Swap minimum element with first element
            int temp = arr[minIndex];
            arr[minIndex] = arr[i];
            arr[i] = temp;

            System.out.println("After pass " + (i+1) + ":");
            printArray(arr);
        }

        System.out.println("\nFinal Sorted Array:");
        printArray(arr);
    }

    static void printArray(int arr[]) {
        for(int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }
}
```



Selection Sort - Output Example

Below is the sample output for the **Selection Sort** program, illustrating how the array is progressively sorted with each pass.

Original Array:

64 25 12 22 11

After pass 1:

11 25 12 22 64

After pass 2:

11 12 25 22 64

After pass 3:

11 12 22 25 64

After pass 4:

11 12 22 25 64

Final Sorted Array:

11 12 22 25 64

This output demonstrates the step-by-step process of Selection Sort, where the smallest element is identified in the unsorted portion and swapped into its correct position after each pass, until the entire array is sorted.

Sorting – Bubble Sort

Bubble Sort works by repeatedly comparing adjacent elements and swapping them if they're in the wrong order. It's called "bubble" because smaller elements "bubble" to the beginning! 

01

Compare Adjacent Elements

Start from the first element and compare each pair of adjacent elements

02

Swap if Necessary

If the left element is greater than the right element, swap them

03

Continue Through Array

After each pass, the largest element moves to its correct position

04

Repeat Until Sorted

Continue passes until no more swaps are needed



Bubble Sort Program with Step-by-Step Output

```
class BubbleSort {
    public static void main(String args[]) {
        int arr[] = {5, 2, 8, 1, 9};
        int n = arr.length;

        System.out.println("Original Array:");
        printArray(arr);

        // Bubble sort algorithm
        for(int i = 0; i < n-1; i++) {
            boolean swapped = false;

            System.out.println("\nPass " + (i+1) + ":");
            for(int j = 0; j < n-i-1; j++) {
                System.out.println("Comparing " +
                    arr[j] + " and " + arr[j+1]);

                if(arr[j] > arr[j+1]) {
                    // Swap elements
                    int temp = arr[j];
                    arr[j] = arr[j+1];
                    arr[j+1] = temp;
                    swapped = true;
                    System.out.println("Swapped!");
                }
                printArray(arr);
            }

            if(!swapped) break; // Optimization
        }

        System.out.println("\nFinal Sorted Array:");
        printArray(arr);
    }

    static void printArray(int arr[]) {
        for(int value : arr) {
            System.out.print(value + " ");
        }
        System.out.println();
    }
}
```




Bubble Sort - Output Example

Below is the sample output for the **Bubble Sort** program, illustrating the step-by-step comparison and swapping process during each pass, leading to the final sorted array.

Original Array:

5 2 8 1 9

Pass 1:

Comparing 5 and 2

Swapped!

2 5 8 1 9

Comparing 5 and 8

2 5 8 1 9

Comparing 8 and 1

Swapped!

2 5 1 8 9

Comparing 8 and 9

2 5 1 8 9

Pass 2:

Comparing 2 and 5

2 5 1 8 9

Comparing 5 and 1

Swapped!

2 1 5 8 9

Comparing 5 and 8

2 1 5 8 9

Comparing 8 and 9

2 1 5 8 9

Pass 3:

Comparing 2 and 1

Swapped!

1 2 5 8 9

Comparing 2 and 5

1 2 5 8 9

Comparing 5 and 8

1 2 5 8 9

Comparing 8 and 9

1 2 5 8 9

Pass 4:

Comparing 1 and 2

1 2 5 8 9

Comparing 2 and 5

1 2 5 8 9

Comparing 5 and 8

1 2 5 8 9

Comparing 8 and 9

1 2 5 8 9

Final Sorted Array:

1 2 5 8 9

This output clearly shows how adjacent elements are compared and swapped in each pass, with larger elements gradually "bubbling up" to their correct positions, until the entire array is sorted.



Searching – Linear Search

Linear Search is the simplest searching technique! It checks each element sequentially until the target is found or the array ends. Perfect for unsorted arrays! 🎯



Start from Index 0

Begin checking from the first element of the array



Compare Elements

Check if current element matches the search key



Found or Continue

If found, return index; otherwise, move to next element



Linear Search Program

```
import java.util.Scanner;

class LinearSearch {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int arr[] = {10, 23, 45, 67, 89, 34, 12};

        System.out.println("Array elements:");
        for(int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();

        System.out.print("Enter element to search: ");
        int key = sc.nextInt();

        int result = linearSearch(arr, key);

        if(result == -1) {
            System.out.println("Element " + key +
                " not found in the array!");
        } else {
            System.out.println("Element " + key +
                " found at index " + result);
        }
    }
}
```

```
        System.out.println("Position: " + (result + 1));
    }

    sc.close();
}

static int linearSearch(int arr[], int key) {
    System.out.println("\nSearching process:");

    for(int i = 0; i < arr.length; i++) {
        System.out.println("Checking index " + i +
            ": " + arr[i]);

        if(arr[i] == key) {
            System.out.println("Found!");
            return i; // Return index where found
        }
    }

    return -1; // Element not found
}
```




Linear Search - Output Example

Below are sample outputs for the **Linear Search** program, demonstrating two scenarios: one where the target element is successfully found, and another where it is not found within the array. This illustrates the sequential checking process.

Element Found Scenario (Searching for 45)

Array elements:

10 23 45 67 89 34 12

Enter element to search: 45

Searching process:

Checking index 0: 10

Checking index 1: 23

Checking index 2: 45

Found!

Element 45 found at index 2

Position: 3

Element Not Found Scenario (Searching for 99)

Array elements:

10 23 45 67 89 34 12

Enter element to search: 99

Searching process:

Checking index 0: 10

Checking index 1: 23

Checking index 2: 45

Checking index 3: 67

Checking index 4: 89

Checking index 5: 34

Checking index 6: 12

Element 99 not found in the array!



The efficiency of Linear Search is characterized by a time complexity of **$O(N)$** , where 'N' is the number of elements in the array. In the worst-case scenario (element not found or at the very end), the algorithm must examine every element once.

⚡ Searching – Binary Search

Binary Search is like playing a guessing game! It only works on **sorted arrays** but is super fast. It repeatedly divides the search space in half! 🎯⚡

Find Middle

Calculate middle index: $mid = (low + high) / 2$

Repeat

Continue until element is found or search space is empty



Compare

Compare middle element with search key

Narrow Search

Eliminate half of the remaining elements

🖥️ Binary Search Program

```
import java.util.Scanner;

class BinarySearch {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        // Note: Array must be sorted for binary search
        int arr[] = {12, 23, 34, 45, 56, 67, 78, 89, 90};

        System.out.println("Sorted Array elements:");
        for(int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();

        System.out.print("Enter element to search: ");
        int key = sc.nextInt();

        int result = binarySearch(arr, key);

        if(result == -1) {
            System.out.println("Element " + key +
                " not found!");
        } else {
            System.out.println("Element " + key +
                " found at index " + result);
        }
    }
}
```

```

        System.out.println("Element " + key +
            " found at index " + result);
    }

    sc.close();
}

static int binarySearch(int arr[], int key) {
    int low = 0, high = arr.length - 1;

    System.out.println("\nBinary Search Process:");

    while(low <= high) {
        int mid = (low + high) / 2;

        System.out.println("Searching between indices "
            + low + " and " + high);
        System.out.println("Middle index: " + mid +
            ", Value: " + arr[mid]);

        if(arr[mid] == key) {
            System.out.println("Found!");
            return mid;
        }

        if(arr[mid] < key) {
            System.out.println(key + " > " + arr[mid] +
                ", search right half");
            low = mid + 1;
        } else {
            System.out.println(key + " < " + arr[mid] +
                ", search left half");
            high = mid - 1;
        }
    }

    return -1;
}
}

```


Binary Search - Output Example

Below are sample outputs for the **Binary Search** program, demonstrating two scenarios: one where the target element is successfully found, and another where it is not found within the array. This illustrates the efficient search process by repeatedly halving the search space.

Element Found Scenario (Searching for 67)

Sorted Array elements:

12 23 34 45 56 67 78 89 90

Enter element to search: 67

Binary Search Process:

Searching between indices 0 and 8

Middle index: 4, Value: 56

67 > 56, search right half

Searching between indices 5 and 8

Middle index: 6, Value: 78

67 < 78, search left half

Searching between indices 5 and 5

Middle index: 5, Value: 67

Found!

Element 67 found at index 5

Element Not Found Scenario (Searching for 100)

Sorted Array elements:

12 23 34 45 56 67 78 89 90

Enter element to search: 100

Binary Search Process:

Searching between indices 0 and 8

Middle index: 4, Value: 56

100 > 56, search right half

Searching between indices 5 and 8

Middle index: 6, Value: 78

100 > 78, search right half

Searching between indices 7 and 8

Middle index: 7, Value: 89

100 > 89, search right half

Searching between indices 8 and 8

Middle index: 8, Value: 90

100 > 90, search right half

Searching between indices 9 and 8

Element 100 not found!

- ❏ The efficiency of Binary Search is characterized by a time complexity of **$O(\log N)$** , where 'N' is the number of elements in the array. This logarithmic complexity arises because the search space is halved in each step, making it significantly faster than linear search for large datasets.

Introduction to Two-Dimensional Arrays (Matrix)

Two-dimensional arrays are like tables or matrices! Think of them as arrays of arrays - perfect for storing data in rows and columns format. 📊✨

Declaration Syntax



```
int arr[][] = new  
int[3][3];  
// Creates 3x3  
matrix
```

First number = rows,
Second number =
columns

Direct Initialization



```
int matrix[][] = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

Values arranged in matrix
format

Accessing Elements



```
matrix[0][0] // First  
element  
matrix[1][2] // Row  
1, Column 2  
matrix[i][j] //  
General access
```

Use double indexing:
[row][column]



Complete 2D Array Program

```
import java.util.Scanner;  
  
class Matrix2D {  
    public static void main(String args[]) {  
        Scanner sc = new Scanner(System.in);  
        int matrix[][] = new int[3][3];  
  
        System.out.println("Enter elements for 3x3 matrix:");  
  
        // Input elements  
        for(int i = 0; i < 3; i++) {  
            for(int j = 0; j < 3; j++) {  
                System.out.print("Enter element [" + i +  
                    "][" + j + "]: ");  
                matrix[i][j] = sc.nextInt();  
            }  
        }  
    }  
}
```

```
// Display matrix
```

```
System.out.println("\nYour Matrix:");
```

```
displayMatrix(matrix);
```

```
// Display matrix dimensions
```

```
System.out.println("Matrix size: " +
```

```
    matrix.length + "x" + matrix[0].length);
```

```
sc.close();
```

```
}
```

```
static void displayMatrix(int matrix[][]) {
```

```
    for(int i = 0; i < matrix.length; i++) {
```

```
        for(int j = 0; j < matrix[i].length; j++) {
```

```
            System.out.print(matrix[i][j] + "\t");
```

```
        }
```

```
        System.out.println(); // New line after each row
```

```
    }
```

```
}
```

```
}
```



2D Array Input/Output - Example

This example demonstrates the complete input and output process for a 2D array (matrix) using the provided Java program. Observe how elements are prompted one by one, and then the final matrix is displayed along with its dimensions.

Enter elements for 3x3 matrix:

Enter element [0][0]: 1

Enter element [0][1]: 2

Enter element [0][2]: 3

Enter element [1][0]: 4

Enter element [1][1]: 5

Enter element [1][2]: 6

Enter element [2][0]: 7

Enter element [2][1]: 8

Enter element [2][2]: 9

Your Matrix:

1 2 3

4 5 6

7 8 9

Matrix size: 3x3

In the output, `matrix.length` refers to the number of rows in the 2D array, which is 3 in this case. `matrix[0].length` refers to the number of columns in the first row (and typically all rows in a rectangular matrix), also 3.

+ Sum of Rows in a 2D Array

Let's calculate the sum of each row in a matrix! This is a super common exam question. Each row's elements are added together separately. 🧮

🖥️ Row Sum Calculation Program

```
class RowSum {
    public static void main(String args[]) {
        int matrix[][] = {
            {10, 20, 30},
            {5, 15, 25},
            {2, 4, 6}
        };

        System.out.println("Original Matrix:");
        displayMatrix(matrix);

        System.out.println("\nRow-wise Sum Calculation:");

        // Calculate sum of each row
        for(int i = 0; i < matrix.length; i++) {
            int rowSum = 0;
            System.out.print("Row " + i + ": ");

            for(int j = 0; j < matrix[i].length; j++) {
                rowSum += matrix[i][j];
                System.out.print(matrix[i][j]);
                if(j < matrix[i].length - 1) {
                    System.out.print(" + ");
                }
            }

            System.out.println(" = " + rowSum);
        }

        System.out.println("\n--- Row Sums Summary ---");
        for(int i = 0; i < matrix.length; i++) {
            int sum = calculateRowSum(matrix, i);
            System.out.println("Sum of Row " + i +
```



```

        ": " + sum);
    }
}

static void displayMatrix(int matrix[][]) {
    for(int i = 0; i < matrix.length; i++) {
        for(int j = 0; j < matrix[i].length; j++) {
            System.out.print(matrix[i][j] + "\t");
        }
        System.out.println();
    }
}

static int calculateRowSum(int matrix[][], int row) {
    int sum = 0;
    for(int j = 0; j < matrix[row].length; j++) {
        sum += matrix[row][j];
    }
    return sum;
}
}

```

1

Outer Loop for Rows

Use `for(int i = 0; i < matrix.length; i++)` to iterate through each row

2

Inner Loop for Columns

Use `for(int j = 0; j < matrix[i].length; j++)` to add all elements in current row

3

Accumulate Sum

Add each element `matrix[i][j]` to the `rowSum` variable



Row Sum - Output Example

Here's a detailed output demonstrating the execution of the Java program designed to calculate the sum of each row in a two-dimensional array. This output clearly illustrates the original matrix, the step-by-step row-wise sum calculations, and a final summary of all row sums.

Original Matrix:

```
10  20  30
 5   15  25
 2   4   6
```

Row-wise Sum Calculation:

Row 0: $10 + 20 + 30 = 60$

Row 1: $5 + 15 + 25 = 45$

Row 2: $2 + 4 + 6 = 12$

--- Row Sums Summary ---

Sum of Row 0: 60

Sum of Row 1: 45

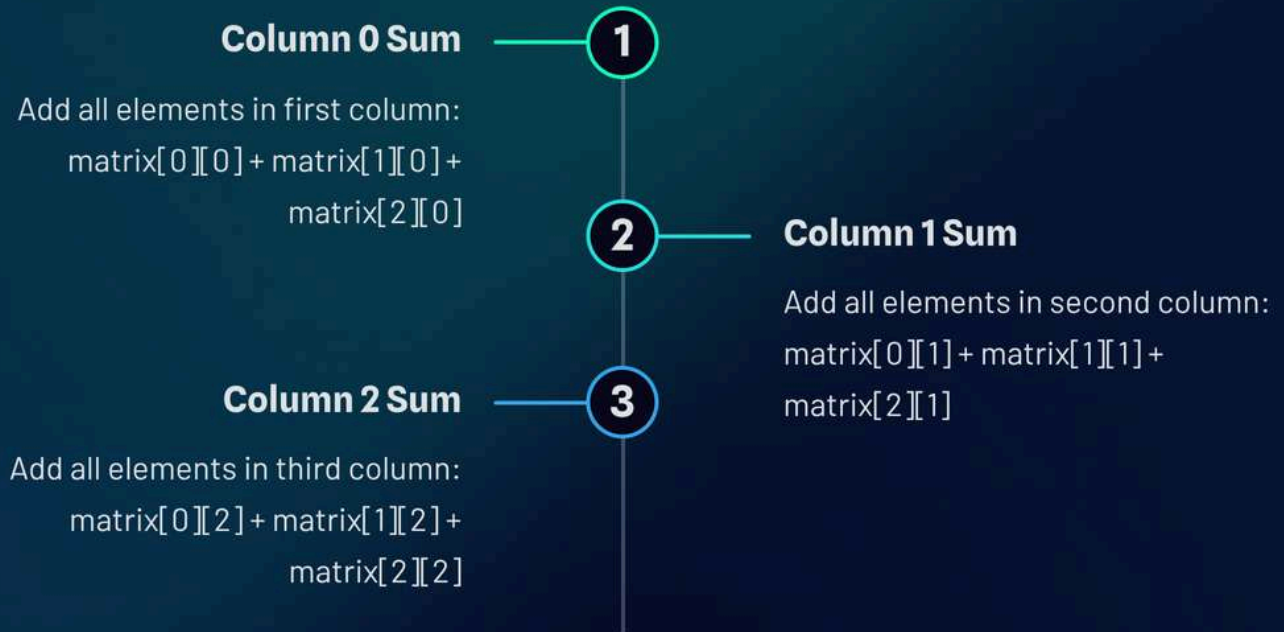
Sum of Row 2: 12

As shown in the output, the program first prints the initial 3x3 matrix. Then, it iterates through each row, explicitly showing the addition of its elements before displaying the total sum for that row. Finally, a concise summary presents the accumulated sum for each individual row, confirming the correct functionality of the row sum logic.



Sum of Columns in a 2D Array

Now let's calculate the sum of each column! This is the opposite of row sums - we add elements vertically instead of horizontally. 📊⬇️



Column Sum Calculation Program

```
class ColumnSum {  
    public static void main(String args[]) {  
        int matrix[][] = {  
            {1, 2, 3, 4},  
            {5, 6, 7, 8},  
            {9, 10, 11, 12}  
        };  
  
        System.out.println("Original Matrix:");  
        displayMatrix(matrix);  
  
        int rows = matrix.length;  
        int cols = matrix[0].length;  
  
        System.out.println("\nColumn-wise Sum Calculation:");  
  
        // Calculate sum of each column  
        for(int j = 0; j < cols; j++) {  
            int colSum = 0;  
            System.out.print("Column " + j + ": ");
```

```

        for(int i = 0; i < rows; i++) {
            colSum += matrix[i][j];
            System.out.print(matrix[i][j]);
            if(i < rows - 1) {
                System.out.print(" ");
            }
        }

        System.out.println(" = " + colSum);
    }

    System.out.println("\n--- Column Sums Array ---");
    int[] columnSums = calculateAllColumnSums(matrix);
    System.out.print("Column Sums: [");
    for(int i = 0; i < columnSums.length; i++) {
        System.out.print(columnSums[i]);
        if(i < columnSums.length - 1) {
            System.out.print(", ");
        }
    }
    System.out.println("]");
}

static void displayMatrix(int matrix[][]) {
    for(int i = 0; i < matrix.length; i++) {
        for(int j = 0; j < matrix[i].length; j++) {
            System.out.print(matrix[i][j] + "\t");
        }
        System.out.println();
    }
}

static int[] calculateAllColumnSums(int matrix[][]) {
    int cols = matrix[0].length;
    int[] sums = new int[cols];

    for(int j = 0; j < cols; j++) {
        for(int i = 0; i < matrix.length; i++) {
            sums[j] += matrix[i][j];
        }
    }

    return sums;
}
}

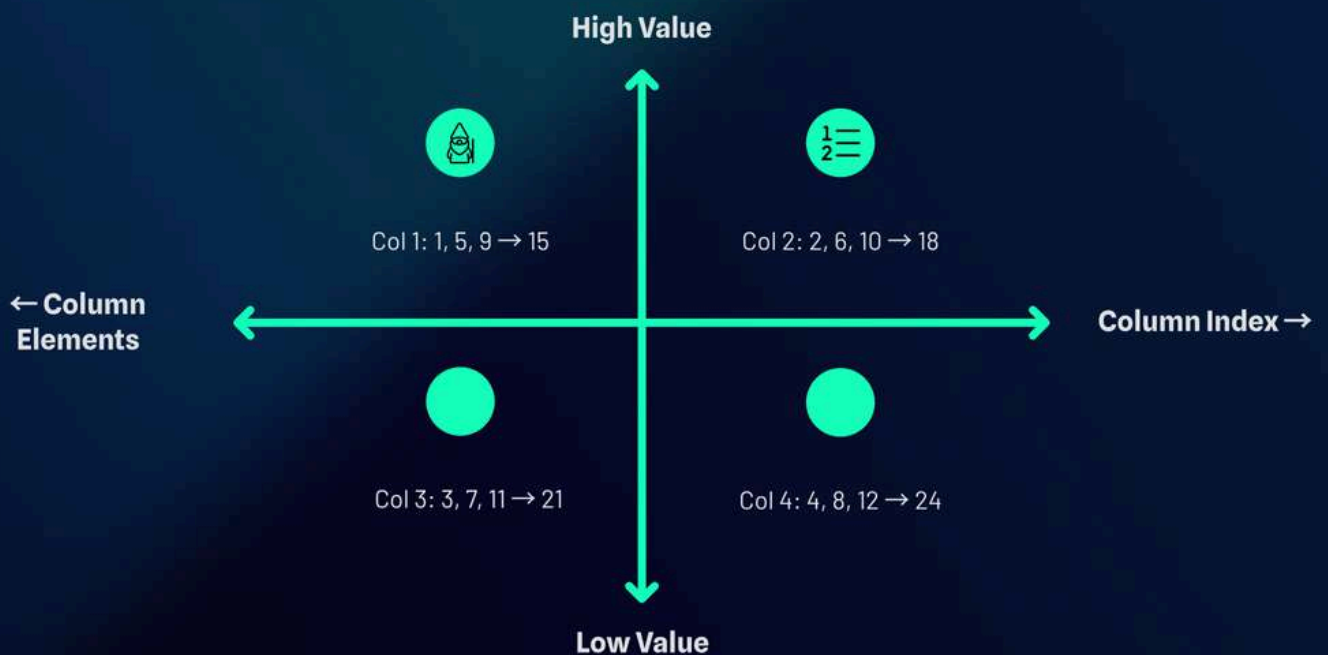
```



Key Difference: Row sums fix i and vary j. Column sums fix j and vary i!



Column Sum - Output Example



Below is the complete output from the column sum calculation program, demonstrating how each column's elements are added together to produce a column-wise sum, along with the final array of column sums.

Original Matrix:

```
1  2  3  4
5  6  7  8
9  10 11 12
```

Column-wise Sum Calculation:

```
Column 0: 1 + 5 + 9 = 15
Column 1: 2 + 6 + 10 = 18
Column 2: 3 + 7 + 11 = 21
Column 3: 4 + 8 + 12 = 24
```

--- Column Sums Array ---

Column Sums: [15, 18, 21, 24]

◆ Diagonal Elements (Left & Right)

Diagonal operations are super important for matrices! **Left diagonal** goes from top-left to bottom-right, while **right diagonal** goes from top-right to bottom-left! 💎



Left Diagonal ↘

Elements where row index = column index

`arr[i][i]` where $i = 0, 1, 2, \dots$

Example: `[0][0], [1][1], [2][2]`



Right Diagonal ↙

Elements where row + column = size - 1

`arr[i][n-1-i]` where $i = 0, 1, 2, \dots$

Example: `[0][2], [1][1], [2][0]`

🖥️ Diagonal Elements Program

```
class DiagonalElements {
    public static void main(String args[]) {
        int matrix[][] = {
            {10, 20, 30},
            {40, 50, 60},
            {70, 80, 90}
        };

        System.out.println("Original 3x3 Matrix:");
        displayMatrix(matrix);

        int n = matrix.length;
        int leftDiagonalSum = 0;
        int rightDiagonalSum = 0;

        System.out.println("\nLeft Diagonal Elements:");
        for(int i = 0; i < n; i++) {
            System.out.println("Position [" + i + "][" +
                i + "] = " + matrix[i][i]);
            leftDiagonalSum += matrix[i][i];
        }
    }
}
```

```

System.out.println("\n--- RESULTS ---");
System.out.println("Left Diagonal Sum: " +
    leftDiagonalSum);
System.out.println("Right Diagonal Sum: " +
    rightDiagonalSum);
System.out.println("Total Diagonal Sum: " +
    (leftDiagonalSum + rightDiagonalSum));

// Check if center element counted twice
if(n % 2 == 1) {
    int center = matrix[n/2][n/2];
    System.out.println("Note: Center element " +
        center + " is part of both diagonals");
    System.out.println("Unique diagonal sum: " +
        (leftDiagonalSum + rightDiagonalSum - center));
}
}

static void displayMatrix(int matrix[][]) {
    for(int i = 0; i < matrix.length; i++) {
        for(int j = 0; j < matrix[i].length; j++) {
            System.out.print(matrix[i][j] + "\t");
        }
        System.out.println();
    }
}
}

```



Diagonal Elements - Output Example

Below is the console output from the Diagonal Elements program, demonstrating how both left and right diagonal elements are identified and summed. Pay close attention to how the center element is handled in odd-sized matrices.

Original 3x3 Matrix:

```
10  20  30
40  50  60
70  80  90
```

Left Diagonal Elements:

```
Position [0][0] = 10
Position [1][1] = 50
Position [2][2] = 90
```

Right Diagonal Elements:

```
Position [0][2] = 30
Position [1][1] = 50
Position [2][0] = 70
```

--- RESULTS ---

Left Diagonal Sum: 150

Right Diagonal Sum: 150

Total Diagonal Sum: 300

Note: Center element 50 is part of both
diagonals

Unique diagonal sum: 250



The program correctly identifies each element by its position and calculates their respective sums. For odd-sized square matrices, the central element is shared by both diagonals, and the code explicitly accounts for this to provide a 'Unique diagonal sum'.



HOTS + CKOP + Recap



Higher Order Thinking Skills (HOTS)

Q1: Compare Linear vs Binary Search

Linear Search: Works on any array, checks sequentially, $O(n)$ time complexity

Binary Search: Requires sorted array, uses divide-and-conquer, $O(\log n)$ time complexity

Why is Binary Search more efficient? It eliminates half the search space in each step!

Q2: When would you use 2D arrays over 1D arrays?

Use 2D arrays for: Game boards, matrices, tables, image pixels, seating arrangements, or any data that naturally has rows and columns structure!



CKOP Questions

Comprehension (C) 📖

Q: Define array and explain its characteristics.

A: Array is a composite data type that stores multiple elements of same type in contiguous memory with fixed size and zero-based indexing.

Knowledge (K) 🧠

Q: List two types of arrays with syntax.

A: 1D Array: `int arr[] = new int[5];` 2D Array: `int arr[][] = new int[3][4];`

Operation (O) ⚙️

Debug this code:

```
int arr[] = new int[5];
System.out.println(arr[5]);
```

Error: `ArrayIndexOutOfBoundsException` - valid indices are 0-4!

Problem Solving (P) 🔧

Q: Write program to sort names alphabetically.

Hint: Use String array with `compareTo()` method in sorting algorithm!

⚡ Quick Recap - Flash Notes

1D

Single Dimensional

Linear array with single index: `arr[i]`

2D

Double Dimensional

Matrix format with double index: `arr[i][j]`

$O(n^2)$

Sorting Time

Selection Sort and Bubble Sort complexity

$O(\log n)$

Binary Search

Fastest search for sorted arrays

🔑 Key Concepts

- Array = Composite data type
- `arr.length` gives size
- Zero-based indexing
- Fixed size after declaration

📈 Sorting Algorithms

- **Selection Sort:** Find minimum, swap
- **Bubble Sort:** Adjacent comparisons
- Both have $O(n^2)$ complexity

🔍 Searching Methods

- **Linear:** Sequential, any array
- **Binary:** Divide-conquer, sorted only
- Binary is much faster!

🎯 2D Array Operations

- Row sum: Fix `i`, vary `j`
- Column sum: Fix `j`, vary `i`
- Left diagonal: `arr[i][i]`
- Right diagonal: `arr[i][n-1-i]`

📋 🎉 **Congratulations!** You've mastered Java Arrays! Practice these programs and you'll ace your ICSE Class 10 exam! 💪 ✨

🌟 String Handling in Java

Welcome to the ultimate guide for conquering String handling in Java! This comprehensive resource is designed specifically for ICSE Class 10 students who want to ace their exams with confidence. We'll dive deep into every String method, explore practical examples, and master string arrays with sorting and searching techniques.

Get ready to transform from a String novice to a Java String wizard! This guide combines exam-focused content with engaging visuals and real-world examples that make learning both effective and enjoyable. Let's unlock the power of String manipulation together! 🚀

HELLO
WORLD



Introduction to String Class

What is a String?

A String is a sequence of characters treated as a single unit. Think of it like a chain of letters, numbers, and symbols all connected together!

Key Property: Immutable

Once a String is created, it cannot be changed. Any "modification" actually creates a brand new String object!

💡 String Declaration Examples

```
// Method 1: String literal
```

```
String s1 = "Hello";
```

```
// Method 2: Using new keyword
```

```
String s2 = new String("World");
```

```
// Method 3: Empty string
```

```
String s3 = "";
```

```
// Method 4: From character array
```

```
char[] chars = {'J', 'a', 'v', 'a'};
```

```
String s4 = new String(chars);
```



Exam Tip: Remember that String literals are stored in the String Pool for memory efficiency, while `new String()` always creates a new object in heap memory!

Understanding Strings is crucial because they're everywhere in programming – from user input to file names, from web URLs to database queries. In Java, the String class provides powerful methods that make text manipulation both simple and efficient. The immutable nature of Strings ensures thread safety and prevents accidental modifications that could cause bugs in your programs.



String trim(), toLowerCase(), toUpperCase()



trim() Method

Removes leading and trailing whitespace from a string. Super useful for cleaning user input!



toLowerCase() Method

Converts all characters to lowercase. Perfect for case-insensitive comparisons.



toUpperCase() Method

Converts all characters to uppercase. Great for standardizing text format.



Complete Program Example

```
public class StringMethods1 {  
    public static void main(String[] args) {  
        String s = " Java Programming ";  
  
        System.out.println("Original: '" + s + "'");  
        System.out.println("Trimmed: '" + s.trim() + "'");  
        System.out.println("Lowercase: " + s.toLowerCase());  
        System.out.println("Uppercase: " + s.toUpperCase());  
  
        // Chain methods together  
        String result = s.trim().toUpperCase();  
        System.out.println("Chained: " + result);  
    }  
}
```



Output

```
Original: ' Java Programming '  
Trimmed: 'Java Programming'  
Lowercase: java programming  
Uppercase: JAVA PROGRAMMING  
Chained: JAVA PROGRAMMING
```



Common Mistake: Remember that these methods return a NEW string – they don't modify the original! Always assign the result to a variable if you want to use it later.



length() & charAt() Methods



length() Method

Returns the number of characters in the string, including spaces and special characters. It's like counting every single character from start to finish!



charAt(index) Method

Returns the character at a specific position (index). Remember: indexing starts from 0, not 1!



Complete Program Example

```
public class StringMethods2 {
    public static void main(String[] args) {
        String s = "Hello World!";

        // Display the string and its length
        System.out.println("String: " + s);
        System.out.println("Length: " + s.length());

        // Access individual characters
        System.out.println("First character: " + s.charAt(0));
        System.out.println("Character at index 6: " + s.charAt(6));
        System.out.println("Last character: " + s.charAt(s.length()-1));

        // Loop through all characters
        System.out.print("All characters: ");
        for(int i = 0; i < s.length(); i++) {
            System.out.print(s.charAt(i) + " ");
        }
    }
}
```



Output

```
String: Hello World!
Length: 12
First character: H
Character at index 6: W
Last character: !
All characters: H e l l o   W o r l d !
```



Exam Alert: `StringIndexOutOfBoundsException` occurs when you try to access an index that doesn't exist. Valid indices are 0 to `length()-1`!

indexOf() & lastIndexOf() Methods



indexOf() Method

Finds the FIRST occurrence of a character or substring. Returns -1 if not found. It's like searching from left to right!



lastIndexOf() Method

Finds the LAST occurrence of a character or substring. Returns -1 if not found. It's like searching from right to left!

Complete Program Example

```
public class StringMethods3 {  
    public static void main(String[] args) {  
        String s = "banana programming";  
  
        System.out.println("String: " + s);  
  
        // Search for single characters  
        System.out.println("First 'a': " + s.indexOf('a'));  
        System.out.println("Last 'a': " + s.lastIndexOf('a'));  
  
        // Search for substrings  
        System.out.println("First 'an': " + s.indexOf("an"));  
        System.out.println("Last 'an': " + s.lastIndexOf("an"));  
  
        // Search from specific position  
        System.out.println("'a' after index 2: " + s.indexOf('a', 3));  
  
        // Character not found  
        System.out.println("'z' found at: " + s.indexOf('z'));  
    }  
}
```

Output

```
String: banana programming  
First 'a': 1  
Last 'a': 5  
First 'an': 1  
Last 'an': 3  
'a' after index 2: 3  
'z' found at: -1
```



Pro Tip: Use indexOf() to check if a substring exists: if(str.indexOf("hello") != -1) means "hello" is found!

concat(), equals() & equalsIgnoreCase()



concat() Method

Joins two strings together. It's like mathematical addition but for text!



equals() Method

Compares two strings for exact equality. Case-sensitive comparison!



equalsIgnoreCase()

Compares two strings ignoring upper/lower case differences.

Complete Program Example

```
public class StringMethods4 {
    public static void main(String[] args) {
        String a = "Hello";
        String b = "World";
        String c = "hello";

        // Concatenation examples
        System.out.println("a + b = " + a.concat(b));
        System.out.println("With space: " + a.concat(" ").concat(b));

        // equals() comparison (case-sensitive)
        System.out.println("a.equals('Hello'): " + a.equals("Hello"));
        System.out.println("a.equals(c): " + a.equals(c));

        // equalsIgnoreCase() comparison
        System.out.println("a.equalsIgnoreCase(c): " + a.equalsIgnoreCase(c));
        System.out.println("a.equalsIgnoreCase('HELLO'): " +
a.equalsIgnoreCase("HELLO"));

        // Why not use == for strings?
        String d = new String("Hello");
        System.out.println("a == d: " + (a == d));
        System.out.println("a.equals(d): " + a.equals(d));
    }
}
```




Output

```
a + b = HelloWorld  
With space: Hello World  
a.equals('Hello'): true  
a.equals(c): false  
a.equalsIgnoreCase(c): true  
a.equalsIgnoreCase('HELLO'): true  
a == d: false  
a.equals(d): true
```

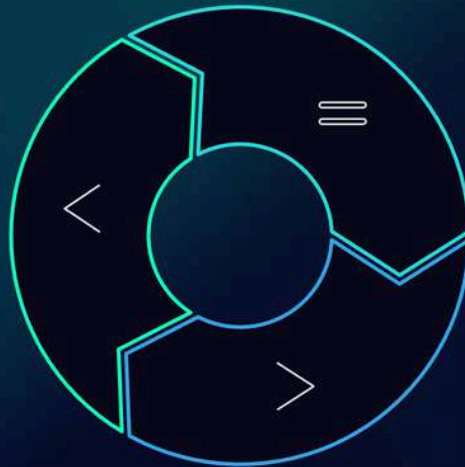


Critical Exam Point: Never use `==` to compare strings! Use `equals()` for exact match or `equalsIgnoreCase()` for case-insensitive comparison.

compareTo() & compareToIgnoreCase()

Returns Negative

When the first string comes BEFORE the second string alphabetically



Returns Zero

When both strings are exactly the same (equal)

Returns Positive

When the first string comes AFTER the second string alphabetically

Complete Program Example

```
public class StringMethods5 {
    public static void main(String[] args) {
        // Basic comparisons
        System.out.println("'apple'.compareTo('banana'): " +
            "apple".compareTo("banana"));
        System.out.println("'zebra'.compareTo('apple'): " + "zebra".compareTo("apple"));
        System.out.println("'hello'.compareTo('hello'): " + "hello".compareTo("hello"));

        // Case-sensitive vs case-insensitive
        System.out.println("'Apple'.compareTo('apple'): " + "Apple".compareTo("apple"));
        System.out.println("'Apple'.compareToIgnoreCase('apple'): " +
            "Apple".compareToIgnoreCase("apple"));

        // Practical usage for sorting
        String[] fruits = {"banana", "apple", "cherry"};
        System.out.println("\nSorting check:");
        for(int i = 0; i < fruits.length-1; i++) {
            int result = fruits[i].compareTo(fruits[i+1]);
            System.out.println(fruits[i] + " vs " + fruits[i+1] + ": " + result);
            if(result > 0) {
                System.out.println(" → Needs swapping for ascending order");
            }
        }
    }
}
```

Output

```
'apple'.compareTo('banana'): -1  
'zebra'.compareTo('apple'): 25  
'hello'.compareTo('hello'): 0  
'Apple'.compareTo('apple'): -32  
'Apple'.compareToIgnoreCase('apple'): 0
```

Sorting check:

banana vs apple: 1

→ Needs swapping for ascending order

apple vs cherry: -2

📌 **Memory Trick:** Think "alphabet order" – if first string comes before second in dictionary, result is negative!



replace() & substring() Methods

01

replace(old, new)

Replaces ALL occurrences of a character or substring with a new one. It's like find-and-replace in a word processor!

02

substring(begin)

Extracts a portion of string starting from 'begin' index to the end of the string.

03

substring(begin, end)

Extracts characters from 'begin' index up to (but not including) 'end' index.



Complete Program Example

```
public class StringMethods6 {
    public static void main(String[] args) {
        String s = "computer programming";
        System.out.println("Original: " + s);

        // replace() examples
        System.out.println("Replace 'o' with 'a': " + s.replace('o', 'a'));
        System.out.println("Replace 'comp' with 'smart': " + s.replace("comp", "smart"));
        System.out.println("Replace all spaces: " + s.replace(" ", "_"));

        // substring() with one parameter
        System.out.println("From index 3: " + s.substring(3));
        System.out.println("From index 9: " + s.substring(9));

        // substring() with two parameters
        System.out.println("From 1 to 4: " + s.substring(1, 4));
        System.out.println("From 9 to 13: " + s.substring(9, 13));

        // Extract file extension example
        String filename = "document.pdf";
        String extension = filename.substring(filename.indexOf('.') + 1);
        System.out.println("File extension: " + extension);
    }
}
```


Output

Original: computer programming

Replace 'o' with 'a': camputer programming

Replace 'comp' with 'smart': smartuter programming

Replace all spaces: computer_programming

From index 3: puter programming

From index 9: programming

From 1 to 4: omp

From 9 to 13: prog

File extension: pdf

 **Index Insight:** substring(begin, end) includes begin but excludes end. Think of it as [begin, end)!

startsWith() & endsWith() Methods

startsWith(prefix)

Returns true if the string begins with the specified prefix. Perfect for checking file types, URL schemes, or name patterns!

endsWith(suffix)

Returns true if the string ends with the specified suffix. Great for file extensions, email domains, or word endings!



Complete Program Example

```
public class StringMethods7 {
    public static void main(String[] args) {
        String s = "Java Programming Language";
        System.out.println("String: " + s);

        // startsWith() examples
        System.out.println("Starts with 'Java': " + s.startsWith("Java"));
        System.out.println("Starts with 'Python': " + s.startsWith("Python"));
        System.out.println("Starts with 'J': " + s.startsWith("J"));

        // endsWith() examples
        System.out.println("Ends with 'Language': " + s.endsWith("Language"));
        System.out.println("Ends with 'ing': " + s.endsWith("ing"));
        System.out.println("Ends with 'age': " + s.endsWith("age"));

        // Practical file extension checker
        String[] files = {"document.pdf", "image.jpg", "script.js", "data.txt"};
        System.out.println("\nFile type checker:");
        for(String file : files) {
            if(file.endsWith(".pdf")) {
                System.out.println(file + " → PDF Document");
            } else if(file.endsWith(".jpg") || file.endsWith(".png")) {
                System.out.println(file + " → Image File");
            } else if(file.endsWith(".js")) {
                System.out.println(file + " → JavaScript File");
            } else {
                System.out.println(file + " → Other File Type");
            }
        }
    }
}
```



Output

String: Java Programming Language

Starts with 'Java': true

Starts with 'Python': false

Starts with 'J': true

Ends with 'Language': true

Ends with 'ing': false

Ends with 'age': true

File type checker:

document.pdf → PDF Document

image.jpg → Image File

script.js → JavaScript File

data.txt → Other File Type

valueOf() Method - The Universal Converter



Complete Program Example

```
public class StringMethods8 {  
    public static void main(String[] args) {  
        // Converting different data types to String  
        int number = 100;  
        double decimal = 25.75;  
        boolean flag = true;  
        char letter = 'A';  
  
        // Using valueOf() method  
        String strNum = String.valueOf(number);  
        String strDec = String.valueOf(decimal);  
        String strBool = String.valueOf(flag);  
        String strChar = String.valueOf(letter);  
  
        System.out.println("Original → String conversion:");  
        System.out.println(number + " → " + strNum + "");  
        System.out.println(decimal + " → " + strDec + "");  
        System.out.println(flag + " → " + strBool + "");  
        System.out.println(letter + " → " + strChar + "");  
  
        // Practical example: Building a formatted message  
        String name = "Alice";  
        int age = 16;  
        double grade = 95.5;  
  
        String report = "Student: " + String.valueOf(name) +  
            ", Age: " + String.valueOf(age) +  
            ", Grade: " + String.valueOf(grade) + "%";  
  
        System.out.println("\nFormatted Report:");  
        System.out.println(report);  
  
        // Array conversion  
        char[] charArray = {'H', 'e', 'l', 'l', 'o'};  
        String fromArray = String.valueOf(charArray);  
        System.out.println("From char array: " + fromArray);  
    }  
}
```


Output

Original → String conversion:

100 → '100'

25.75 → '25.75'

true → 'true'

A → 'A'

Formatted Report:

Student: Alice, Age: 16, Grade: 95.5%

From char array: Hello

 **Expert Tip:** `valueOf()` is often used implicitly when you use `+` operator with strings. "Age: " + 16 automatically calls `String.valueOf(16)`!



Character Extraction & Modification Magic

Now let's combine multiple string methods to solve real programming challenges! Character extraction and modification are essential skills for text processing, data cleaning, and creating interactive applications.

01

Extract Specific Characters

Use `charAt()` and loops to access individual characters at any position in the string.

02

Modify Character Patterns

Use `replace()` methods to change specific characters or patterns throughout the string.

03

Build New Strings

Combine extraction and modification to create entirely new strings based on specific rules.



Program: Replace All Vowels with Asterisks

```
public class CharacterModification {
    public static void main(String[] args) {
        String original = "Programming is Fun and Exciting!";
        System.out.println("Original String: " + original);

        // Method 1: Using replace() for each vowel
        String modified1 = original.toLowerCase()
            .replace('a', '*')
            .replace('e', '*')
            .replace('i', '*')
            .replace('o', '*')
            .replace('u', '*');

        System.out.println("Method 1 Result: " + modified1);

        // Method 2: Character-by-character approach
        String modified2 = "";
        for(int i = 0; i < original.length(); i++) {
            char ch = original.charAt(i);
            char lower = Character.toLowerCase(ch);
```

```

        if(lower == 'a' || lower == 'e' || lower == 'i' ||
           lower == 'o' || lower == 'u') {
            modified2 += '*';
        } else {
            modified2 += ch;
        }
    }
}

```

```

System.out.println("Method 2 Result: " + modified2);

```

```

// Bonus: Count vowels replaced

```

```

int vowelCount = 0;

```

```

for(int i = 0; i < original.length(); i++) {

```

```

    char ch = Character.toLowerCase(original.charAt(i));

```

```

    if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {

```

```

        vowelCount++;

```

```

    }

```

```

}

```

```

System.out.println("Total vowels replaced: " + vowelCount);

```

```

}

```

```

}

```

Output

Original String: Programming is Fun and Exciting!

Method 1 Result: pr*gr*mm*ng *s f*n *nd *xc*t*ng!

Method 2 Result: Pr*gr*mm*ng *s F*n *nd *xc*t*ng!

Total vowels replaced: 10

 **Exam Strategy:** Practice both approaches! Method 1 is more concise, but Method 2 shows better understanding of loops and character manipulation.



String Arrays - Alphabetical Sorting (Bubble Sort)

Bubble Sort is like bubbles rising to the surface! In each pass, the largest (alphabetically last) string "bubbles up" to its correct position. It's simple to understand and implement, making it perfect for ICSE exams.



Compare Adjacent

Compare each pair of adjacent strings using compareTo() method



Swap if Needed

If left string > right string alphabetically, swap their positions



Repeat Process

Continue until no more swaps are needed (array is sorted)



Complete Bubble Sort Program

```
public class BubbleSortStrings {
    public static void main(String[] args) {
        String[] names = {"Zara", "Alice", "Charlie", "Bob", "Diana", "Eve"};

        System.out.println("Original Array:");
        printArray(names);

        // Bubble Sort Algorithm
        int n = names.length;
        for(int i = 0; i < n-1; i++) {
            System.out.println("\n--- Pass " + (i+1) + " ---");
            boolean swapped = false;

            for(int j = 0; j < n-1-i; j++) {
                System.out.print("Comparing '" + names[j] + "' with '" + names[j+1] + "': ");

                if(names[j].compareTo(names[j+1]) > 0) {
                    // Swap strings
                    String temp = names[j];
                    names[j] = names[j+1];
                    names[j+1] = temp;
                    swapped = true;
                    System.out.println("SWAPPED");
                } else {
                    System.out.println("No swap needed");
                }
            }
        }
    }
}
```



```

        printArray(names);

        // Early termination if array is already sorted
        if(!swapped) {
            System.out.println("Array is sorted! Stopping early.");
            break;
        }
    }

    System.out.println("\nFinal Sorted Array:");
    printArray(names);
}

public static void printArray(String[] arr) {
    System.out.print("[ ");
    for(String name : arr) {
        System.out.print(name + " ");
    }
    System.out.println("]");
}
}

```

Output

Original Array:

[Zara Alice Charlie Bob Diana Eve]

--- Pass 1 ---

Comparing 'Zara' with 'Alice': No swap needed

Comparing 'Alice' with 'Charlie': SWAPPED

Comparing 'Charlie' with 'Bob': No swap needed

Comparing 'Bob' with 'Diana': SWAPPED

Comparing 'Diana' with 'Eve': SWAPPED

[Alice Zara Bob Charlie Diana Eve]

--- Pass 2 ---

Comparing 'Alice' with 'Zara': SWAPPED

Comparing 'Zara' with 'Bob': No swap needed

Comparing 'Bob' with 'Charlie': SWAPPED

Comparing 'Charlie' with 'Diana': SWAPPED

[Alice Bob Zara Charlie Diana Eve]

Final Sorted Array:

[Alice Bob Charlie Diana Eve Zara]

String Arrays - Selection Sort Algorithm

Selection Sort works by finding the smallest element and placing it at the beginning, then finding the second smallest, and so on. It's like organizing a deck of cards by always picking the smallest card from the unsorted pile!

Find Minimum	Swap to Position	Expand Sorted
Search through unsorted portion to find the alphabetically smallest string	Move the minimum string to the current position in sorted portion	Increase the sorted portion by one and repeat for remaining elements

Complete Selection Sort Program

```
public class SelectionSortStrings {
    public static void main(String[] args) {
        String[] fruits = {"Orange", "Apple", "Mango", "Banana", "Cherry", "Grape"};

        System.out.println("Original Array:");
        printArray(fruits);

        int n = fruits.length;

        // Selection Sort Algorithm
        for(int i = 0; i < n-1; i++) {
            System.out.println("\n--- Step " + (i+1) + ": Finding minimum from index " + i + " --");

            // Find minimum element in remaining unsorted array
            int minIndex = i;
            for(int j = i+1; j < n; j++) {
                System.out.println("Comparing '" + fruits[minIndex] + "' with '" + fruits[j] + "'");

                if(fruits[j].compareTo(fruits[minIndex]) < 0) {
                    minIndex = j;
                    System.out.println(" New minimum found: '" + fruits[j] + "' at index " + j);
                }
            }
        }
    }
}
```

```
// Swap minimum element with first element of unsorted part
```

```
if(minIndex != i) {
```

```
    System.out.println("Swapping '" + fruits[i] + "' with '" + fruits[minIndex] + "'");
```

```
    String temp = fruits[minIndex];
```

```
    fruits[minIndex] = fruits[i];
```

```
    fruits[i] = temp;
```

```
} else {
```

```
    System.out.println("'" + fruits[i] + "' is already in correct position");
```

```
}
```

```
System.out.print("Array after step " + (i+1) + ": ");
```

```
printArray(fruits);
```

```
// Show sorted vs unsorted portions
```

```
System.out.print("Sorted portion: [ ");
```

```
for(int k = 0; k <= i; k++) {
```

```
    System.out.print(fruits[k] + " ");
```

```
}
```

```
System.out.println("]");
```

```
}
```

```
System.out.println("\nFinal Sorted Array:");
```

```
printArray(fruits);
```

```
}
```

```
public static void printArray(String[] arr) {
```

```
    System.out.print("[ ");
```

```
    for(String item : arr) {
```

```
        System.out.print(item + " ");
```

```
    }
```

```
    System.out.println("]");
```

```
}
```

```
}
```

Output

Original Array:

[Orange Apple Mango Banana Cherry Grape]

--- Step 1: Finding minimum from index 0 ---

Comparing 'Orange' with 'Apple'

New minimum found: 'Apple' at index 1

Comparing 'Apple' with 'Mango'

Comparing 'Apple' with 'Banana'

Comparing 'Apple' with 'Cherry'

Comparing 'Apple' with 'Grape'

Swapping 'Orange' with 'Apple'

Array after step 1: [Apple Orange Mango Banana Cherry Grape]

Sorted portion: [Apple]

Final Sorted Array:

[Apple Banana Cherry Grape Mango Orange]



String Array Searching - Linear Search

What is Linear Search?

Linear search checks each element one by one from start to finish until the target is found or the array ends. It's simple but thorough!

Time Complexity

Best case: $O(1)$ if element is first. Worst case: $O(n)$ if element is last or not found.
Average: $O(n/2)$.



Complete Linear Search Program

```
import java.util.Scanner;

public class LinearSearchStrings {
    public static void main(String[] args) {
        String[] countries = {"India", "USA", "China", "Japan", "Germany",
            "France", "Brazil", "Australia", "Canada", "Russia"};

        Scanner sc = new Scanner(System.in);

        System.out.println("Available countries:");
        printArrayWithIndex(countries);

        System.out.print("\nEnter country name to search: ");
        String target = sc.nextLine();

        // Perform linear search
        int result = linearSearch(countries, target);

        if(result != -1) {
            System.out.println("\nFound at index: " + result);
        } else {
            System.out.println("\nNot found");
        }
    }
}
```



HOTS + CKOP + Quick Recap

🤔 HOTS (Higher Order Thinking)

Q1: Why are Strings immutable in Java?

Answer: Security, thread safety, memory efficiency (String pool), and hashcode caching.

Q2: Difference between == and .equals() for Strings?

Answer: == compares object references, .equals() compares actual content.

📖 CKOP Framework

C (Comprehension): Define substring() with example.

K (Knowledge): Name 5 String methods.

O (Output): "Hello".compareTo("Hello") → 0

P (Program): Count vowels in a string.



Vowel Counter Program (CKOP Practice)

```
public class VowelCounter {
    public static void main(String[] args) {
        String text = "Programming is Amazing!";
        int vowelCount = 0;

        System.out.println("Text: " + text);
        System.out.print("Vowels found: ");

        for(int i = 0; i < text.length(); i++) {
            char ch = Character.toLowerCase(text.charAt(i));
            if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
                System.out.print(text.charAt(i) + "(" + i + ") ");
                vowelCount++;
            }
        }

        System.out.println("\nTotal vowels: " + vowelCount);
        System.out.println("Vowel percentage: " +
            String.format("%.1f", (vowelCount * 100.0 / text.length())) + "%");
    }
}
```

15

String Methods

Essential methods mastered
for ICSE success

3

Search & Sort

Algorithms learned for string
array manipulation

100%

Exam Ready

Complete preparation for
String handling topics

Quick Recap Flash Notes



String Fundamentals

String = class, immutable, sequence of
characters



Key Methods Mastered

trim(), length(), charAt(), indexOf(),
substring(), valueOf(), equals(),
compareTo()



String Arrays

Sorting (bubble, selection) and
searching (linear) algorithms



Exam Success Tips

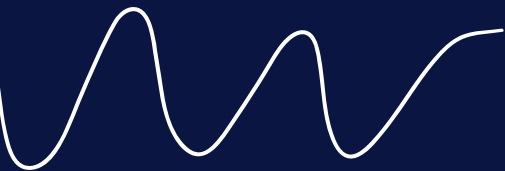
Practice programs with outputs,
understand immutability, never use ==
for comparison



Congratulations! You've mastered String handling in Java! Practice these programs,
understand the concepts, and you'll ace your ICSE Class 10 exams. Keep coding and stay
awesome! 💪 ✨



50 MOST IMPORTANT PROGRAMS



CONDITIONAL CONSTRUCTS

WRITE A PROGRAM TO INPUT THREE NUMBERS (POSITIVE OR NEGATIVE). IF THEY ARE UNEQUAL THEN DISPLAY THE GREATEST NUMBER OTHERWISE, DISPLAY THEY ARE EQUAL. THE PROGRAM ALSO DISPLAYS WHETHER THE NUMBERS ENTERED BY THE USER ARE 'ALL POSITIVE', 'ALL NEGATIVE' OR 'MIXED NUMBERS'.

```
import java.util.Scanner;

public class NumberAnalysis
{
    public static void main(String args[]) {

        Scanner in = new Scanner(System.in);

        System.out.print("Enter first number: ");
        int a = in.nextInt();

        System.out.print("Enter second number: ");
        int b = in.nextInt();

        System.out.print("Enter third number: ");
        int c = in.nextInt();

        int greatestNumber = a;

        if ((a == b) && (b == c)) {
            System.out.println("Entered numbers are equal.");
        }
        else {
            if (b > greatestNumber) {
                greatestNumber = b;
            }

            if (c > greatestNumber) {
                greatestNumber = c;
            }

            System.out.println("The greatest number is " + greatestNumber);

            if ((a >= 0) && (b >= 0) && (c >= 0)) {
                System.out.println("Entered numbers are all positive numbers.");
            }
            else if ((a < 0) && (b < 0) && (c < 0)) {
                System.out.println("Entered numbers are all negative numbers.");
            }
            else {
                System.out.println("Entered numbers are mixed numbers.");
            }
        }
    }
}
```

CONDITIONAL CONSTRUCTS



A TRIANGLE IS SAID TO BE AN 'EQUABLE TRIANGLE', IF THE AREA OF THE TRIANGLE IS EQUAL TO ITS PERIMETER. WRITE A PROGRAM TO ENTER THREE SIDES OF A TRIANGLE. CHECK AND PRINT WHETHER THE TRIANGLE IS EQUABLE OR NOT. FOR EXAMPLE, A RIGHT ANGLED TRIANGLE WITH SIDES 5, 12 AND 13 HAS ITS AREA AND PERIMETER BOTH EQUAL TO 30.

```
import java.util.Scanner;

public class EquableTriangle
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);

        System.out.println("Please enter the 3 sides of the triangle.");

        System.out.print("Enter the first side: ");
        double a = in.nextDouble();

        System.out.print("Enter the second side: ");
        double b = in.nextDouble();

        System.out.print("Enter the third side: ");
        double c = in.nextDouble();

        double p = a + b + c;
        double s = p / 2;
        double area = Math.sqrt(s * (s - a) * (s - b) * (s - c));

        if (area == p)
            System.out.println("Entered triangle is equable.");
        else
            System.out.println("Entered triangle is not equable.");
    }
}
```

CONDITIONAL CONSTRUCTS



A SPECIAL TWO-DIGIT NUMBER IS SUCH THAT WHEN THE SUM OF ITS DIGITS IS ADDED TO THE PRODUCT OF ITS DIGITS, THE RESULT IS EQUAL TO THE ORIGINAL TWO-DIGIT NUMBER. EXAMPLE: CONSIDER THE NUMBER 59. SUM OF DIGITS = $5 + 9 = 14$ PRODUCT OF DIGITS = $5 * 9 = 45$ TOTAL OF THE SUM OF DIGITS AND PRODUCT OF DIGITS = $14 + 45 = 59$ WRITE A PROGRAM TO ACCEPT A TWO-DIGIT NUMBER. ADD THE SUM OF ITS DIGITS TO THE PRODUCT OF ITS DIGITS. IF THE VALUE IS EQUAL TO THE NUMBER INPUT, DISPLAY THE MESSAGE "SPECIAL 2 - DIGIT NUMBER" OTHERWISE, DISPLAY THE MESSAGE "NOT A SPECIAL TWO-DIGIT NUMBER".

```
import java.util.Scanner;

public class SpecialNumber
{
    public static void main(String args[]) {

        Scanner in = new Scanner(System.in);

        System.out.print("Enter a 2 digit number: ");
        int orgNum = in.nextInt();

        if (orgNum < 10 || orgNum > 99) {
            System.out.println("Invalid input. Entered number is not a 2 digit number");
            System.exit(0);
        }

        int num = orgNum;

        int digit1 = num % 10;
        int digit2 = num / 10;
        num /= 10;

        int digitSum = digit1 + digit2;
        int digitProduct = digit1 * digit2;
        int grandSum = digitSum + digitProduct;

        if (grandSum == orgNum)
            System.out.println("Special 2-digit number");
        else
            System.out.println("Not a special 2-digit number");

    }
}
```

CONDITIONAL CONSTRUCTS



WRITE A PROGRAM THAT ACCEPTS THREE NUMBERS FROM THE USER AND DISPLAYS THEM EITHER IN "INCREASING ORDER" OR IN "DECREASING ORDER" AS PER THE USER'S CHOICE.

CHOICE 1: ASCENDING ORDER

CHOICE 2: DESCENDING ORDER

SAMPLE INPUTS: 394, 475, 296

CHOICE 2

SAMPLE OUTPUT

FIRST NUMBER : 475

SECOND NUMBER: 394

THIRD NUMBER : 296

THE NUMBERS ARE IN DECREASING ORDER.

```
import java.util.Scanner;

public class BusFare
{
    public static void main(String args[]) {

        Scanner in = new Scanner(System.in);

        System.out.print("Enter distance travelled: ");
        int dist = in.nextInt();

        int fare = 0;

        if (dist <= 0)
            fare = 0;
        else if (dist <= 10)
            fare = 80;
        else if (dist <= 20)
            fare = 80 + (dist - 10) * 6;
        else if (dist <= 30)
            fare = 80 + 60 + (dist - 20) * 5;
        else if (dist > 30)
            fare = 80 + 60 + 50 + (dist - 30) * 4;

        System.out.println("Fare = " + fare);
    }
}
```


CONDITIONAL CONSTRUCTS

A COURIER COMPANY CHARGES DIFFERENTLY FOR 'ORDINARY' BOOKING AND 'EXPRESS' BOOKING BASED ON THE WEIGHT OF THE PARCEL AS PER THE TARIFF GIVEN BELOW:

WEIGHT OF PARCEL	ORDINARY BOOKING.	EXPRESS BOOKING
UP TO 100 GM	₹80	₹100
101 TO 500 GM	₹150	₹200
501 GM TO 1000 GM	₹210	₹250
MORE THAN 1000 GM	₹250	₹300

WRITE A PROGRAM TO INPUT WEIGHT OF A PARCEL AND TYPE OF BOOKING ('O' FOR ORDINARY AND 'E' FOR EXPRESS). CALCULATE AND PRINT THE CHARGES ACCORDINGLY.

```
import java.util.Scanner;

public class CourierCompany
{
    public static void main(String args[]) {

        Scanner in = new Scanner(System.in);

        System.out.print("Enter weight of parcel: ");
        double wt = in.nextDouble();

        System.out.print("Enter type of booking: ");
        char type = in.next().charAt(0);

        double charge = 0;

        if (wt <= 0)
            charge = 0;
        else if (wt <= 100 && type == 'O')
            charge = 80;
        else if (wt <= 100 && type == 'E')
            charge = 100;
        else if (wt <= 500 && type == 'O')
            charge = 150;
        else if (wt <= 500 && type == 'E')
            charge = 200;
        else if (wt <= 1000 && type == 'O')
            charge = 210;
        else if (wt <= 1000 && type == 'E')
            charge = 250;
        else if (wt > 1000 && type == 'O')
            charge = 250;
        else if (wt > 1000 && type == 'E')
            charge = 300;

        System.out.println("Parcel charges = " + charge);
    }
}
```

CONDITIONAL CONSTRUCTS



WRITE THE PROGRAMS IN JAVA TO FIND THE SUM OF THE FOLLOWING SERIES:

(A) $S = 1 + 1 + 2 + 3 + 5 + \dots$ TO N TERMS

```
import java.util.Scanner;

public class Series
{
    public void computeSeriesSum() {

        Scanner in = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = in.nextInt();

        int a = 1, b = 1;
        int sum = a + b;

        for (int i = 3; i <= n; i++) {
            int term = a + b;
            sum += term;
            a = b;
            b = term;
        }

        System.out.println("Sum=" + sum);
    }
}
```

ITERATIVE CONSTRUCTS



WRITE THE PROGRAMS IN JAVA TO FIND THE SUM OF THE FOLLOWING SERIES:

(B) $S = 2 - 4 + 6 - 8 + \dots$ TO N

```
import java.util.Scanner;

public class Series
{
    public void computeSeriesSum() {

        Scanner in = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = in.nextInt();

        int sum = 0;

        for (int i = 1, j = 2; i <= n; i++, j = j + 2) {
            if (i % 2 == 0) {
                sum -= j;
            }
            else {
                sum += j;
            }
        }

        System.out.println("Sum=" + sum);
    }
}
```

ITERATIVE CONSTRUCTS



WRITE THE PROGRAMS IN JAVA TO FIND THE SUM OF THE FOLLOWING SERIES:

(C) $S = 1 + (1+2) + (1+2+3) + \dots + (1+2+3+ \dots + N)$

```
import java.util.Scanner;

public class Series
{
    public void computeSeriesSum() {

        Scanner in = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = in.nextInt();

        int a = 1, b = 1;
        int sum = a + b;

        for (int i = 3; i <= n; i++) {
            int term = a + b;
            sum += term;
            a = b;
            b = term;
        }

        System.out.println("Sum=" + sum);
    }
}
```


ITERATIVE CONSTRUCTS



WRITE THE PROGRAM TO FIND THE SUM OF THE FOLLOWING SERIES:

$$S = (A/2) + (A/5) + (A/8) + (A/11) + \dots + (A/20)$$

```
import java.util.Scanner;

public class Series
{
    public void computeSeriesSum() {

        Scanner in = new Scanner(System.in);
        System.out.print("Enter a: ");
        int a = in.nextInt();
        double sum = 0;

        for (int i = 2; i <= 20; i = i + 3) {
            sum += a / (double)i;
        }

        System.out.println("Sum=" + sum);
    }
}
```

ITERATIVE CONSTRUCTS



WRITE A PROGRAM TO ENTER TWO NUMBERS AND CHECK WHETHER THEY ARE CO-PRIME OR NOT.
[TWO NUMBERS ARE SAID TO BE CO-PRIME, IF THEIR HCF IS 1 (ONE).]
SAMPLE INPUT: 14, 15
SAMPLE OUTPUT: THEY ARE CO-PRIME.

```
import java.util.Scanner;

public class Coprime
{
    public void checkCoprime() {

        Scanner in = new Scanner(System.in);
        System.out.print("Enter a: ");
        int a = in.nextInt();
        System.out.print("Enter b: ");
        int b = in.nextInt();

        int hcf = 1;

        for (int i = 1; i <= a && i <= b; i++) {
            if (a % i == 0 && b % i == 0)
                hcf = i;
        }

        if (hcf == 1)
            System.out.println(a + " and " + b + " are co-prime");
        else
            System.out.println(a + " and " + b + " are not co-prime");
    }
}
```

ITERATIVE CONSTRUCTS



WRITE A PROGRAM TO INPUT A NUMBER AND CHECK AND PRINT WHETHER IT IS A PRONIC NUMBER OR NOT. [PRONIC NUMBER IS THE NUMBER WHICH IS THE PRODUCT OF TWO CONSECUTIVE INTEGERS.]

EXAMPLES:

$12 = 3 * 4$

$20 = 4 * 5$

$42 = 6 * 7$

```
import java.util.Scanner;

public class PronicNumber
{
    public void pronicCheck() {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter the number to check: ");
        int num = in.nextInt();

        boolean isPronic = false;

        for (int i = 1; i <= num - 1; i++) {
            if (i * (i + 1) == num) {
                isPronic = true;
                break;
            }
        }

        if (isPronic)
            System.out.println(num + " is a pronic number");
        else
            System.out.println(num + " is not a pronic number");
    }
}
```

ITERATIVE CONSTRUCTS



WRITE A PROGRAM TO INPUT A NUMBER AND CHECK WHETHER IT IS A HARSHAD NUMBER OR NOT. [A NUMBER IS SAID TO BE HARSHAD NUMBER, IF IT IS DIVISIBLE BY THE SUM OF ITS DIGITS. THE PROGRAM DISPLAYS THE MESSAGE ACCORDINGLY.]

FOR EXAMPLE;

SAMPLE INPUT: 132

SUM OF DIGITS = 6 AND 132 IS DIVISIBLE BY 6.

OUTPUT: IT IS A HARSHAD NUMBER.

SAMPLE INPUT: 353

OUTPUT: IT IS NOT A HARSHAD NUMBER.

```
import java.util.*;

public class HarshadNumber
{
    public static void main(String args[])
    {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int num = in.nextInt();
        int x = num, rem = 0, sum = 0;

        while(x > 0) {
            rem = x % 10;
            sum = sum + rem;
            x = x / 10;
        }

        int r = num % sum;

        if(r == 0)
            System.out.println(num + " is a Harshad Number.");
        else
            System.out.println(num + " is not a Harshad Number.");
    }
}
```


ITERATIVE CONSTRUCTS



WRITE A PROGRAM TO INPUT A NUMBER AND CHECK WHETHER IT IS A PRIME NUMBER OR NOT. IF IT IS NOT A PRIME NUMBER THEN DISPLAY THE NEXT NUMBER THAT IS PRIME.

SAMPLE INPUT: 14

SAMPLE OUTPUT: 17

```
import java.util.Scanner;

public class PrimeCheck
{
    public void primeCheck() {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter number: ");
        int num = in.nextInt();

        boolean isPrime = true;
        for (int i = 2; i <= num / 2; i++) {
            if (num % i == 0) {
                isPrime = false;
                break;
            }
        }

        if (isPrime) {
            System.out.println(num + " is a prime number");
        }
        else {
            for (int newNum = num + 1; newNum <= Integer.MAX_VALUE; newNum++)
            {
                isPrime = true;
                for (int i = 2; i <= newNum / 2; i++) {
                    if (newNum % i == 0) {
                        isPrime = false;
                        break;
                    }
                }
                if (isPrime) {
                    System.out.println("Next prime number = " + newNum);
                    break;
                }
            }
        }
    }
}
```

ITERATIVE CONSTRUCTS



USING A SWITCH STATEMENT, WRITE A MENU DRIVEN PROGRAM TO:

(A) GENERATE AND DISPLAY THE FIRST 10 TERMS OF THE FIBONACCI SERIES

0, 1, 1, 2, 3, 5

THE FIRST TWO FIBONACCI NUMBERS ARE 0 AND 1, AND EACH SUBSEQUENT NUMBER IS THE SUM OF THE PREVIOUS TWO.

(B) FIND THE SUM OF THE DIGITS OF AN INTEGER THAT IS INPUT BY THE USER.

SAMPLE INPUT: 15390

SAMPLE OUTPUT: SUM OF THE DIGITS = 18

FOR AN INCORRECT CHOICE, AN APPROPRIATE ERROR MESSAGE SHOULD BE DISPLAYED.

```
import java.util.Scanner;

public class FibonacciNDigitSum
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("1. Fibonacci Series");
        System.out.println("2. Sum of digits");
        System.out.print("Enter your choice: ");
        int ch = in.nextInt();

        switch (ch) {
            case 1:
                int a = 0, b = 1;
                System.out.print(a + " " + b);
                /*
                 * i is starting from 3 below
                 * instead of 1 because we have
                 * already printed 2 terms of
                 * the series. The for loop will
                 * print the series from third
                 * term onwards.
                 */
                for (int i = 3; i <= 10; i++) {
                    int term = a + b;
                    System.out.print(" " + term);
                    a = b;
                    b = term;
                }

                break;
```

ITERATIVE CONSTRUCTS



```
case 2:
    System.out.print("Enter number: ");
    int num = in.nextInt();
    int sum = 0;
    while (num != 0) {
        sum += num % 10;
        num /= 10;
    }
    System.out.println("Sum of Digits " + " = " + sum);
    break;

default:
    System.out.println("Incorrect choice");
    break;
}
```

ITERATIVE CONSTRUCTS



USING A SWITCH STATEMENT, WRITE A MENU DRIVEN PROGRAM TO:

(A) GENERATE AND DISPLAY THE FIRST 10 TERMS OF THE FIBONACCI SERIES

0, 1, 1, 2, 3, 5

THE FIRST TWO FIBONACCI NUMBERS ARE 0 AND 1, AND EACH SUBSEQUENT NUMBER IS THE SUM OF THE PREVIOUS TWO.

(B) FIND THE SUM OF THE DIGITS OF AN INTEGER THAT IS INPUT BY THE USER.

SAMPLE INPUT: 15390

SAMPLE OUTPUT: SUM OF THE DIGITS = 18

FOR AN INCORRECT CHOICE, AN APPROPRIATE ERROR MESSAGE SHOULD BE DISPLAYED.

```
import java.util.Scanner;

public class FibonacciNDigitSum
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("1. Fibonacci Series");
        System.out.println("2. Sum of digits");
        System.out.print("Enter your choice: ");
        int ch = in.nextInt();

        switch (ch) {
            case 1:
                int a = 0, b = 1;
                System.out.print(a + " " + b);
                /*
                 * i is starting from 3 below
                 * instead of 1 because we have
                 * already printed 2 terms of
                 * the series. The for loop will
                 * print the series from third
                 * term onwards.
                 */
                for (int i = 3; i <= 10; i++) {
                    int term = a + b;
                    System.out.print(" " + term);
                    a = b;
                    b = term;
                }

                break;
```


ITERATIVE CONSTRUCTS



```
case 2:
    System.out.print("Enter number: ");
    int num = in.nextInt();
    int sum = 0;
    while (num != 0) {
        sum += num % 10;
        num /= 10;
    }
    System.out.println("Sum of Digits " + " = " + sum);
    break;

default:
    System.out.println("Incorrect choice");
    break;
}
```

ITERATIVE CONSTRUCTS



USING SWITCH STATEMENT, WRITE A MENU DRIVEN PROGRAM FOR THE FOLLOWING:

(A) TO FIND AND DISPLAY THE SUM OF THE SERIES GIVEN BELOW:

$S = X^1 - X^2 + X^3 - X^4 + X^5 - \dots - X^{20}$

(B) TO FIND AND DISPLAY THE SUM OF THE SERIES GIVEN BELOW:

$S = 1/A^2 + 1/A^4 + 1/A^6 + 1/A^8 + \dots$ TO N TERMS

FOR AN INCORRECT OPTION, AN APPROPRIATE ERROR MESSAGE SHOULD BE DISPLAYED.

```
import java.util.Scanner;

public class SeriesSum
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("1. Sum of x^1 - x^2 + .... - x^20");
        System.out.println("2. Sum of 1/a^2 + 1/a^4 + .... to n terms");
        System.out.print("Enter your choice: ");
        int ch = in.nextInt();

        switch (ch) {
            case 1:
                System.out.print("Enter the value of x: ");
                int x = in.nextInt();
                long sum1 = 0;
                for (int i = 1; i <= 20; i++) {
                    int term = (int) Math.pow(x, i);
                    if (i % 2 == 0)
                        sum1 -= term;
                    else
                        sum1 += term;
                }
                System.out.println("Sum = " + sum1);
                break;
```

ITERATIVE CONSTRUCTS



```
case 2:
System.out.print("Enter the number of terms: ");
int n = in.nextInt();
System.out.print("Enter the value of a: ");
int a = in.nextInt();
int c = 2;
double sum2 = 0;
for(int i = 1; i <= n; i++) {
sum2 += (1/Math.pow(a, c));
c += 2;
}
System.out.println("Sum = " + sum2);
break;

default:
System.out.println("Incorrect choice");
break;
}
```

NESTED LOOPS



WRITE A PROGRAM IN JAVA TO ENTER A NUMBER CONTAINING THREE DIGITS OR MORE. ARRANGE THE DIGITS OF THE ENTERED NUMBER IN ASCENDING ORDER AND DISPLAY THE RESULT.
SAMPLE INPUT: ENTER A NUMBER 4972
SAMPLE OUTPUT: 2, 4, 7, 9

```
import java.util.Scanner;

public class DigitSort
{
    public void sortDigits() {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter a number having 3 or more digits: ");
        int OrgNum = in.nextInt();

        for (int i = 0; i <= 9; i++) {
            int num = OrgNum;
            int c = 0;
            while (num != 0) {
                if (num % 10 == i)
                    c++;
                num /= 10;
            }
            for (int j = 1; j <= c; j++) {
                System.out.print(i + ", ");
            }
        }

        System.out.println();
    }
}
```


NESTED LOOPS



A NUMBER IS SAID TO BE MULTIPLE HARSHAD NUMBER, WHEN DIVIDED BY THE SUM OF ITS DIGITS, PRODUCES ANOTHER 'HARSHAD NUMBER'. WRITE A PROGRAM TO INPUT A NUMBER AND CHECK WHETHER IT IS A MULTIPLE HARSHAD NUMBER OR NOT. (WHEN A NUMBER IS DIVISIBLE BY THE SUM OF ITS DIGIT, IT IS CALLED 'HARSHAD NUMBER').

SAMPLE INPUT: 6804

HINT: $6804 \Rightarrow 6+8+0+4 = 18 \Rightarrow 6804/18 = 378$

$378 \Rightarrow 3+7+8 = 18 \Rightarrow 378/18 = 21$

$21 \Rightarrow 2+1 = 3 \Rightarrow 21/3 = 7$

SAMPLE OUTPUT: MULTIPLE HARSHAD NUMBER

```
import java.util.Scanner;

public class MultipleHarshad
{
    public void checkMultipleHarshad() {

        Scanner in = new Scanner(System.in);
        System.out.print("Enter number to check: ");
        int num = in.nextInt();
        int dividend = num;
        int divisor;
        int count = 0;

        while (dividend > 1) {
            divisor=0;
            int t = dividend;
            while (t > 0) {
                int d = t % 10;
                divisor += d;
                t /= 10;
            }

            if (dividend % divisor == 0 && divisor != 1) {
                dividend = dividend / divisor;
                count++;
            }
            else {
                break;
            }
        }

        if (dividend == 1 && count > 1)
            System.out.println(num + " is Multiple Harshad Number");
        else
            System.out.println(num + " is not Multiple Harshad Number");
    }
}
```

NESTED LOOPS



A NUMBER IS SAID TO BE MULTIPLE HARSHAD NUMBER, WHEN DIVIDED BY THE SUM OF ITS DIGITS, PRODUCES ANOTHER 'HARSHAD NUMBER'. WRITE A PROGRAM TO INPUT A NUMBER AND CHECK WHETHER IT IS A MULTIPLE HARSHAD NUMBER OR NOT. (WHEN A NUMBER IS DIVISIBLE BY THE SUM OF ITS DIGIT, IT IS CALLED 'HARSHAD NUMBER').

SAMPLE INPUT: 6804

HINT: $6804 \Rightarrow 6+8+0+4 = 18 \Rightarrow 6804/18 = 378$

$378 \Rightarrow 3+7+8 = 18 \Rightarrow 378/18 = 21$

$21 \Rightarrow 2+1 = 3 \Rightarrow 21/3 = 7$

SAMPLE OUTPUT: MULTIPLE HARSHAD NUMBER

```
import java.util.Scanner;

public class MultipleHarshad
{
    public void checkMultipleHarshad() {

        Scanner in = new Scanner(System.in);
        System.out.print("Enter number to check: ");
        int num = in.nextInt();
        int dividend = num;
        int divisor;
        int count = 0;

        while (dividend > 1) {
            divisor=0;
            int t = dividend;
            while (t > 0) {
                int d = t % 10;
                divisor += d;
                t /= 10;
            }

            if (dividend % divisor == 0 && divisor != 1) {
                dividend = dividend / divisor;
                count++;
            }
            else {
                break;
            }
        }

        if (dividend == 1 && count > 1)
            System.out.println(num + " is Multiple Harshad Number");
        else
            System.out.println(num + " is not Multiple Harshad Number");
    }
}
```

NESTED LOOPS



WRITE THE PROGRAMS TO DISPLAY THE FOLLOWING PATTERNS:

(A)

1

3 1

5 3 1

7 5 3 1

9 7 5 3 1

```
public class Pattern
{
    public void displayPattern() {
        for (int i = 1; i < 10; i = i + 2) {
            for (int j = i; j > 0; j = j - 2) {
                System.out.print(j + " ");
            }
            System.out.println();
        }
    }
}
```

NESTED LOOPS



WRITE THE PROGRAMS TO DISPLAY THE FOLLOWING PATTERNS:

(B)

1 2 3 4 5

6 7 8 9

10 11 12

13 14

15

```
public class Pattern
{
    public void displayPattern() {
        int a = 1;
        for (int i = 5; i > 0; i--) {
            for (int j = 1; j <= i; j++) {
                System.out.print(a++ + "\t");
            }
            System.out.println();
        }
    }
}
```


NESTED LOOPS



WRITE THE PROGRAMS TO DISPLAY THE FOLLOWING PATTERNS:
(C)

```
15 14 13 12 11
10 9 8 7
6 5 4
3 2
1
```

```
public class Pattern
{
    public void displayPattern() {
        int a = 15;
        for (int i = 5; i > 0; i--) {
            for (int j = 1; j <= i; j++) {
                System.out.print(a-- + "\t");
            }
            System.out.println();
        }
    }
}
```

NESTED LOOPS



WRITE THE PROGRAMS TO DISPLAY THE FOLLOWING PATTERNS:

(D)
1
1 0
1 0 1
1 0 1 0
1 0 1 0 1

```
public class Pattern
{
    public void displayPattern() {
        int a = 1, b = 0;
        for (int i = 1; i <= 5; i++) {
            for (int j = 1; j <= i; j++) {
                if (j % 2 == 0)
                    System.out.print(b + " ");
                else System.out.print(a + " ");
            }
            System.out.println();
        }
    }
}
```

NESTED LOOPS



WRITE THE PROGRAMS TO DISPLAY THE FOLLOWING PATTERNS:
(G)

```
*  
* #  
* # *  
* # * #  
* # * # *
```

```
public class Pattern  
{  
    public void displayPattern() {  
        for (int i = 1; i <= 5; i++) {  
            for (int j = 1; j <= i; j++) {  
                if (j % 2 == 0)  
                    System.out.print("# ");  
                else  
                    System.out.print("* ");  
            }  
            System.out.println();  
        }  
    }  
}
```

NESTED LOOPS



USING THE SWITCH STATEMENT, WRITE A MENU DRIVEN PROGRAM FOR THE FOLLOWING:

(A) TO PRINT THE FLOYD'S TRIANGLE:

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

(B) TO DISPLAY THE FOLLOWING PATTERN:

```
I
I C
I C S
I C S E
```

FOR AN INCORRECT OPTION, AN APPROPRIATE ERROR MESSAGE SHOULD BE DISPLAYED.

```
import java.util.Scanner;

public class Pattern
{
    public void choosePattern() {
        Scanner in = new Scanner(System.in);
        System.out.println("Type 1 for Floyd's triangle");
        System.out.println("Type 2 for an ICSE pattern");

        System.out.print("Enter your choice: ");
        int ch = in.nextInt();

        switch (ch) {
            case 1:
                int a = 1;
                for (int i = 1; i <= 5; i++) {
                    for (int j = 1; j <= i; j++) {
                        System.out.print(a++ + "\t");
                    }
                    System.out.println();
                }
                break;
```


NESTED LOOPS



```
case 2:
String s = "ICSE";
for (int i = 0; i < s.length(); i++) {
for (int j = 0; j <= i; j++) {
System.out.print(s.charAt(j) + " ");
}
System.out.println();
}
break;

default:
System.out.println("Incorrect Choice");
}
```

CHARACTER MANIPULATION



WRITE A PROGRAM IN JAVA TO ACCEPT AN INTEGER NUMBER N SUCH THAT $0 < N < 27$. DISPLAY THE CORRESPONDING LETTER OF THE ALPHABET (I.E. THE LETTER AT POSITION N).
[HINT: IF N = 1 THEN DISPLAY A]

```
import java.util.Scanner;

public class Integer2Letter
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter integer: ");
        int n = in.nextInt();

        if (n > 0 && n < 27) {
            char ch = (char)(n + 64);
            System.out.println("Corresponding letter = " + ch);
        }
        else {
            System.out.println("Please enter a number in 1 to 26 range");
        }
    }
}
```

CHARACTER MANIPULATION



WRITE A PROGRAM TO INPUT A LETTER. FIND ITS ASCII CODE. REVERSE THE ASCII CODE AND DISPLAY THE EQUIVALENT CHARACTER.

SAMPLE INPUT: Y

SAMPLE OUTPUT: ASCII CODE = 89

REVERSE THE CODE = 98

EQUIVALENT CHARACTER: B

```
import java.util.Scanner;

public class ASCIIReverse
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter a letter: ");
        char l = in.next().charAt(0);

        int a = (int)l;
        System.out.println("ASCII Code = " + a);

        int r = 0;
        while (a > 0) {
            int digit = a % 10;
            r = r * 10 + digit;
            a /= 10;
        }

        System.out.println("Reversed Code = " + r);
        System.out.println("Equivalent character = " + (char)r);
    }
}
```

CHARACTER MANIPULATION



**WRITE A MENU DRIVEN PROGRAM TO DISPLAY
(I) FIRST FIVE UPPER CASE LETTERS
(II) LAST FIVE LOWER CASE LETTERS AS PER THE USER'S CHOICE.
ENTER '1' TO DISPLAY UPPER CASE LETTERS AND ENTER '2' TO
DISPLAY LOWER CASE LETTERS.**

```
import java.util.Scanner;

public class MenuUpLowerCase
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter '1' to display upper case letters");
        System.out.println("Enter '2' to display lower case letters");

        System.out.print("Enter your choice: ");
        int ch = in.nextInt();

        switch (ch) {
            case 1:
                for (int i = 65; i <= 69; i++)
                    System.out.println((char)i);
                break;

            case 2:
                for (int i = 118; i <= 122; i++)
                    System.out.println((char)i);
                break;

            default:
                break;
        }
    }
}
```


CHARACTER MANIPULATION



WRITE A PROGRAM IN JAVA TO DISPLAY THE FOLLOWING PATTERNS:

```
(I)
A
AB
ABC
ABCD
ABCDE
```

```
public class Pattern
{
    public static void main(String args[]) {
        for (int i = 65; i < 70; i++) {
            for (int j = 65; j <= i; j++) {
                if (i % 2 == 0)
                    System.out.print((char)(j+32));
                else System.out.print((char)j);
            }
            System.out.println();
        }
    }
}
```

CHARACTER MANIPULATION



WRITE A PROGRAM IN JAVA TO DISPLAY THE FOLLOWING PATTERNS:

(II)

ZYXWU

ZYXW

ZYX

ZY

Z

```
public class Pattern
{
    public static void main(String args[]) {
        for (int i = 86; i <= 90; i++) {
            for (int j = 90; j >= i; j--) {
                System.out.print((char)j);
            }
            System.out.println();
        }
    }
}
```

WRITE A PROGRAM IN JAVA TO STORE 20 NUMBERS (EVEN AND ODD NUMBERS) IN A SINGLE DIMENSIONAL ARRAY (SDA). CALCULATE AND DISPLAY THE SUM OF ALL EVEN NUMBERS AND ALL ODD NUMBERS SEPARATELY.

```
import java.util.Scanner;

public class SDAOddEvenSum
{
    public static void main(String args[]) {

        Scanner in = new Scanner(System.in);
        int arr[ ] = new int[20];

        System.out.println("Enter 20 numbers");
        for (int i = 0; i < arr.length; i++) {
            arr[i] = in.nextInt();
        }

        int oddSum = 0, evenSum = 0;

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] % 2 == 0)
                evenSum += arr[i];
            else oddSum += arr[i];
        }

        System.out.println("Sum of Odd numbers = " + oddSum);
        System.out.println("Sum of Even numbers = " + evenSum);
    }
}
```

**WRITE A PROGRAM IN JAVA TO STORE 20 TEMPERATURES IN °F IN A SINGLE DIMENSIONAL ARRAY (SDA) AND DISPLAY ALL THE TEMPERATURES AFTER CONVERTING THEM INTO °C.
HINT: $(C/5) = (F - 32) / 9$**

```
import java.util.Scanner;

public class SDAF2C
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        double arr[] = new double[20];

        System.out.println("Enter 20 temperatures in degree Fahrenheit");
        for (int i = 0; i < arr.length; i++) {
            arr[i] = in.nextDouble();
        }

        System.out.println("Temperatures in degree Celsius");
        for (int i = 0; i < arr.length; i++) {
            double tc = 5 * ((arr[i] - 32) / 9);
            System.out.println(tc);
        }
    }
}
```


WRITE A PROGRAM IN JAVA USING ARRAYS:

(A) TO STORE THE ROLL NO., NAME AND MARKS IN SIX SUBJECTS FOR 100 STUDENTS.

(B) CALCULATE THE PERCENTAGE OF MARKS OBTAINED BY EACH CANDIDATE. THE MAXIMUM MARKS IN EACH SUBJECT ARE 100.

(C) CALCULATE THE GRADE AS PER THE GIVEN CRITERIA:

PERCENTAGE	MARKS	GRADE
FROM 80 TO 100	A	
FROM 60 TO 79.	B	
FROM 40 TO 59	C	
LESS THAN 40.	D	

```
import java.util.Scanner;

public class SDAGrades
{
    public static void main(String args[]) {
        final int TOTAL_STUDENTS = 100;
        Scanner in = new Scanner(System.in);

        int rollNo[] = new int[TOTAL_STUDENTS];
        String name[] = new String[TOTAL_STUDENTS];
        int s1[] = new int[TOTAL_STUDENTS];
        int s2[] = new int[TOTAL_STUDENTS];
        int s3[] = new int[TOTAL_STUDENTS];
        int s4[] = new int[TOTAL_STUDENTS];
        int s5[] = new int[TOTAL_STUDENTS];
        int s6[] = new int[TOTAL_STUDENTS];
        double p[] = new double[TOTAL_STUDENTS];
        char g[] = new char[TOTAL_STUDENTS];

        for (int i = 0; i < TOTAL_STUDENTS; i++)
        {
            System.out.println("Enter student " + (i+1) + " details:");
            System.out.print("Roll No: ");
            rollNo[i] = in.nextInt();
            in.nextLine();
            System.out.print("Name: ");
            name[i] = in.nextLine();
            System.out.print("Subject 1 Marks: ");
            s1[i] = in.nextInt();
            System.out.print("Subject 2 Marks: ");
            s2[i] = in.nextInt();
            System.out.print("Subject 3 Marks: ");
            s3[i] = in.nextInt();
            System.out.print("Subject 4 Marks: ");
            s4[i] = in.nextInt();
            System.out.print("Subject 5 Marks: ");
            s5[i] = in.nextInt();
            System.out.print("Subject 6 Marks: ");
            s6[i] = in.nextInt();
        }
    }
}
```

WRITE A PROGRAM IN JAVA USING ARRAYS:

(A) TO STORE THE ROLL NO., NAME AND MARKS IN SIX SUBJECTS FOR 100 STUDENTS.

(B) CALCULATE THE PERCENTAGE OF MARKS OBTAINED BY EACH CANDIDATE. THE MAXIMUM MARKS IN EACH SUBJECT ARE 100.

(C) CALCULATE THE GRADE AS PER THE GIVEN CRITERIA:

PERCENTAGE	MARKS	GRADE
FROM 80 TO 100	A	
FROM 60 TO 79.	B	
FROM 40 TO 59	C	
LESS THAN 40.	D	

```
import java.util.Scanner;

public class SDAGrades
{
    public static void main(String args[]) {
        final int TOTAL_STUDENTS = 100;
        Scanner in = new Scanner(System.in);

        int rollNo[] = new int[TOTAL_STUDENTS];
        String name[] = new String[TOTAL_STUDENTS];
        int s1[] = new int[TOTAL_STUDENTS];
        int s2[] = new int[TOTAL_STUDENTS];
        int s3[] = new int[TOTAL_STUDENTS];
        int s4[] = new int[TOTAL_STUDENTS];
        int s5[] = new int[TOTAL_STUDENTS];
        int s6[] = new int[TOTAL_STUDENTS];
        double p[] = new double[TOTAL_STUDENTS];
        char g[] = new char[TOTAL_STUDENTS];

        for (int i = 0; i < TOTAL_STUDENTS; i++)
        {
            System.out.println("Enter student " + (i+1) + " details:");
            System.out.print("Roll No: ");
            rollNo[i] = in.nextInt();
            in.nextLine();
            System.out.print("Name: ");
            name[i] = in.nextLine();
            System.out.print("Subject 1 Marks: ");
            s1[i] = in.nextInt();
            System.out.print("Subject 2 Marks: ");
            s2[i] = in.nextInt();
            System.out.print("Subject 3 Marks: ");
            s3[i] = in.nextInt();
            System.out.print("Subject 4 Marks: ");
            s4[i] = in.nextInt();
            System.out.print("Subject 5 Marks: ");
            s5[i] = in.nextInt();
            System.out.print("Subject 6 Marks: ");
            s6[i] = in.nextInt();
        }
    }
}
```

```
p[i] = (((s1[i] + s2[i] + s3[i] + s4[i]
+ s5[i] + s6[i]) / 600.0) * 100);

if (p[i] < 40)
    g[i] = 'D';
else if (p[i] < 60)
    g[i] = 'C';
else if (p[i] < 80)
    g[i] = 'B';
else g[i] = 'A';
}

System.out.println();

for (int i = 0; i < TOTAL_STUDENTS; i++) {
    System.out.println(rollNo[i] + "\t"
+ name[i] + "\t"
+ p[i] + "\t"
+ g[i]);
}
}
```

WRITE A PROGRAM TO ACCEPT A LIST OF 20 INTEGERS. SORT THE FIRST 10 NUMBERS IN ASCENDING ORDER AND NEXT THE 10 NUMBERS IN DESCENDING ORDER BY USING 'BUBBLE SORT' TECHNIQUE. FINALLY, PRINT THE COMPLETE LIST OF INTEGERS.

```
import java.util.Scanner;

public class SDABubbleSort
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        int arr[] = new int[20];
        System.out.println("Enter 20 numbers:");

        for (int i = 0; i < arr.length; i++) {
            arr[i] = in.nextInt();
        }

        //Sort first half in ascending order
        for (int i = 0; i < arr.length / 2 - 1; i++) {
            for (int j = 0; j < arr.length / 2 - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int t = arr[j + 1];
                    arr[j + 1] = arr[j];
                    arr[j] = t;
                }
            }
        }

        //Sort second half in descending order
        for (int i = 0; i < arr.length / 2 - 1; i++) {
            for (int j = arr.length / 2; j < arr.length - i - 1; j++) {
                if (arr[j] < arr[j + 1]) {
                    int t = arr[j + 1];
                    arr[j + 1] = arr[j];
                    arr[j] = t;
                }
            }
        }

        //Print the final sorted array
        System.out.println("\nSorted Array:");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```


WRITE A PROGRAM IN JAVA TO ACCEPT 20 NUMBERS IN A SINGLE DIMENSIONAL ARRAY ARR[20]. TRANSFER AND STORE ALL THE EVEN NUMBERS IN AN ARRAY EVEN[] AND ALL THE ODD NUMBERS IN ANOTHER ARRAY ODD[]. FINALLY, PRINT THE ELEMENTS OF BOTH THE ARRAYS.

```
import java.util.Scanner;

public class SDAEvenOdd
{
    public static void main(String args[]) {

        final int NUM_COUNT = 20;
        Scanner in = new Scanner(System.in);
        int i = 0;

        int arr[] = new int[NUM_COUNT];
        int even[] = new int[NUM_COUNT];
        int odd[] = new int[NUM_COUNT];

        System.out.println("Enter 20 numbers:");
        for (i = 0; i < NUM_COUNT; i++) {
            arr[i] = in.nextInt();
        }

        int eldx = 0, oldx = 0;
        for (i = 0; i < NUM_COUNT; i++) {
            if (arr[i] % 2 == 0)
                even[eldx++] = arr[i];
            else odd[oldx++] = arr[i];
        }

        System.out.println("Even Numbers:");
        for (i = 0; i < eldx; i++) {
            System.out.print(even[i] + " ");
        }

        System.out.println("\nOdd Numbers:");
        for (i = 0; i < oldx; i++) {
            System.out.print(odd[i] + " ");
        }
    }
}
```

A DOUBLE DIMENSIONAL ARRAY IS DEFINED AS N[4][4] TO STORE NUMBERS. WRITE A PROGRAM TO FIND THE SUM OF ALL EVEN NUMBERS AND PRODUCT OF ALL ODD NUMBERS OF THE ELEMENTS STORED IN DOUBLE DIMENSIONAL ARRAY (DDA).

```
import java.util.Scanner;

public class DDAEvenOdd
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        int N[][] = new int[4][4];
        long evenSum = 0, oddProd = 1;
        System.out.println("Enter the elements of 4x4 DDA: ");
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 4; j++) {
                N[i][j] = in.nextInt();
                if (N[i][j] % 2 == 0)
                    evenSum += N[i][j];
                else
                    OddProd *= N[i][j];
            }
        }

        System.out.println("Sum of all even numbers = " + evenSum);
        System.out.println("Product of all odd numbers = " + oddProd);
    }
}
```



WRITE A PROGRAM TO INPUT A SENTENCE. FIND AND DISPLAY THE FOLLOWING:

(I) NUMBER OF WORDS PRESENT IN THE SENTENCE

(II) NUMBER OF LETTERS PRESENT IN THE SENTENCE

ASSUME THAT THE SENTENCE HAS NEITHER INCLUDE ANY DIGIT NOR A SPECIAL CHARACTER.

```
import java.util.Scanner;

public class WordsNLetters
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter a sentence:");
        String str = in.nextLine();

        int wCount = 0, lCount = 0;
        int len = str.length();
        for (int i = 0; i < len; i++) {
            char ch = str.charAt(i);
            if (ch == ' ')
                wCount++;
            else lCount++;
        }

        System.out.println("No. of words = " + wCount);
        System.out.println("No. of letters = " + lCount);
    }
}
```



WRITE A PROGRAM IN JAVA TO ACCEPT A WORD/A STRING AND DISPLAY THE NEW STRING AFTER REMOVING ALL THE VOWELS PRESENT IN IT.
SAMPLE INPUT: COMPUTER APPLICATIONS
SAMPLE OUTPUT: CMPTR PPLCTNS

```
import java.util.Scanner;

public class VowelRemoval
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter a word or sentence:");
        String str = in.nextLine();

        int len = str.length();
        String newStr = "";

        for (int i = 0; i < len; i++) {
            char ch = Character.toUpperCase(str.charAt(i));

            if (ch == 'A' ||
                ch == 'E' ||
                ch == 'I' ||
                ch == 'O' ||
                ch == 'U') {
                continue;
            }

            newStr = newStr + ch;
        }

        System.out.println("String with vowels removed");
        System.out.println(newStr);
    }
}
```


WRITE A PROGRAM IN JAVA TO ACCEPT A NAME(CONTAINING THREE WORDS) AND DISPLAY ONLY THE INITIALS (I.E., FIRST LETTER OF EACH WORD).

SAMPLE INPUT: LAL KRISHNA ADVANI

SAMPLE OUTPUT: L K A

```
import java.util.Scanner;

public class NameInitials
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter a name of 3 or more words:");
        String str = in.nextLine();
        int len = str.length();

        System.out.print(str.charAt(0) + " ");
        for (int i = 1; i < len; i++) {
            char ch = str.charAt(i);
            if (ch == ' ') {
                char ch2 = str.charAt(i + 1);
                System.out.print(ch2 + " ");
            }
        }
    }
}
```



WRITE A PROGRAM IN JAVA TO ACCEPT A NAME CONTAINING THREE WORDS AND DISPLAY THE SURNAME FIRST, FOLLOWED BY THE FIRST AND MIDDLE NAMES.
SAMPLE INPUT: MOHANDAS KARAMCHAND GANDHI
SAMPLE OUTPUT: GANDHI MOHANDAS KARAMCHAND

```
import java.util.Scanner;

public class SurnameFirst
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter a name of 3 words:");
        String name = in.nextLine();

        int lastSpaceldx = name.lastIndexOf(' ');

        String surname = name.substring(lastSpaceldx + 1);
        String initialName = name.substring(0, lastSpaceldx);

        System.out.println(surname + " " + initialName);
    }
}
```



WRITE A PROGRAM IN JAVA TO ENTER A STRING/SENTENCE AND DISPLAY THE LONGEST WORD AND THE LENGTH OF THE LONGEST WORD PRESENT IN THE STRING.

SAMPLE INPUT: "TATA FOOTBALL ACADEMY WILL PLAY AGAINST MOHAN BAGAN"

SAMPLE OUTPUT: THE LONGEST WORD: FOOTBALL: THE LENGTH OF THE WORD: 8

```
import java.util.Scanner;

public class LongestWord
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter a word or sentence:");
        String str = in.nextLine();

        str += " "; //Add space at end of string
        String word = "", lWord = "";
        int len = str.length();

        for (int i = 0; i < len; i++) {
            char ch = str.charAt(i);
            if (ch == ' ') {

                if (word.length() > lWord.length())
                    lWord = word;

                word = "";
            }
            else {
                word += ch;
            }
        }

        System.out.println("The longest word: " + lWord +
            ": The length of the word: " + lWord.length());
    }
}
```



WRITE A PROGRAM IN JAVA TO ACCEPT A WORD AND DISPLAY THE ASCII CODE OF EACH CHARACTER OF THE WORD.

SAMPLE INPUT: BLUEJ

SAMPLE OUTPUT:

ASCII OF B = 66

ASCII OF L = 76

ASCII OF U = 85

ASCII OF E = 69

ASCII OF J = 74

```
import java.util.Scanner;
```

```
public class LongestWord
```

```
{  
    public static void main(String args[]) {  
        Scanner in = new Scanner(System.in);  
        System.out.println("Enter a word or sentence:");  
        String str = in.nextLine();
```

```
        str += " "; //Add space at end of string  
        String word = "", lWord = "";  
        int len = str.length();
```

```
        for (int i = 0; i < len; i++) {  
            char ch = str.charAt(i);  
            if (ch == ' ')
```

```
            {  
                if (word.length() > lWord.length())  
                    lWord = word;
```

```
                word = "";  
            }  
            else {  
                word += ch;  
            }  
        }
```

```
        System.out.println("The longest word: " + lWord +  
            ": The length of the word: " + lWord.length());  
    }
```

```
}
```


WRITE A PROGRAM IN JAVA TO ACCEPT A STRING IN UPPER CASE AND REPLACE ALL THE VOWELS PRESENT IN THE STRING WITH ASTERISK (*) SIGN.

SAMPLE INPUT: "TATA STEEL IS IN JAMSHEDPUR"

SAMPLE OUTPUT: T*T* ST**L *S *N J*MSH*DP*R

```
import java.util.Scanner;

public class VowelReplace
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter a string in uppercase:");
        String str = in.nextLine();
        String newStr = "";
        int len = str.length();

        for (int i = 0; i < len; i++) {
            char ch = str.charAt(i);
            if (ch == 'A' ||
                ch == 'E' ||
                ch == 'I' ||
                ch == 'O' ||
                ch == 'U') {
                newStr = newStr + '*';
            }
            else {
                newStr = newStr + ch;
            }
        }

        System.out.println(newStr);
    }
}
```



**WRITE A PROGRAM IN JAVA TO ENTER A SENTENCE. FRAME A WORD BY JOINING ALL THE FIRST CHARACTERS OF EACH WORD OF THE SENTENCE. DISPLAY THE WORD.
SAMPLE INPUT: VITAL INFORMATION RESOURCE UNDER SEIZE
SAMPLE OUTPUT: VIRUS**

```
import java.util.Scanner;

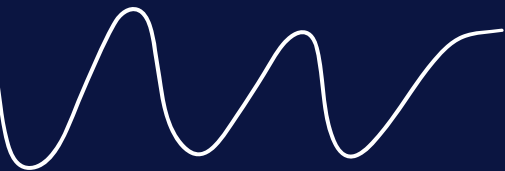
public class FrameWord
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter a sentence:");
        String str = in.nextLine();
        String word = "" + str.charAt(0);
        int len = str.length();

        for (int i = 0; i < len; i++) {
            char ch = str.charAt(i);
            if (ch == ' ')
                word += str.charAt(i + 1);
        }

        System.out.println(word);
    }
}
```



LAST 10 YEARS SOLUTIONS



2013 COMPUTER APPLICATION PAPER

SECTION A

Question 1

(a) What is meant by precedence of operators?

Answer

Precedence of operators refers to the order in which the operators are applied to the operands in an expression.

(b) What is a literal?

Answer

Literals are data items that are fixed data values. Java provides different types of literals like:

- Integer Literals
- Floating-Point Literals
- Boolean Literals
- Character Literals
- String Literals
- null Literal

(c) State the Java concept that is implemented through:

**a superclass and a subclass.
the act of representing essential features without including background details.**

Answer

- Inheritance
- Abstraction

(d) Give a difference between a constructor and a method.

Answer

Constructor has the same name as class name whereas function should have a different name than class name.

(e) What are the types of casting shown by the following examples?

```
double x = 15.2;  
int y = (int) x;
```

```
int x = 12;  
long y = x;
```

Answer

Explicit Type Casting
Implicit Type Casting

2013 COMPUTER APPLICATION PAPER

SECTION A

Question 2

(a) Name any two wrapper classes.

Answer

Two wrapper classes are:

1. Integer
2. Character

(b) What is the difference between a break statement and a continue statement when they occur in a loop?

Answer

When the break statement gets executed, it terminates its loop completely and control reaches to the statement immediately following the loop. The continue statement terminates only the current iteration of the loop by skipping rest of the statements in the body of the loop.

(c) Write statements to show how finding the length of a character array char[] differs from finding the length of a String object str.

Answer

Java code snippet to find length of character array:

```
char ch[] = {'x', 'y', 'z'};  
int len = ch.length
```

Java code snippet to find length of String object str:

```
String str = "Test";  
int len = str.length();
```

(d) Name the Java keyword that:

1. indicates that a method has no return type.
2. stores the address of the currently-calling object.

Answer

1. void
2. this

(e) What is an exception?

Answer

An exception is an abnormal condition that arises in a code sequence at run time. Exceptions indicate to a calling method that an abnormal condition has occurred.

2013 COMPUTER APPLICATION PAPER

Question 3

(a) Write a Java statement to create an object mp4 of class Digital.

Answer

```
Digital MP4 = new Digital();
```

(b) State the values stored in the variables str1 and str2:

```
String s1 = "good";  
String s2 = "world matters";  
String str1 = s2.substring(5).replace('t', 'n');  
String str2 = s1.concat(str1);
```

Answer

Value stored in str1 is " manners" and str2 is "good manners". (Note that str1 begins with a space.)

Explanation

s2.substring(5) gives a substring from index 5 till the end of the string which is " matters". The replace method replaces all 't' in " matters" with 'n' giving " manners". s1.concat(str1) joins together "good" and " manners" so "good manners" is stored in str2.

(c) What does a class encapsulate?

Answer

(d) Rewrite the following program segment using the if..else statement:

```
comm = (sale > 15000)? sale * 5 / 100 : 0;
```

Answer

```
if(sale > 15000)  
    comm = sale * 5 / 100;  
else comm = 0;
```

(e) How many times will the following loop execute? What value will be returned?

```
int x = 2, y = 50;  
do{  
    ++x;  
    y -= x++;  
}while(x <= 10);  
return y;
```

Answer

The loop will run 5 times and the value returned is 15.

2013 COMPUTER APPLICATION PAPER

(f) What is the data type that the following library functions return?

1. `isWhitespace(char ch)`
2. `Math.random()`

Answer

1. `boolean`
2. `double`

(g) Write a Java expression for:

$ut + \frac{1}{2} ft^2$

Answer

`u * t + 1 / 2.0 * f * t * t`

(h) If `int n[] = {1, 2, 3, 5, 7, 9, 13, 16}`, what are the values of `x` and `y`?

```
x = Math.pow(n[4], n[2]);  
y = Math.sqrt(n[5] + n[7]);
```

Answer

```
x = Math.pow(n[4], n[2]);  
⇒ x = Math.pow(7, 3);  
⇒ x = 343.0;  
y = Math.sqrt(n[5] + n[7]);  
⇒ y = Math.sqrt(9 + 16);  
⇒ y = Math.sqrt(25);  
⇒ y = 5.0;
```

(i) What is the final value of `ctr` when the iteration process given below, executes?

```
int ctr = 0;  
for(int i = 1; i <= 5; i++)  
    for(int j = 1; j <= 5; j += 2)  
        ++ctr;
```

Answer

The final value of `ctr` is 15.

The outer loop executes 5 times. For each iteration of outer loop, the inner loop executes 3 times. So the statement `++ctr;` is executed a total of $5 \times 3 = 15$ times.

(j) Name the method of `Scanner` class that:

1. is used to input an integer data from the standard input stream.
2. is used to input a `String` data from the standard input stream.

Answer

1. `nextInt()`
2. `nextLine()`

2013 COMPUTER APPLICATION PAPER

SECTION B

Question 4_

Define a class named FruitJuice with the following description:

Data Members.

Purpose

int product_code.

stores the product code number

String flavour.

stores the flavour of the juice (e.g., orange, apple, etc.)

String pack_type.

stores the type of packaging (e.g., tera-pack, PET bottle, etc.)

int pack_size.

stores package size (e.g., 200 mL, 400 mL, etc.)

int product_price.

stores the price of the product

Member Methods

Purpose

FruitJuice().

constructor to initialize integer data members to 0 and string data members to ""

void input().

to input and store the product code, flavour, pack type, pack size and product price

void discount().

to reduce the product price by 10

void display().

to display the product code, flavour, pack type, pack size and product price

2013 COMPUTER APPLICATION PAPER



Answer

```
import java.util.Scanner;

public class FruitJuice
{
    private int product_code;
    private String flavour;
    private String pack_type;
    private int pack_size;
    private int product_price;

    public FruitJuice() {
        product_code = 0;
        flavour = "";
        pack_type = "";
        pack_size = 0;
        product_price = 0;
    }

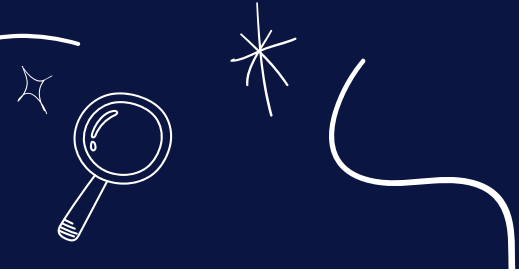
    public void input() {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter Flavour: ");
        flavour = in.nextLine();
        System.out.print("Enter Pack Type: ");
        pack_type = in.nextLine();
        System.out.print("Enter Product Code: ");
        product_code = in.nextInt();
        System.out.print("Enter Pack Size: ");
        pack_size = in.nextInt();
        System.out.print("Enter Product Price: ");
        product_price = in.nextInt();
    }

    public void discount() {
        product_price -= 10;
    }

    public void display() {
        System.out.println("Product Code: " + product_code);
        System.out.println("Flavour: " + flavour);
        System.out.println("Pack Type: " + pack_type);
        System.out.println("Pack Size: " + pack_size);
        System.out.println("Product Price: " + product_price);
    }

    public static void main(String args[]) {
        FruitJuice obj = new FruitJuice();
        obj.input();
        obj.discount();
        obj.display();
    }
}
```

2013 COMPUTER APPLICATION PAPER



Question 5

The International Standard Book Number (ISBN) is a unique numeric book identifier which is printed on every book. The ISBN is based upon a 10-digit code.

The ISBN is legal if:

$1 \times \text{digit1} + 2 \times \text{digit2} + 3 \times \text{digit3} + 4 \times \text{digit4} + 5 \times \text{digit5} + 6 \times \text{digit6} + 7 \times \text{digit7} + 8 \times \text{digit8} + 9 \times \text{digit9} + 10 \times \text{digit10}$ is divisible by 11.

Example:

For an ISBN 1401601499

Sum = $1 \times 1 + 2 \times 4 + 3 \times 0 + 4 \times 1 + 5 \times 6 + 6 \times 0 + 7 \times 1 + 8 \times 4 + 9 \times 9 + 10 \times 9 = 253$ which is divisible by 11.

Write a program to:

1. Input the ISBN code as a 10-digit integer.
2. If the ISBN is not a 10-digit integer, output the message "Illegal ISBN" and terminate the program.
3. If the number is divisible by 11, output the message "Legal ISBN". If the sum is not divisible by 11, output the message "Illegal ISBN".

Answer

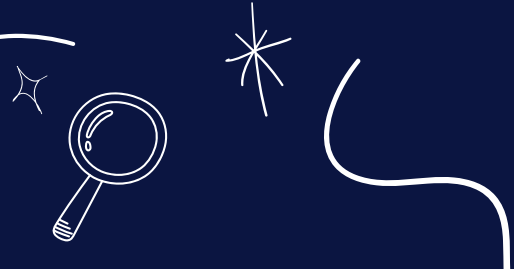
```
import java.util.Scanner;

public class ISBNCheck
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter the ISBN: ");
        long isbn = in.nextLong();

        int sum = 0, count = 0, m = 10;
        while (isbn != 0) {
            int d = (int)(isbn % 10);
            count++;
            sum += d * m;
            m--;
            isbn /= 10;
        }

        if (count != 10) {
            System.out.println("Illegal ISBN");
        }
        else if (sum % 11 == 0) {
            System.out.println("Legal ISBN");
        }
        else {
            System.out.println("Illegal ISBN");
        }
    }
}
```


2013 COMPUTER APPLICATION PAPER



Question 6

Write a program that encodes a word into Piglatin. To translate word into Piglatin word, convert the word into uppercase and then place the first vowel of the original word as the start of the new word along with the remaining alphabets. The alphabets present before the vowel being shifted towards the end followed by "AY".

Sample Input 1: London
Output: ONDONLAY

Sample Input 2: Olympics
Output: OLYMPICSA

Answer

```
import java.util.Scanner;

public class PigLatin
{
    public static void main(String args[]) {

        Scanner in = new Scanner(System.in);
        System.out.print("Enter word: ");
        String word = in.next();
        int len = word.length();

        word=word.toUpperCase();
        String piglatin="";
        int flag=0;

        for(int i = 0; i < len; i++)
        {
            char x = word.charAt(i);
            if(x=='A' || x=='E' || x=='I' || x=='O' || x=='U')
            {
                piglatin=word.substring(i) + word.substring(0,i) + "AY";
                flag=1;
                break;
            }
        }

        if(flag == 0)
        {
            piglatin = word + "AY";
        }
        System.out.println(word + " in Piglatin format is " + piglatin);
    }
}
```

2013 COMPUTER APPLICATION PAPER



Question 7

Write a program to input 10 integer elements in an array and sort them in descending order using bubble sort technique.

Answer

```
import java.util.Scanner;

public class BubbleSortDsc
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        int n = 10;
        int arr[] = new int[n];

        System.out.println("Enter the elements of the array:");
        for (int i = 0; i < n; i++) {
            arr[i] = in.nextInt();
        }

        //Bubble Sort
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] < arr[j + 1]) {
                    int t = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = t;
                }
            }
        }

        System.out.println("Sorted Array:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

2013 COMPUTER APPLICATION PAPER



Question 8

Design a class to overload a function series() as follows:

1. **double series(double n)** with one double argument and returns the sum of the series $\text{sum} = (1/1) + (1/2) + (1/3) + \dots + (1/n)$
2. **double series(double a, double n)** with two double arguments and returns the sum of the series. $\text{sum} = (1/a^2) + (4/a^3) + (7/a^8) + (10/a^{11}) + \dots$ to n terms

Answer

```
public class Series
{
    double series(double n) {
        double sum = 0;
        for (int i = 1; i <= n; i++) {
            double term = 1.0 / i;
            sum += term;
        }
        return sum;
    }

    double series(double a, double n) {
        double sum = 0;
        int x = 1;
        for (int i = 1; i <= n; i++) {
            int e = x + 1;
            double term = x / Math.pow(a, e);
            sum += term;
            x += 3;
        }
        return sum;
    }

    public static void main(String args[]) {
        KboatSeries obj = new KboatSeries();
        System.out.println("First series sum = " + obj.series(5));
        System.out.println("Second series sum = " + obj.series(3, 8));
    }
}
```

2013 COMPUTER APPLICATION PAPER



Question 9

Using the switch statement, write a menu driven program:

1. To check and display whether a number input by the user is a composite number or not. A number is said to be composite, if it has one or more than one factors excluding 1 and the number itself. Example: 4, 6, 8, 9...
2. To find the smallest digit of an integer that is input: Sample input: 6524 Sample output: Smallest digit is 2

For an incorrect choice, an appropriate error message should be displayed.

Answer

```
import java.util.Scanner;

public class Number
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("Type 1 for Composite Number");
        System.out.println("Type 2 for Smallest Digit");
        System.out.print("Enter your choice: ");
        int ch = in.nextInt();

        switch (ch) {
            case 1:
                System.out.print("Enter Number: ");
                int n = in.nextInt();
                int c = 0;
                for (int i = 1; i <= n; i++) {
                    if (n % i == 0)
                        c++;
                }

                if (c > 2)
                    System.out.println("Composite Number");
                else
                    System.out.println("Not a Composite Number");
                break;

            case 2:
                System.out.print("Enter Number: ");
                int num = in.nextInt();
                int s = 10;
                while (num != 0) {
                    int d = num % 10;
                    if (d < s)
                        s = d;
                    num /= 10;
                }
                System.out.println("Smallest digit is " + s);
                break;

            default:
                System.out.println("Wrong choice");
        }
    }
}
```

2014 COMPUTER APPLICATION PAPER

SECTION A

Question 1

(a) Which of the following are valid comments ?

1. `/*comment*/`
2. `/*comment`
3. `//comment`
4. `*/comment*/`

Answer

The valid comments are:

Option 1 — `/*comment*/`

Option 3 — `//comment`

(b) What is meant by a package ? Name any two java Application Programming Interface packages.

Answer

A package is a named collection of Java classes that are grouped on the basis of their functionality. Two java Application Programming Interface packages are:

1. `java.util`
2. `java.io`

(c) Name the primitive data type in Java that is:

1. a 64-bit integer and is used when you need a range of values wider than those provided by `int`.
2. a single 16-bit Unicode character whose default value is `'\u0000'`.

Answer

1. `long`
2. `char`

(d) State one difference between the floating point literals `float` and `double`.

Answer

Float literals	double literals
float literals have a size of 32 bits so they can store a fractional number with around 6-7 total digits of precision.	double literals have a size of 64 bits so they can store a fractional number with 15-16 total digits of precision.

2014 COMPUTER APPLICATION PAPER

(e) Find the errors in the given program segment and re-write the statements correctly to assign values to an integer array.

```
int a = new int (5);
for (int i=0; i<=5; i++) a[i]=i;
```

Answer

Corrected Code:

```
int a[] = new int[5];
for (int i = 0; i < 5; i++)
    a[i] = i;
```

Question 2

(a) Operators with higher precedence are evaluated before operators with relatively lower precedence. Arrange the operators given below in order of higher precedence to lower precedence:

1. &&
2. %
3. >=
4. ++

Answer

```
++
%
>=
&&
```

(b) Identify the statements listed below as assignment, increment, method invocation or object creation statements.

1. `System.out.println("Java");`
2. `costPrice = 457.50;`
3. `Car hybrid = new Car();`
4. `petrolPrice++;`

Answer

1. Method Invocation
2. Assignment
3. Object Creation
4. Increment

(c) Give two differences between the switch statement and the if-else statement

switch	if-else
- switch can only test if the expression is equal to any of its case constants - It is a multiple branching flow of control statement	- if-else can test for any boolean expression like less than, greater than, equal to, not equal to, etc. - It is a bi-directional flow of control statement

2014 COMPUTER APPLICATION PAPER

(d) What is an infinite loop? Write an infinite loop statement.

Answer

A loop which continues iterating indefinitely and never stops is termed as infinite loop. Below is an example of infinite loop:

```
for (;;)
    System.out.println("Infinite Loop");
```

(e) What is constructor? When is it invoked?

Answer

A constructor is a member method that is written with the same name as the class name and is used to initialize the data members or instance variables. A constructor does not have a return type. It is invoked at the time of creating any object of the class.

Question 3

(a) List the variables from those given below that are composite data types:

1. **static int x;**
2. **arr[i]=10;**
3. **obj.display();**
4. **boolean b;**
5. **private char chr;**
6. **String str;**

Answer

The composite data types are:

- arr[i]=10;
- obj.display();
- String str;

(b) State the output of the following program segment:

```
String str1 = "great"; String str2 = "minds";
System.out.println(str1.substring(0,2).concat(str2.substring(1)));
System.out.println(("WH" + (str1.substring(2).toUpperCase())));
```

Answer

The output of the above code is:

```
grinds
WHEAT
```

Explanation:

1. str1.substring(0,2) returns "gr". str2.substring(1) returns "inds". Due to concat both are joined together to give the output as "grinds".
2. str1.substring(2) returns "eat". It is converted to uppercase and joined to "WH" to give the output as "WHEAT".

2014 COMPUTER APPLICATION PAPER

(c) What are the final values stored in variable x and y below?

```
double a = -6.35;
double b = 14.74;
double x = Math.abs(Math.ceil(a));
double y = Math rint(Math.max(a,b));
```

Answer

The final value stored in variable x is 6.0 and y is 15.0.

Explanation:

1. Math.ceil(a) gives -6.0 and Math.abs(-6.0) is 6.0.
2. Math.max(a,b) gives 14.74 and Math.rint(14.74) gives 15.0.

(d) Rewrite the following program segment using if-else statements instead of the ternary operator:

```
String grade = (marks >= 90) ? "A" : (marks >= 80) ? "B" : "C";
```

Answer

```
String grade;
if (marks >= 90)
    grade = "A";
else if (marks >= 80)
    grade = "B";
else grade = "C";
```

(e) Give the output of the following method:

```
public static void main (String [] args){
    int a = 5;
    a++;
    System.out.println(a);
    a -= (a--) - (--a);
    System.out.println(a);}
Answer
```

Output

6

4

Explanation

1. a++ increments value of a by 1 so a becomes 6.
2. a -= (a--) - (--a)
 - ⇒ a = a - ((a--) - (--a))
 - ⇒ a = 6 - (6 - 4)
 - ⇒ a = 6 - 2
 - ⇒ a = 4

2014 COMPUTER APPLICATION PAPER

(f) What is the data type returned by the library functions :

1. `compareTo()`
2. `equals()`

Answer

1. `int`
2. `boolean`

(g) State the value of characteristic and mantissa when the following code is executed:

```
String s = "4.3756";  
int n = s.indexOf('.');  
int characteristic=Integer.parseInt(s.substring(0,n));  
int mantissa=Integer.valueOf(s.substring(n+1));
```

Answer

The value of characteristic is 4 and mantissa is 3756.

Index of . in String s is 1 so variable n is initialized to 1. `s.substring(0,n)` gives 4. `Integer.parseInt()` converts string "4" into integer 4 and this 4 is assigned to the variable characteristic.

`s.substring(n+1)` gives 3756 i.e. the string starting at index 2 till the end of s. `Integer.valueOf()` converts string "3756" into integer 3756 and this value is assigned to the variable mantissa.

(h) Study the method and answer the given questions.

```
public void sampleMethod()  
{  
    for (int i=0; i < 3; i++)  
    {  
        for (int j = 0; j<2; j++)  
        {  
            int number = (int)(Math.random() * 10);  
            System.out.println(number);  
        }  
    }  
}
```

1. How many times does the loop execute?
2. What is the range of possible values stored in the variable number?

Answer

1. The loops execute 6 times.
2. The range is from 0 to 9.

2014 COMPUTER APPLICATION PAPER



(i) Consider the following class:

```
public class myClass {  
    public static int x=3, y=4;  
    public int a=2, b=3;}  
}
```

1. **Name the variables for which each object of the class will have its own distinct copy.**
2. **Name the variables that are common to all objects of the class.**

Answer

1. a, b
2. x, y

(j) What will be the output when the following code segments are executed?

(i)

```
String s = "1001";  
int x = Integer.valueOf(s);  
double y = Double.valueOf(s);  
System.out.println("x="+x);  
System.out.println("y="+y);
```

(ii)

```
System.out.println("The king said \"Begin at the beginning!\" t
```

Answer

(i) The output of the code is:

x=1001

y=1001.0

(ii) The output of the code is:

The king said "Begin at the beginning!" to me.

2014 COMPUTER APPLICATION PAPER

SECTION B

Question 4_

Define a class named movieMagic with the following description:

Data Members.

int year

String title

float rating

(.

Purpose

To store the year of release of a movie

To store the title of the movie

To store the popularity rating of the movie

minimum rating=0.0 and maximum rating=5.0

Member Methods.

movieMagic().

Purpose

Default constructor to initialize numeric data members to 0 and String data member to "".

void accept().

To input and store year, title and rating

void display().

To display the title of the movie and a message based on the rating as per the table given below

Ratings table

Rating.

Message to be displayed

0.0 to 2.0

Flop

2.1 to 3.4.

Semi-Hit

3.5 to 4.4.

Hit

4.5 to 5.0.

Super-Hit

Write a main method to create an object of the class and call the above member methods.

Answer

```
import java.util.Scanner;

public class movieMagic
{
    private int year;
    private String title;
    private float rating;

    public movieMagic() {
        year = 0;
        title = "";
        rating = 0.0f;
    }

    public void accept() {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter Title of Movie: ");
        title = in.nextLine();
        System.out.print("Enter Year of Movie: ");
        year = in.nextInt();
        System.out.print("Enter Rating of Movie: ");
        rating = in.nextFloat();
    }
}
```

2014 COMPUTER APPLICATION PAPER

```
public void display() {
    String message = "Invalid Rating";
    if (rating <= 2.0f)
        message = "Flop";
    else if (rating <= 3.4f)
        message = "Semi-Hit";
    else if (rating <= 4.4f)
        message = "Hit";
    else if (rating <= 5.0f)
        message = "Super-Hit";

    System.out.println(title);
    System.out.println(message);
}

public static void main(String args[]) {
    movieMagic obj = new movieMagic();
    obj.accept();
    obj.display();
}
}
```

Question 5

A special two-digit number is such that when the sum of its digits is added to the product of its digits, the result is equal to the original two-digit number.

Example: Consider the number 59.

Sum of digits = $5 + 9 = 14$

Product of digits = $5 * 9 = 45$

Sum of the sum of digits and product of digits = $14 + 45 = 59$

Write a program to accept a two-digit number. Add the sum of its digits to the product of its digits. If the value is equal to the number input, then display the message "Special two—digit number" otherwise, display the message "Not a special two-digit number".

2014 COMPUTER APPLICATION PAPER

Answer

```
import java.util.Scanner;

public class SpecialNumber
{
    public void checkNumber() {

        Scanner in = new Scanner(System.in);

        System.out.print("Enter a 2 digit number: ");
        int orgNum = in.nextInt();

        int num = orgNum;
        int count = 0, digitSum = 0, digitProduct = 1;

        while (num != 0) {
            int digit = num % 10;
            num /= 10;
            digitSum += digit;
            digitProduct *= digit;
            count++;
        }

        if (count != 2)
            System.out.println("Invalid input, please enter a 2-digit number");
        else if ((digitSum + digitProduct) == orgNum)
            System.out.println("Special 2-digit number");
        else System.out.println("Not a special 2-digit number");

    }
}
```

Question 6

Write a program to assign a full path and file name as given below. Using library functions, extract and output the file path, file name and file extension separately as shown.

Input

C:\Users\admin\Pictures\flower.jpg

Output

Path: C:\Users\admin\Pictures\

File name: flower

Extension: jpg

2014 COMPUTER APPLICATION PAPER

Answer

```
import java.util.Scanner;

public class FilepathSplit
{
    public static void main(String args[]) {

        Scanner in = new Scanner(System.in);
        System.out.print("Enter full path: ");
        String filepath = in.next();

        char pathSep = '\\';
        char dotSep = '.';

        int pathSepIdx = filepath.lastIndexOf(pathSep);
        System.out.println("Path:\t\t" + filepath.substring(0, pathSepIdx));

        int dotIdx = filepath.lastIndexOf(dotSep);
        System.out.println("File Name:\t" + filepath.substring(pathSepIdx + 1,
        dotIdx));

        System.out.println("Extension:\t" + filepath.substring(dotIdx + 1));
    }
}
```

Question 7

Design a class to overload a function area() as follows:

- 1. double area (double a, double b, double c) with three double arguments, returns the area of a scalene triangle using the formula: $\text{area} = \sqrt{s(s-a)(s-b)(s-c)}$ where $s = (a+b+c) / 2$**
- 2. double area (int a, int b, int height) with three integer arguments, returns the area of a trapezium using the formula: $\text{area} = (1/2)\text{height}(a + b)$**
- 3. double area (double diagonal1, double diagonal2) with two double arguments, returns the area of a rhombus using the formula: $\text{area} = 1/2(\text{diagonal1} \times \text{diagonal2})$**

2014 COMPUTER APPLICATION PAPER

Answer

```
import java.util.Scanner;

public class KboatOverload
{
    double area(double a, double b, double c) {
        double s = (a + b + c) / 2;
        double x = s * (s-a) * (s-b) * (s-c);
        double result = Math.sqrt(x);
        return result;
    }

    double area (int a, int b, int height) {
        double result = (1.0 / 2.0) * height * (a + b);
        return result;
    }

    double area (double diagonal1, double diagonal2) {
        double result = 1.0 / 2.0 * diagonal1 * diagonal2;
        return result;
    }
}
```

Question 8

Using the switch statement, write a menu driven program to calculate the maturity amount of a Bank Deposit.

The user is given the following options:

1. Term Deposit
2. Recurring Deposit

For option 1, accept principal (P), rate of interest(r) and time period in years(n). Calculate and output the maturity amount(A) receivable using the formula:

$$A = P[1 + r / 100]^n$$

For option 2, accept Monthly Installment (P), rate of interest (r) and time period in months (n). Calculate and output the maturity amount (A) receivable using the formula:

$$A = P \times n + P \times (n(n+1) / 2) \times r / 100 \times 1 / 12$$

For an incorrect option, an appropriate error message should be displayed.

2014 COMPUTER APPLICATION PAPER

Answer

```
import java.util.Scanner;

public class BankDeposit
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("Type 1 for Term Deposit");
        System.out.println("Type 2 for Recurring Deposit");
        System.out.print("Enter your choice: ");
        int ch = in.nextInt();
        double p = 0.0, r = 0.0, a = 0.0;
        int n = 0;

        switch (ch) {
            case 1:
                System.out.print("Enter Principal: ");
                p = in.nextDouble();
                System.out.print("Enter Interest Rate: ");
                r = in.nextDouble();
                System.out.print("Enter time in years: ");
                n = in.nextInt();
                a = p * Math.pow(1 + r / 100, n);
                System.out.println("Maturity amount = " + a);
                break;

            case 2:
                System.out.print("Enter Monthly Installment: ");
                p = in.nextDouble();
                System.out.print("Enter Interest Rate: ");
                r = in.nextDouble();
                System.out.print("Enter time in months: ");
                n = in.nextInt();
                a = p * n + p * ((n * (n + 1)) / 2) * (r / 100) * (1 / 12.0);
                System.out.println("Maturity amount = " + a);
                break;

            default:
                System.out.println("Invalid choice");
        }
    }
}
```

2014 COMPUTER APPLICATION PAPER

Question 9

Write a program to accept the year of graduation from school as an integer value from the user. Using the binary search technique on the sorted array of integers given below, output the message "Record exists" if the value input is located in the array. If not, output the message "Record does not exist".

Sample Input:

n[0].	1982
n[1].	1987
n[2].	1993
n[3].	1996
n[4].	1999
n[5].	2003
n[6].	2006
n[7].	2007
n[8].	2009
n[9].	2010

Answer

```
import java.util.Scanner;

public class GraduationYear
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        int n[] = {1982, 1987, 1993, 1996, 1999, 2003, 2006, 2007, 2009, 2010};

        System.out.print("Enter graduation year to search: ");
        int year = in.nextInt();

        int l = 0, h = n.length - 1, idx = -1;
        while (l <= h) {
            int m = (l + h) / 2;
            if (n[m] == year) {
                idx = m;
                break;
            }
            else if (n[m] < year) {
                l = m + 1;
            }
            else {
                h = m - 1;
            }
        }

        if (idx == -1)
            System.out.println("Record does not exist");
        else
            System.out.println("Record exists");
    }
}
```

2015 COMPUTER APPLICATION PAPER

SECTION A

(a) What are the default values of the primitive data type int and float?

Answer

Default value of int is 0 and float is 0.0f.

(b) Name any two OOP's principles.

Answer

1. Encapsulation
2. Inheritance

(c) What are identifiers ?

Answer

Identifiers are symbolic names given to different parts of a program such as variables, methods, classes, objects, etc.

(d) Identify the literals listed below:

(i) 0.5 (ii) 'A' (iii) false (iv) "a"

Answer

1. Real Literal
2. Character Literal
3. Boolean Literal
4. String Literal

(e) Name the wrapper classes of char type and boolean type.

Answer

Wrapper class of char type is Character and boolean type is Boolean.

2015 COMPUTER APPLICATION PAPER



(a) Evaluate the value of n if value of p = 5, q = 19
`int n = (q - p) > (p - q) ? (q - p) : (p - q);`

Answer

```
int n = (q - p) > (p - q) ? (q - p) : (p - q)
⇒ int n = (19 - 5) > (5 - 19) ? (19 - 5) : (5 - 19)
⇒ int n = 14 > -14 ? 14 : -14
⇒ int n = 14 [∵ 14 > -14 is true so ? : returns 14]
Final value of n is 14
```

(b) Arrange the following primitive data types in an ascending order of their size:

(i) char (ii) byte (iii) double (iv) int

Answer

byte, char, int, double

(c) What is the value stored in variable res given below:
`double res = Math.pow ("345".indexOf('5'), 3);`

Answer

Value of res is 8.0
Index of '5' in "345" is 2. Math.pow(2, 3) means 2³ which is equal to 8

(d) Name the two types of constructors

Answer

Parameterized constructors and Non-Parameterized constructors.

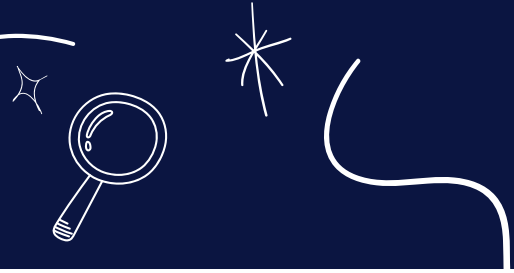
(e) What are the values of a and b after the following function is executed, if the values passed are 30 and 50:

```
void paws(int a, int b)
{
    a=a+b;
    b=a-b;
    a=a-b;
    System.out.println(a+ "," +b);
}
```

Answer

Value of a is 50 and b is 30.
Output of the code is:
50,30

2015 COMPUTER APPLICATION PAPER



(e) (i) Name the mathematical function which is used to find sine of an angle given in radians.

(ii) Name a string function which removes the blank spaces provided in the prefix and suffix of a string.

Answer

(i) Math.sin() (ii) trim()

(f) (i) What will this code print ?

```
int arr[]=new int[5];  
System.out.println(arr);
```

1. 0
2. value stored in arr[0]
3. 0000
4. garbage value

Answer

This code will print garbage value

(ii) Name the keyword which is used- to resolve the conflict between method parameter and instance variables/fields.

Answer

this keyword

(g) State the package that contains the class:

1. **BufferedReader**
2. **Scanner**

Answer

1. java.io
2. java.util

(h) Write the output of the following program code:

```
char ch ;  
int x=97;  
do  
{  
    ch=(char)x;  
    System.out.print(ch + " " );  
    if(x%10 == 0)  
        break;  
    ++x;  
} while(x<=100);
```

Answer

The output of the above code is:

a b c d

2015 COMPUTER APPLICATION PAPER



(a) State the data type and value of y after the following is executed:
`char x='7';`
`y=Character.isLetter(x);`

Answer

Data type of y is boolean and value is false. `Character.isLetter()` method checks if its argument is a letter or not and as 7 is a number not a letter hence it returns false.

(b) What is the function of catch block in exception handling ? Where does it appear in a program ?

Answer

Catch block contains statements that we want to execute in case an exception is thrown in the try block. Catch block appears immediately after the try block in a program.

(c) State the output when the following program segment is executed:
`String a ="Smartphone", b="Graphic Art";`
`String h=a.substring(2, 5);`
`String k=b.substring(8).toUpperCase();`
`System.out.println(h);`
`System.out.println(k.equalsIgnoreCase(h));`

Answer

The output of the above code is:
art
true

Explanation

`a.substring(2, 5)` extracts the characters of string a starting from index 2 till index 4. So h contains the string "art". `b.substring(8)` extracts characters of b from index 8 till the end so it returns "Art". `toUpperCase()` converts it into uppercase. Thus string "ART" gets assigned to k. Values of h and k are "art" and "ART", respectively. As both h and k differ in case only hence `equalsIgnoreCase` returns true.

(d) The access specifier that gives the most accessibility is _____ and the least accessibility is _____.

Answer

The access specifier that gives the most accessibility is public and the least accessibility is private.

2015 COMPUTER APPLICATION PAPER



(i) Write the Java expressions for:

$a^2 + b^2 / 2ab$

Answer

$(a * a + b * b) / (2 * a * b)$

(j) If `int y = 10` then find `int z = (++y * (y++ + 5));`

Answer

$z = (++y * (y++ + 5))$
 $\Rightarrow z = (11 * (11 + 5))$
 $\Rightarrow z = (11 * 16)$
 $\Rightarrow z = 176$

SECTION B

Question 4

Define a class called `ParkingLot` with the following description:

Instance variables/data members:

`int vno` — To store the vehicle number.

`int hours` — To store the number of hours the vehicle is parked in the parking lot.

`double bill` — To store the bill amount.

Member Methods:

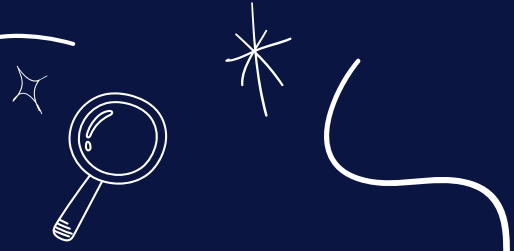
`void input()` — To input and store the `vno` and `hours`.

`void calculate()` — To compute the parking charge at the rate of ₹3 for the first hour or part thereof, and ₹1.50 for each additional hour or part thereof.

`void display()` — To display the detail.

Write a `main()` method to create an object of the class and call the above methods.

2015 COMPUTER APPLICATION PAPER



Answer

```
import java.util.Scanner;

public class ParkingLot
{
    private int vno;
    private int hours;
    private double bill;

    public void input() {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter vehicle number: ");
        vno = in.nextInt();
        System.out.print("Enter hours: ");
        hours = in.nextInt();
    }

    public void calculate() {
        if (hours <= 1)
            bill = 3;
        else bill = 3 + (hours - 1) * 1.5;
    }

    public void display() {
        System.out.println("Vehicle number: " + vno);
        System.out.println("Hours: " + hours);
        System.out.println("Bill: " + bill);
    }

    public static void main(String args[]) {
        ParkingLot obj = new ParkingLot();
        obj.input();
        obj.calculate();
        obj.display();
    }
}
```

2015 COMPUTER APPLICATION PAPER



Question 5

Write two separate programs to generate the following patterns using iteration (loop) statements:

(a)

```
*
* #
* # *
* # * #
* # * # *
```

(b)

```
5 4 3 2 1
5 4 3 2
5 4 3
5 4
5
```

Answer

```
public class Pattern
{
    public static void main(String args[]) {
        for (int i = 1; i <= 5; i++) {
            for (int j = 1; j <= i; j++) {
                if (j % 2 == 0)
                    System.out.print("# ");
                else System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```

2015 COMPUTER APPLICATION PAPER

(B)

Answer

```
public class Pattern
{
    public static void main(String args[]) {
        for (int i = 1; i <=5; i++) {
            for (int j = 5; j >= i; j--) {
                System.out.print(j + " ");
            }
            System.out.println();
        }
    }
}
```

Question 6

Write a program to input and store roll numbers, names and marks in 3 subjects of n number of students in five single dimensional arrays and display the remark based on average marks as given below:

Average Marks.

85 — 100.

75 — 84.

60 — 74.

40 — 59.

Less than 40.

Remarks

Excellent

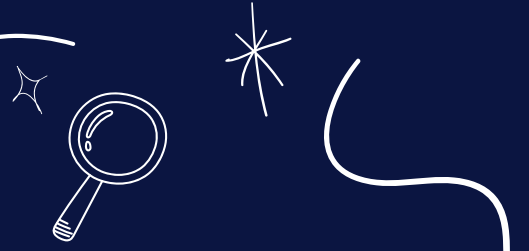
Distinction

First Class

Pass

Poor

2015 COMPUTER APPLICATION PAPER



Answer

```
import java.util.Scanner;

public class AvgMarks
{
    public static void main(String args[]) {

        Scanner in = new Scanner(System.in);
        System.out.print("Enter number of students: ");
        int n = in.nextInt();

        int rollNo[] = new int[n];
        String name[] = new String[n];
        int s1[] = new int[n];
        int s2[] = new int[n];
        int s3[] = new int[n];
        double avg[] = new double[n];

        for (int i = 0; i < n; i++) {
            System.out.println("Enter student " + (i+1) + " details:");
            System.out.print("Roll No: ");
            rollNo[i] = in.nextInt();
            in.nextLine();
            System.out.print("Name: ");
            name[i] = in.nextLine();
            System.out.print("Subject 1 Marks: ");
            s1[i] = in.nextInt();
            System.out.print("Subject 2 Marks: ");
            s2[i] = in.nextInt();
            System.out.print("Subject 3 Marks: ");
            s3[i] = in.nextInt();
            avg[i] = (s1[i] + s2[i] + s3[i]) / 3.0;
        }

        System.out.println("Roll No\tName\tRemark");
        for (int i = 0; i < n; i++) {
            String remark;
            if (avg[i] < 40)
                remark = "Poor";
            else if (avg[i] < 60)
                remark = "Pass";
            else if (avg[i] < 75)
                remark = "First Class";
            else if (avg[i] < 85)
                remark = "Distinction";
            else remark = "Excellent";
            System.out.println(rollNo[i] + "\t"
                + name[i] + "\t"
                + remark);
        }
    }
}
```

2015 COMPUTER APPLICATION PAPER

Question 7

Design a class to overload a function Joysting() as follows:

- void Joysting(String s, char ch1, char ch2) with one string argument and two character arguments that replaces the character argument ch1 with the character argument ch2 in the given String s and prints the new string.

Example:

Input value of s = "TECHNALAGY"

ch1 = 'A'

ch2 = 'O'

Output: "TECHNOLOGY"

- void Joysting(String s) with one string argument that prints the position of the first space and the last space of the given String s.

Example:

Input value of s = "Cloud computing means Internet based computing"

Output:

First index: 5

Last Index: 36

- void Joysting(String s1, String s2) with two string arguments that combines the two strings with a space between them and prints the resultant string.

Example:

Input value of s1 = "COMMON WEALTH"

Input value of s2 = "GAMES"

Output: COMMON WEALTH GAMES

(Use library functions)

2015 COMPUTER APPLICATION PAPER



Answer

```
public class StringOverload
{
    public void joysting(String s, char ch1, char ch2) {
        String newStr = s.replace(ch1, ch2);
        System.out.println(newStr);
    }

    public void joysting(String s) {
        int f = s.indexOf(' ');
        int l = s.lastIndexOf(' ');
        System.out.println("First index: " + f);
        System.out.println("Last index: " + l);
    }

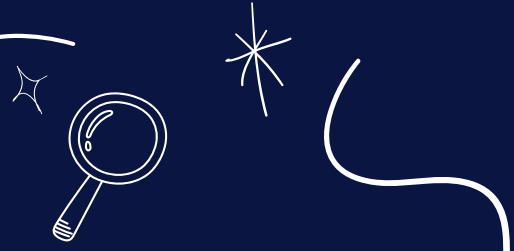
    public void joysting(String s1, String s2) {
        String newStr = s1.concat(" ").concat(s2);
        System.out.println(newStr);
    }

    public static void main(String args[]) {
        KboatStringOverload obj = new KboatStringOverload();
        obj.joysting("TECHNALAGY", 'A', 'O');
        obj.joysting("Cloud computing means Internet based computing");
        obj.joysting("COMMON WEALTH", "GAMES");
    }
}
```

Question 8

Write a program to input twenty names in an array. Arrange these names in descending order of letters, using the bubble sort technique.

2015 COMPUTER APPLICATION PAPER



Answer

```
import java.util.Scanner;

public class ArrangeNames
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        String names[] = new String[20];
        System.out.println("Enter 20 names:");
        for (int i = 0; i < names.length; i++) {
            names[i] = in.nextLine();
        }

        //Bubble Sort
        for (int i = 0; i < names.length - 1; i++) {
            for (int j = 0; j < names.length - 1 - i; j++) {
                if (names[j].compareToIgnoreCase(names[j + 1]) < 0) {
                    String temp = names[j + 1];
                    names[j + 1] = names[j];
                    names[j] = temp;
                }
            }
        }

        System.out.println("\nSorted Names");
        for (int i = 0; i < names.length; i++) {
            System.out.println(names[i]);
        }
    }
}
```

Question 9

Using switch statement, write a menu driven program to:

(i) To find and display all the factors of a number input by the user (including 1 and the excluding the number itself).

Example:

Sample Input : n = 15

Sample Output : 1, 3, 5

(ii) To find and display the factorial of a number input by the user (the factorial of a non-negative integer n, denoted by n!, is the product of all integers less than or equal to n.)

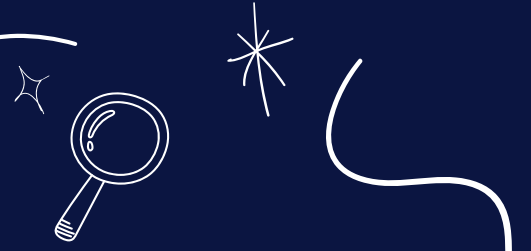
Example:

Sample Input : n = 5

Sample Output : 5! = 1*2*3*4*5 = 120

For an incorrect choice, an appropriate error message should be displayed.

2015 COMPUTER APPLICATION PAPER



Answer

```
import java.util.Scanner;

public class Menu
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("1. Factors of number");
        System.out.println("2. Factorial of number");
        System.out.print("Enter your choice: ");
        int choice = in.nextInt();
        int num;

        switch (choice) {
            case 1:
                System.out.print("Enter number: ");
                num = in.nextInt();
                for (int i = 1; i < num; i++) {
                    if (num % i == 0) {
                        System.out.print(i + " ");
                    }
                }
                System.out.println();
                break;

            case 2:
                System.out.print("Enter number: ");
                num = in.nextInt();
                int f = 1;
                for (int i = 1; i <= num; i++)
                    f *= i;
                System.out.println("Factorial = " + f);
                break;

            default:
                System.out.println("Incorrect Choice");
                break;
        }
    }
}
```


2016 COMPUTER APPLICATION PAPER

SECTION A

Question 1

(a) Define Encapsulation.

Answer

Encapsulation is a mechanism that binds together code and the data it manipulates. It keeps them both safe from the outside world, preventing any unauthorised access or misuse. Only member methods, which are wrapped inside the class, can access the data and other methods.

(b) What are keywords ? Give an example.

Answer

Keywords are reserved words that have a special meaning to the Java compiler. Example: class, public, int, etc.

(c) Name any two library packages.

Answer

Two library packages are:

1. java.io
2. java.util

(d) Name the type of error (syntax, runtime or logical error) in each case given below:

1. `Math.sqrt (36 – 45)`
2. `int a;b;c;`

Answer

1. Runtime Error
2. Syntax Error

(e) If `int x[] = { 4, 3, 7, 8, 9, 10};` what are the values of p and q ?

1. `p = x.length`
2. `q = x[2] + x[5] * x[1]`

Answer

1. 6
2. $q = x[2] + x[5] * x[1]$
 $q = 7 + 10 * 3$
 $q = 7 + 30$
 $q = 37$

2016 COMPUTER APPLICATION PAPER

(a) State the difference between == operator and equals() method.

Answer

equals()	==
- It is a method - It is used to check if the contents of two strings are same or not Example: <code>String s1 = new String("hello"); String s2 = new String("hello"); boolean res = s1.equals(s2); System.out.println(res);</code> - The output of this code snippet is true as contents of s1 and s2 are the same.	- It is a relational operator - It is used to check if two variables refer to the same object in memory - Example: <code>String s1 = new String("hello"); String s2 = new String("hello"); boolean res = s1 == s2; System.out.println(res);</code> - The output of this code snippet is false as s1 and s2 point to different String objects.

(b) What are the types of casting shown by the following examples:

- `char c = (char) 120;`
- `int x = 't';`

Answer

- Explicit Cast.
- Implicit Cast.

(c) Differentiate between formal parameter and actual parameter.

Answer

Formal parameter	Actual parameter
- Formal parameters appear in function definition. - They represent the values received by the called function.	- Actual parameters appear in function call statement. - They represent the values passed to the called function.

2016 COMPUTER APPLICATION PAPER



**(d) Write a function prototype of the following:
A function PosChar which takes a string argument and a character argument and returns an integer value.**

Answer

```
int PosChar(String s, char ch)
```

(e) Name any two types of access specifiers.

Answer

1. public
2. private

Question 3

(a) Give the output of the following string functions:

1. "MISSISSIPPI".indexOf('S') + "MISSISSIPPI".lastIndexOf('I')
2. "CABLE".compareTo("CADET")

Answer

1. $2 + 10 = 12$
2. ASCII Code of B - ASCII Code of D
 $\Rightarrow 66 - 68$
 $\Rightarrow -2$

(b) Give the output of the following Math functions:

1. `Math.ceil(4.2)`
2. `Math.abs(-4)`

Answer

1. 5.0
2. 4

(c) What is a parameterized constructor ?

Answer

A Parameterised constructor receives parameters at the time of creating an object and initializes the object with the received values.

2016 COMPUTER APPLICATION PAPER



(d) Write down java expression for:

Answer

$$T = \sqrt{A^2 + B^2 + C^2}$$

```
T = Math.sqrt(a*a + b*b + c*c)
```

(e) Rewrite the following using ternary operator:

if (x%2 == 0)

System.out.print("EVEN");

else System.out.print("ODD");

Answer

```
System.out.print(x%2 == 0 ? "EVEN" : "ODD");
```

(f) Convert the following while loop to the corresponding for loop:

```
int m = 5, n = 10;
```

```
while (n>=1)
```

```
{  
    System.out.println(m*n);  
    n--;
```

```
}
```

Answer

```
int m = 5;
```

```
for (int n = 10; n >= 1; n--) {  
    System.out.println(m*n);
```

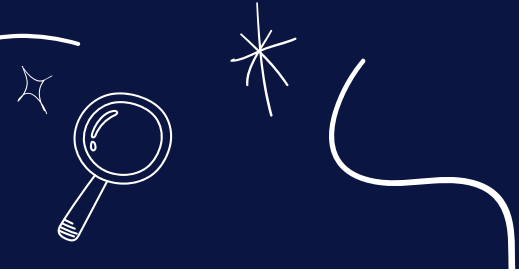
```
}
```

(g) Write one difference between primitive data types and composite data types.

Answer

Primitive Data Types are built-in data types defined by Java language specification whereas Composite Data Types are defined by the programmer.

2016 COMPUTER APPLICATION PAPER



(h) Analyze the given program segment and answer the following questions:

1. Write the output of the program segment.

2. How many times does the body of the loop gets executed ?

```
for(int m = 5; m <= 20; m += 5)
{
    if(m % 3 == 0)
        break;
    elseif(m % 5 == 0)
        System.out.println(m);
    continue;
}
```

Answer

Output

5
10
Loop executes 3 times

(i) Give the output of the following expression:

$a += a++ + ++a + --a + a--$; when $a = 7$

Answer

$a += a++ + ++a + --a + a--$
 $\Rightarrow a = a + (a++ + ++a + --a + a--)$
 $\Rightarrow a = 7 + (7 + 9 + 8 + 8)$
 $\Rightarrow a = 7 + 32$
 $\Rightarrow a = 39$

(j) Write the return type of the following library functions:

1. isLetterOrDigit(char)

2. replace(char, char)

Answer

1. boolean
2. String

2016 COMPUTER APPLICATION PAPER



(h) Analyze the given program segment and answer the following questions:

1. Write the output of the program segment.

2. How many times does the body of the loop gets executed ?

```
for(int m = 5; m <= 20; m += 5)
{
    if(m % 3 == 0)
        break;
    elseif(m % 5 == 0)
        System.out.println(m);
    continue;
}
```

Answer

Output

5
10
Loop executes 3 times

(i) Give the output of the following expression:

$a += a++ + ++a + --a + a--$; when $a = 7$

Answer

$a += a++ + ++a + --a + a--$
 $\Rightarrow a = a + (a++ + ++a + --a + a--)$
 $\Rightarrow a = 7 + (7 + 9 + 8 + 8)$
 $\Rightarrow a = 7 + 32$
 $\Rightarrow a = 39$

(j) Write the return type of the following library functions:

1. isLetterOrDigit(char)

2. replace(char, char)

Answer

1. boolean
2. String

2016 COMPUTER APPLICATION PAPER

SECTION B

Question 4

Define a class named BookFair with the following description:

Instance variables/Data members:

String Bname — stores the name of the book

double price — stores the price of the book

Member methods:

(i) BookFair() — Default constructor to initialize data members

(ii) void Input() — To input and store the name and the price of the book.

(iii) void calculate() — To calculate the price after discount.

Discount is calculated based on the following criteria.

Price.

Less than or equal to ₹1000.

More than ₹1000 and less than or equal to ₹3000.

More than ₹3000.

Discount

2% of price

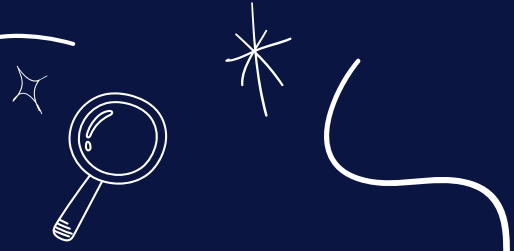
10% of price

15% of price

(iv) void display() — To display the name and price of the book after discount.

Write a main method to create an object of the class and call the above member methods.

2016 COMPUTER APPLICATION PAPER



```
import java.util.Scanner;

public class BookFair
{
    private String bname;
    private double price;

    public BookFair() {
        bname = "";
        price = 0.0;
    }

    public void input() {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter name of the book: ");
        bname = in.nextLine();
        System.out.print("Enter price of the book: ");
        price = in.nextDouble();
    }

    public void calculate() {
        double disc;
        if (price <= 1000)
            disc = price * 0.02;
        else if (price <= 3000)
            disc = price * 0.1;
        elsedisc = price * 0.15;

        price -= disc;
    }

    public void display() {
        System.out.println("Book Name: " + bname);
        System.out.println("Price after discount: " + price);
    }

    public static void main(String args[]) {
        BookFair obj = new BookFair();
        obj.input();
        obj.calculate();
        obj.display();
    }
}
```

2016 COMPUTER APPLICATION PAPER



Question 5

Using the switch statement, write a menu driven program for the following:

(i) To print the Floyd's triangle [Given below]

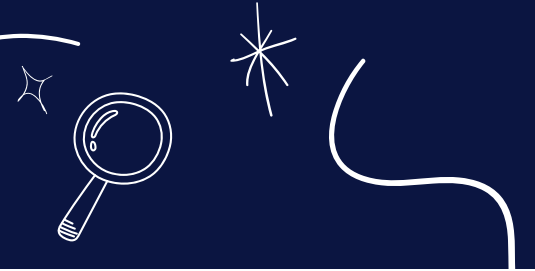
```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

(b) To display the following pattern:

```
I
I C
I C S
I C S E
```

For an incorrect option, an appropriate error message should be displayed.

2016 COMPUTER APPLICATION PAPER



```
import java.util.Scanner;
```

```
public class Pattern  
{
```

```
    public static void main(String args[]) {  
        Scanner in = new Scanner(System.in);  
        System.out.println("Type 1 for Floyd's triangle");  
        System.out.println("Type 2 for an ICSE pattern");
```

```
  
        System.out.print("Enter your choice: ");  
        int ch = in.nextInt();
```

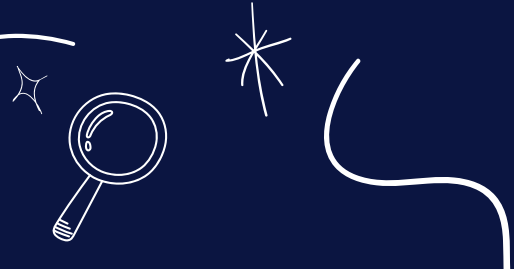
```
  
        switch (ch) {  
            case 1:  
                int a = 1;  
                for (int i = 1; i <= 5; i++) {  
                    for (int j = 1; j <= i; j++) {  
                        System.out.print(a++ + "\t");  
                    }  
                    System.out.println();  
                }  
                break;
```

```
  
            case 2:  
                String s = "ICSE";  
                for (int i = 0; i < s.length(); i++) {  
                    for (int j = 0; j <= i; j++) {  
                        System.out.print(s.charAt(j) + " ");  
                    }  
                    System.out.println();  
                }  
                break;
```

```
  
            default:  
                System.out.println("Incorrect Choice");
```

```
        }  
    }  
}
```


2016 COMPUTER APPLICATION PAPER



Question 6

Special words are those words which start and end with the same letter.

Example: EXISTENCE, COMIC, WINDOW

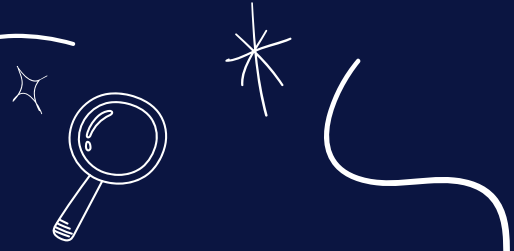
Palindrome words are those words which read the same from left to right and vice-versa.

Example: MALYALAM, MADAM, LEVEL, ROTATOR, CIVIC

All palindromes are special words but all special words are not palindromes.

Write a program to accept a word. Check and display whether the word is a palindrome or only a special word or none of them.

2016 COMPUTER APPLICATION PAPER



Answer

```
import java.util.Scanner;

public class SpecialPalindrome
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter a word: ");
        String str = in.next();
        str = str.toUpperCase();
        int len = str.length();

        if (str.charAt(0) == str.charAt(len - 1)) {
            boolean isPalin = true;
            for (int i = 1; i < len / 2; i++) {
                if (str.charAt(i) != str.charAt(len - 1 - i)) {
                    isPalin = false;
                    break;
                }
            }

            if (isPalin) {
                System.out.println("Palindrome");
            }
            else {
                System.out.println("Special");
            }
        }
        else {
            System.out.println("Neither Special nor Palindrome");
        }
    }
}
```

2016 COMPUTER APPLICATION PAPER



question7

Design a class to overload a function `sumSeries()` as follows:

(i) `void sumSeries(int n, double x)`: with one integer argument and one double argument to find and display the sum of the series given below:

$s = x/1 - x/2 + x/3 - x/4 + x/5 \dots \dots \dots$ to n terms

(ii) `void sumSeries()`: to find and display the sum of the following series:

$s = 1 + (1 \times 2) + (1 \times 2 \times 3) + \dots \dots \dots + (1 \times 2 \times 3 \times 4 \dots \dots \dots \times 20)$
 $s = 1 + (1 \times 2) + (1 \times 2 \times 3) + \dots \dots \dots + (1 \times 2 \times 3 \times 4 \dots \dots \dots \times 20)$

Answer

```
public class Overload
{
    void sumSeries(int n, double x) {
        double sum = 0;
        for (int i = 1; i <= n; i++) {
            double t = x / i;
            if (i % 2 == 0)
                sum -= t;
            else sum += t;
        }
        System.out.println("Sum = " + sum);
    }

    void sumSeries() {
        long sum = 0, term = 1;
        for (int i = 1; i <= 20; i++) {
            term *= i;
            sum += term;
        }
        System.out.println("Sum=" + sum);
    }
}
```

2016 COMPUTER APPLICATION PAPER



Question 8

Design a class to overload a function `sumSeries()` as follows:

(i) `void sumSeries(int n, double x)`: with one integer argument and one double argument to find and display the sum of the series given below:

$s = x/1 - x/2 + x/3 - x/4 + x/5 \dots \dots \dots$ to n terms

(ii) `void sumSeries()`: to find and display the sum of the following series:

$s = 1 + (1 \times 2) + (1 \times 2 \times 3) + \dots \dots \dots + (1 \times 2 \times 3 \times 4 \dots \dots \dots \times 20)$
 $s = 1 + (1 \times 2) + (1 \times 2 \times 3) + \dots \dots \dots + (1 \times 2 \times 3 \times 4 \dots \dots \dots \times 20)$

Answer

```
import java.util.Scanner;
```

```
public class NivenNumber
{
```

```
    public void checkNiven() {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter number: ");
        int num = in.nextInt();
        int orgNum = num;
```

```
        int digitSum = 0;
```

```
        while (num != 0) {
            int digit = num % 10;
            num /= 10;
            digitSum += digit;
        }
```

```
        if (digitSum != 0 && orgNum % digitSum == 0)
            System.out.println(orgNum + " is a Niven number");
        else
            system.out.println(orgNum + " is not a Niven number");
```

```
    }
}
```

2016 COMPUTER APPLICATION PAPER



Question 9

Write a program to initialize the seven Wonders of the World along with their locations in two different arrays. Search for a name of the country input by the user. If found, display the name of the country along with its Wonder, otherwise display "Sorry not found!".

Seven Wonders:

CHICHEN ITZA, CHRIST THE REDEEMER, TAJ MAHAL, GREAT WALL OF CHINA, MACHU PICCHU, PETRA, COLOSSEUM

Locations:

MEXICO, BRAZIL, INDIA, CHINA, PERU, JORDAN, ITALY

Examples:

Country name: INDIA

Output: TAJ MAHAL

Country name: USA

Output: Sorry Not found!

Answer

```
import java.util.Scanner;

public class 7Wonders
{
    public static void main(String args[]) {

        String wonders[] = {"CHICHEN ITZA", "CHRIST THE REDEEMER", "TAJMAHAL",
            "GREAT WALL OF CHINA", "MACHU PICCHU", "PETRA", "COLOSSEUM"};

        String locations[] = {"MEXICO", "BRAZIL", "INDIA", "CHINA", "PERU", "JORDAN",
            "ITALY"};

        Scanner in = new Scanner(System.in);
        System.out.print("Enter country: ");
        String c = in.nextLine();
        int i;

        for (i = 0; i < locations.length; i++) {
            if (locations[i].equals(c)) {
                System.out.println(locations[i] + " - " + wonders[i]);
                break;
            }
        }

        if (i == locations.length)
            System.out.println("Sorry Not Found!");
    }
}
```


2017 COMPUTER APPLICATION PAPER

SECTION A

Question 1

(a) What is inheritance?

Answer

Inheritance is the mechanism by which a class acquires the properties and methods of another class.

(b) Name the operators listed below:

1. <
2. ++
3. &&
4. ? :

Answer

1. Less than operator. (It is a relational operator)
2. Increment operator. (It is an arithmetic operator)
3. Logical AND operator. (It is a logical operator)
4. Ternary operator. (It is a conditional operator)

(c) State the number of bytes occupied by char and int data types.

Answer

char occupies 2 bytes and int occupies 4 bytes.

(d) Write one difference between / and % operator.

Answer

/ operator computes the quotient whereas % operator computes the remainder.

(e) String x[] = {"SAMSUNG", "NOKIA", "SONY", "MICROMAX", "BLACKBERRY"};

Give the output of the following statements:

1. `System.out.println(x[1]);`
2. `System.out.println(x[3].length());`

Answer

1. The output of `System.out.println(x[1]);` is NOKIA. `x[1]` gives the second element of the array x which is "NOKIA"
2. The output of `System.out.println(x[3].length());` is 8. `x[3].length()` gives the number of characters in the fourth element of the array x which is MICROMAX.

2017 COMPUTER APPLICATION PAPER

Question 2

(a) Name the following:

1. A keyword used to call a package in the program.
2. Any one reference data type.

Answer

1. import
2. Array

(b) What are the two ways of invoking functions?

Answer

Call by value and call by reference.

(c) State the data type and value of res after the following is executed:

```
char ch='t';  
res= Character.toUpperCase(ch);
```

Answer

Data type of res is char and its value is T.

(d) Give the output of the following program segment and also mention the number of times the loop is executed:

```
int a,b;  
for (a = 6, b = 4; a <= 24; a = a + 6)  
{  
    if (a%b == 0)  
        break;  
}  
System.out.println(a);
```

Answer

Output of the above code is 12 and loop executes 2 times.

(e) Write the output:

```
char ch = 'F';  
int m = ch;  
m=m+5;  
System.out.println(m + " " + ch);
```

Answer

Output of the above code is:

75 F

Explanation

The statement `int m = ch;` assigns the ASCII value of F (which is 70) to variable m. Adding 5 to m makes it 75. In the `println` statement, the current value of m which is 75 is printed followed by space and then value of ch which is F.

2017 COMPUTER APPLICATION PAPER

Question 3

(a) Write a Java expression for the following:

$ax^5 + bx^3 + c$

Answer

`a * Math.pow(x, 5) + b * Math.pow(x, 3) + c`

(b) What is the value of x1 if x=5?

`x1 = ++x - x++ + --x`

Answer

`x1 = ++x - x++ + --x`
 $\Rightarrow x1 = 6 - 6 + 6$
 $\Rightarrow x1 = 0 + 6$
 $\Rightarrow x1 = 6$

(c) Why is an object called an instance of a class?

Answer

A class can create objects of itself with different characteristics and common behaviour. So, we can say that an Object represents a specific state of the class. For these reasons, an Object is called an Instance of a Class.

(d) Convert following do-while loop into for loop.

```
int i = 1;
int d = 5;
do {
    d = d * 2;
    System.out.println(d);
    i++; } while ( i <= 5);
```

Answer

```
int i = 1;
int d = 5;
for (i = 1; i <= 5; i++) {
    d = d * 2;
    System.out.println(d);
}
```

(e) Differentiate between constructor and function.

Answer

Constructor	Function
• Constructor is a block of code that initializes a newly created object. • Constructor has the same name as class name. • Constructor has no return type not even void.	• Function is a group of statements that can be called at any point in the program using its name to perform a specific task. • Function should have a different name than class name. • Function requires a valid return type.

2017 COMPUTER APPLICATION PAPER



(f) Write the output for the following:

```
String s="Today is Test";  
System.out.println(s.indexOf('T'));  
System.out.println(s.substring(0,7) + " " + "Holiday");
```

Answer

Output of the above code is:

0

Today i Holiday

Explanation

As the first T is present at index 0 in string s so s.indexOf('T') returns 0. s.substring(0,7) extracts the characters from index 0 to index 6 of string s. So its result is Today i. After that println statement prints Today i followed by a space and Holiday

(g) What are the values stored in variables r1 and r2:

1. **double r1 = Math.abs(Math.min(-2.83, -5.83));**
2. **double r2 = Math.sqrt(Math.floor(16.3));**

Answer

1.r1 has 5.83

2.r2 has 4.0

Explanation

- 1.Math.min(-2.83, -5.83) returns -5.83 as -5.83 is less than -2.83. (Note that these are negative numbers). Math.abs(-5.83) returns 5.83.
- 2.Math.floor(16.3) returns 16.0. Math.sqrt(16.0) gives square root of 16.0 which is 4.0.

(h) Give the output of the following code:

```
String A ="26", B="100";  
String D=A+B+"200";  
int x= Integer.parseInt(A);  
int y = Integer.parseInt(B);  
int d = x+y;  
System.out.println("Result 1 = " + D);  
System.out.println("Result 2 = " + d);
```

Answer

Output of the above code is:

Result 1 = 26100200

Result 2 = 126

Explanation

- 1.As A and B are strings so String D=A+B+"200"; joins A, B and "200" storing the string "26100200" in D.
- 2.Integer.parseInt() converts strings A and B into their respective numeric values. Thus, x becomes 26 and y becomes 100.
- 3.As Java is case-sensitive so D and d are treated as two different variables.
- 4.Sum of x and y is assigned to d. Thus, d gets a value of 126.

2017 COMPUTER APPLICATION PAPER



(i) Analyze the given program segment and answer the following questions:

```
for(int i=3;i<=4;i++ ) {  
for(int j=2;j<i;j++ ) {  
System.out.print(" " ); }  
System.out.println("WIN" ); }
```

1. How many times does the inner loop execute?
2. Write the output of the program segment.

Answer

1. Inner loop executes 3 times. (For 1st iteration of outer loop, inner loop executes once. For 2nd iteration of outer loop, inner loop executes two times.)
2. Output:
WIN
WIN

(j) What is the difference between the Scanner class functions next() and nextLine()?

Answer

next()	nextLine
• It reads the input only till space so it can read only a single word. • It places the cursor in the same line after reading the input.	• It reads the input till the end of line so it can read a full sentence including spaces. • It places the cursor in the next line after reading the input.

2017 COMPUTER APPLICATION PAPER

SECTION B

Question 4 Define a class ElectricBill with the following specifications:

class : ElectricBill

Instance variables / data member:

String n — to store the name of the customer

int units — to store the number of units consumed

double bill — to store the amount to be paid

Member methods:

void accept() — to accept the name of the customer and number of units consumed

void calculate() — to calculate the bill as per the following tariff:

Number of units.	Rate per unit
First 100 units.	Rs.2.00
Next 200 units.	Rs.3.00
Above 300 units.	Rs.5.00

A surcharge of 2.5% charged if the number of units consumed is above 300 units.

void print() — To print the details as follows:

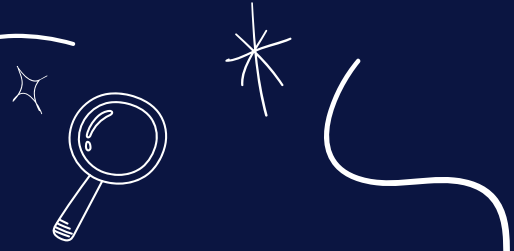
Name of the customer:

Number of units consumed:

Bill amount:

Write a main method to create an object of the class and call the above member methods.

2017 COMPUTER APPLICATION PAPER



Answer

```
import java.util.Scanner;

public class ElectricBill
{
    private String n;
    private int units;
    private double bill;

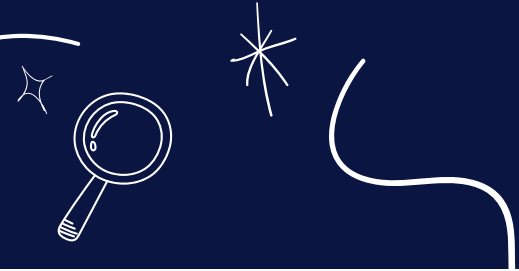
    public void accept() {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter customer name: ");
        n = in.nextLine();
        System.out.print("Enter units consumed: ");
        units = in.nextInt();
    }

    public void calculate() {
        if (units <= 100)
            bill = units * 2;
        else if (units <= 300)
            bill = 200 + (units - 100) * 3;
        else {
            double amt = 200 + 600 + (units - 300) * 5;
            double surcharge = (amt * 2.5) / 100.0;
            bill = amt + surcharge;
        }
    }

    public void print() {
        System.out.println("Name of the customer\t\t: " + n);
        System.out.println("Number of units consumed\t\t: " + units);
        System.out.println("Bill amount\t\t\t: " + bill);
    }

    public static void main(String args[]) {
        ElectricBill obj = new ElectricBill();
        obj.accept();
        obj.calculate();
        obj.print();
    }
}
```

2017 COMPUTER APPLICATION PAPER



Question 5

Write a program to accept a number and check and display whether it is a spy number or not. (A number is spy if the sum of its digits equals the product of its digits.)

Example: consider the number 1124.

Sum of the digits = $1 + 1 + 2 + 4 = 8$

Product of the digits = $1 \times 1 \times 2 \times 4 = 8$

Answer

```
import java.util.Scanner;

public class SpyNumber
{
    public static void main(String args[]) {

        Scanner in = new Scanner(System.in);

        System.out.print("Enter Number: ");
        int num = in.nextInt();

        int digit, sum = 0;
        int orgNum = num;
        int prod = 1;

        while (num > 0) {
            digit = num % 10;

            sum += digit;
            prod *= digit;
            num /= 10;
        }

        if (sum == prod)
            System.out.println(orgNum + " is Spy Number");
        else
            System.out.println(orgNum + " is not Spy Number");
    }
}
```

2017 COMPUTER APPLICATION PAPER

Question 6

Using switch statement, write a menu driven program for the following:

1. To find and display the sum of the series given below:

$$S = x^1 - x^2 + x^3 - x^4 + x^5 \dots - x^{20} \text{ (where } x = 2)$$

To display the following series: 1 11 111 1111 11111

For an incorrect option, an appropriate error message should be displayed.

Answer

```
import java.util.Scanner;

public class KboatSeriesMenu
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("1. Sum of the series");
        System.out.println("2. Display series");
        System.out.print("Enter your choice: ");
        int choice = in.nextInt();

        switch (choice) {
            case 1:
                int sum = 0;
                for (int i = 1; i <= 20; i++) {
                    int x = 2;
                    int term = (int) Math.pow(x, i);
                    if (i % 2 == 0)
                        sum -= term;
                    else
                        sum += term;
                }
                System.out.println("Sum=" + sum);
                break;

            case 2:
                int term = 1;
                for (int i = 1; i <= 5; i++) {
                    System.out.print(term + " ");
                    term = term * 10 + 1;
                }
                break;

            default:
                System.out.println("Incorrect Choice");
                break;
        }
    }
}
```

2017 COMPUTER APPLICATION PAPER



Question 7

Write a program to input integer elements into an array of size 20 and perform the following operations:

- 1. Display largest number from the array.**
- 2. Display smallest number from the array.**
- 3. Display sum of all the elements of the array**

Answer

```
import java.util.Scanner;

public class KboatSDAMinMaxSum
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        int arr[] = new int[20];
        System.out.println("Enter 20 numbers:");
        for (int i = 0; i < 20; i++) {
            arr[i] = in.nextInt();
        }
        int min = arr[0], max = arr[0], sum = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] < min)
                min = arr[i];

            if (arr[i] > max)
                max = arr[i];

            sum += arr[i];
        }

        System.out.println("Largest Number = " + max);
        System.out.println("Smallest Number = " + min);
        System.out.println("Sum = " + sum);
    }
}
```


2017 COMPUTER APPLICATION PAPER

Question8

Design a class to overload a function check() as follows:

void check (String str , char ch) — to find and print the frequency of a character in a string.Example:

Input:

str = "success"

ch = 's'

Output:

number of s present is = 3

void check(String s1) — to display only vowels from string s1, after converting it to lower case.Example:

Input:

s1 ="computer"

Output : o u e

Answer

```
public class Overload
{
    void check (String str , char ch ) {
        int count = 0;
        int len = str.length();
        for (int i = 0; i < str.length(); i++) {
            char c = str.charAt(i);
            if (ch == c) {
                count++;
            }
        }
        System.out.println("Frequency of " + ch + " = " + count);
    }

    void check(String s1) {
        String s2 = s1.toLowerCase();
        int len = s2.length();
        System.out.println("Vowels:");
        for (int i = 0; i < len; i++) {
            char ch = s2.charAt(i);
            if (ch == 'a' ||
                ch == 'e' ||
                ch == 'i' ||
                ch == 'o' ||
                ch == 'u')
                System.out.print(ch + " ");
        }
    }
}
```

2017 COMPUTER APPLICATION PAPER



Question 9

Write a program to input forty words in an array. Arrange these words in descending order of alphabets, using selection sort technique. Print the sorted array.

Answer

```
import java.util.Scanner;

public class SelectionSort
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        String a[] = new String[40];
        int n = a.length;

        System.out.println("Enter 40 Names: ");
        for (int i = 0; i < n; i++) {
            a[i] = in.nextLine();
        }

        for (int i = 0; i < n - 1; i++) {
            int idx = i;
            for (int j = i + 1; j < n; j++) {
                if (a[j].compareToIgnoreCase(a[idx]) < 0) {
                    idx = j;
                }
            }
            String t = a[idx];
            a[idx] = a[i];
            a[i] = t;
        }

        System.out.println("Sorted Names");
        for (int i = 0; i < n; i++) {
            System.out.println(a[i]);
        }
    }
}
```

2018 COMPUTER APPLICATION PAPER

SECTION A

Question 1

(a) Define abstraction.

Answer

Abstraction refers to the act of representing essential features without including the background details or explanation.

(b) Differentiate between searching and sorting.

Answer

Searching	Sorting
• Searching means to search for a term or value in an array. • Linear search and Binary search are examples of search techniques.	• Sorting means to arrange the elements of the array in ascending or descending order. • Bubble sort and Selection sort are examples of sorting techniques.

(c) Write a difference between the functions `isUpperCase( )` and `toUpperCase( )`.

Answer

<code>isUpperCase()</code>	<code>toUpperCase()</code>
• <code>isUpperCase()</code> function checks if a given character is in uppercase or not. • Its return type is boolean.	• <code>toUpperCase()</code> function converts a given character to uppercase. • Its return type is char.

2018 COMPUTER APPLICATION PAPER

(d) How are private members of a class different from public members?

Answer

private members of a class can only be accessed by other member methods of the same class whereas public members of a class can be accessed by methods of other classes as well.

(e) Classify the following as primitive or non-primitive datatypes:

1. **char**
2. **arrays**
3. **int**
4. **classes**

Answer

1. Primitive
2. Non-Primitive
3. Primitive
4. Non-Primitive

Question 2

(a) (i) `int res = 'A';`

What is the value of `res`?

Answer

Value of `res` is 65 which is the ASCII code of A. As `res` is an `int` variable and we are trying to assign it the character A so through implicit type conversion the ASCII code of A is assigned to `res`.

(ii) Name the package that contains wrapper classes.

Answer

`java.lang`

(b) State the difference between `while` and `do while` loop.

Answer

while	do-while
- It is an entry-controlled loop. - It is helpful in situations where numbers of iterations are not known.	- it is an exit-controlled loop. - It is suitable when we need to display a menu to the user.

2018 COMPUTER APPLICATION PAPER

(c) `System.out.print("BEST ");`
`System.out.println("OF LUCK");`

Choose the correct option for the output of the above statements

1. BEST OF LUCK
2. BEST
3. OF LUCK

Answer

Option 1 — BEST OF LUCK is the correct option.

`System.out.print` does not print a newline at the end of its output so the `println` statement begins printing on the same line. So the output is BEST OF LUCK printed on a single line.

(d) Write the prototype of a function check which takes an integer as an argument and returns a character.

Answer

`char check(int a)`

(e) Write the return data type of the following function.

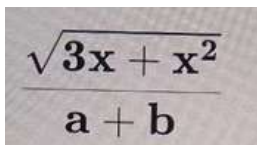
1. `endsWith()`
2. `log()`

Answer

1. boolean
2. double

Question 3

(a) Write a Java expression for the following:



$$\frac{\sqrt{3x + x^2}}{a + b}$$

Answer

`Math.sqrt(3 * x + x * x) / (a + b)`

(b) What is the value of y after evaluating the expression given below?

`y += ++y + y-- + --y;` when `int y=8`

Answer

`y += ++y + y-- + --y`
 $\Rightarrow y = y + (++y + y-- + --y)$
 $\Rightarrow y = 8 + (9 + 9 + 7)$
 $\Rightarrow y = 8 + 25$
 $\Rightarrow y = 33$

2018 COMPUTER APPLICATION PAPER

(c) Give the output of the following:

1. `Math.floor (-4.7)`
2. `Math.ceil(3.4) + Math.pow(2, 3)`

Answer

1. -5.0
2. 12.0

(d) Write two characteristics of a constructor.

Answer

Two characteristics of a constructor are:

1. A constructor has the same name as its class.
2. A constructor does not have a return type.

(e) Write the output for the following:

`System.out.println("Incredible"+"\\n"+"world");`

Answer

Output of the above code is:

Incredible
world

\\n is the escape sequence for newline so Incredible and world are printed on two different lines.

(f) Convert the following if else if construct into switch case

```
if( var==1)
    System.out.println("good");
else if(var==2)
    System.out.println("better");
else if(var==3)
    System.out.println("best");
else System.out.println("invalid");
```

Answer

```
switch (var) {
    case 1:
        System.out.println("good");
        break;

    case 2:
        System.out.println("better");
        break;

    case 3:
        System.out.println("best");
        break;

    default:
        System.out.println("invalid");
}
```

2018 COMPUTER APPLICATION PAPER

(g) Give the output of the following string functions:

1. "ACHIEVEMENT".replace('E', 'A')
2. "DEDICATE".compareTo("DEVOTE")

Answer

1. ACHIAVAMANT
2. -18

Explanation

1. "ACHIEVEMENT".replace('E', 'A') will replace all the E's in ACHIEVEMENT with A's.
2. The first letters that are different in DEDICATE and DEVOTE are D and V respectively. So the output will be: ASCII code of D - ASCII code of V
⇒ 68 - 86
⇒ -18

(h) Consider the following String array and give the output
String arr[]= {"DELHI", "CHENNAI", "MUMBAI", "LUCKNOW", "JAIPUR"};

System.out.println(arr[0].length() > arr[3].length());
System.out.print(arr[4].substring(0,3));

Answer

Output of the above code is:

false

JAI

Explanation

1. arr[0] is DELHI and arr[3] is LUCKNOW. Length of DELHI is 5 and LUCKNOW is 7. As 5 > 7 is false so the output is false.
2. arr[4] is JAIPUR. arr[4].substring(0,3) extracts the characters from index 0 till index 2 of JAIPUR so JAI is the output.

(i) Rewrite the following using ternary operator:
if (bill > 10000)

discount = bill * 10.0/100;

elsediscount = bill * 5.0/100;

Answer

discount = bill > 10000 ? bill * 10.0/100 : bill * 5.0/100;

2018 COMPUTER APPLICATION PAPER



(j) Give the output of the following program segment and also mention how many times the loop is executed:

```
int i;  
for ( i = 5 ; i > 10; i ++ )  
System.out.println( i );  
System.out.println( i * 4 );
```

Answer

Output of the above code is:

20

Loop executes 0 times.

Explanation

In the loop, i is initialized to 5 but the condition of the loop is $i > 10$. As 5 is less than 10 so the condition of the loop is false and it is not executed. There are no curly braces after the loop which means that the statement `System.out.println( i );` is inside the loop and the statement `System.out.println( i * 4 );` is outside the loop. Loop is not executed so `System.out.println( i );` is not executed but `System.out.println( i * 4 );` being outside the loop is executed and prints the output as 20 ($i * 4 \Rightarrow 5 * 4 \Rightarrow 20$).

2018 COMPUTER APPLICATION PAPER

SECTION B

Question 4

Design a class **RailwayTicket** with following description:

Instance variables/data members:

String name — To store the name of the customer

String coach — To store the type of coach customer wants to travel

long mobno — To store customer's mobile number

int amt — To store basic amount of ticket

int totalamt — To store the amount to be paid after updating the original amount

Member methods:

void accept() — To take input for name, coach, mobile number and amount.

void update() — To update the amount as per the coach selected (extra amount to be added in the amount as follows)

Type of Coaches.	Amount
First_AC.	700
Second_AC.	500
Third_AC.	250
sleeper.	None

void display() — To display all details of a customer such as name, coach, total amount and mobile number.

Write a main method to create an object of the class and call the above member methods.

2018 COMPUTER APPLICATION PAPER

Answer

```
import java.util.Scanner;

public class RailwayTicket
{
    private String name;
    private String coach;
    private long mobno;
    private int amt;
    private int totalamt;

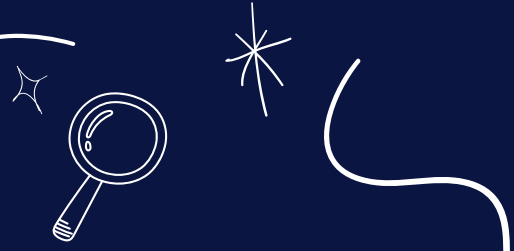
    private void accept() {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter name: ");
        name = in.nextLine();
        System.out.print("Enter coach: ");
        coach = in.nextLine();
        System.out.print("Enter mobile no: ");
        mobno = in.nextLong();
        System.out.print("Enter amount: ");
        amt = in.nextInt();
    }

    private void update() {
        if(coach.equalsIgnoreCase("First_AC"))
            totalamt = amt + 700;
        else if(coach.equalsIgnoreCase("Second_AC"))
            totalamt = amt + 500;
        else if(coach.equalsIgnoreCase("Third_AC"))
            totalamt = amt + 250;
        else if(coach.equalsIgnoreCase("Sleeper"))
            totalamt = amt;
    }

    private void display() {
        System.out.println("Name: " + name);
        System.out.println("Coach: " + coach);
        System.out.println("Total Amount: " + totalamt);
        System.out.println("Mobile number: " + mobno);
    }

    public static void main(String args[]) {
        RailwayTicket obj = new RailwayTicket();
        obj.accept();
        obj.update();
        obj.display();
    }
}
```


2018 COMPUTER APPLICATION PAPER



Question 5

Write a program to input a number and check and print whether it is a Pronic number or not. (Pronic number is the number which is the product of two consecutive integers)

Examples:

$$12 = 3 \times 4$$

$$20 = 4 \times 5$$

$$42 = 6 \times 7$$

Answer

```
import java.util.Scanner;

public class PronicNumber
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter the number to check: ");
        int num = in.nextInt();

        boolean isPronic = false;

        for (int i = 1; i <= num - 1; i++) {
            if (i * (i + 1) == num) {
                isPronic = true;
                break;
            }
        }

        if (isPronic)
            System.out.println(num + " is a pronic number");
        else
            System.out.println(num + " is not a pronic number");
    }
}
```

2018 COMPUTER APPLICATION PAPER



Question 6

Write a program in Java to accept a string in lower case and change the first letter of every word to upper case. Display the new string.

Sample input: we are in cyber world

Sample output: We Are In Cyber World

Answer

```
import java.util.Scanner;

public class String
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter a sentence:");
        String str = in.nextLine();
        String word = "";

        for (int i = 0; i < str.length(); i++) {
            if (i == 0 || str.charAt(i - 1) == ' ') {
                word += Character.toUpperCase(str.charAt(i));
            }
            else {
                word += str.charAt(i);
            }
        }

        System.out.println(word);
    }
}
```

2018 COMPUTER APPLICATION PAPER

Question 7

Design a class to overload a function volume() as follows:

- double volume (double R) – with radius (R) as an argument, returns the volume of sphere using the formula.
 $V = \frac{4}{3} \times \frac{22}{7} \times R^3$
- double volume (double H, double R) – with height(H) and radius(R) as the arguments, returns the volume of a cylinder using the formula. $V = \frac{22}{7} \times R^2 \times H$
- double volume (double L, double B, double H) – with length(L), breadth(B) and Height(H) as the arguments, returns the volume of a cuboid using the formula. $V = L \times B \times H$

Answer

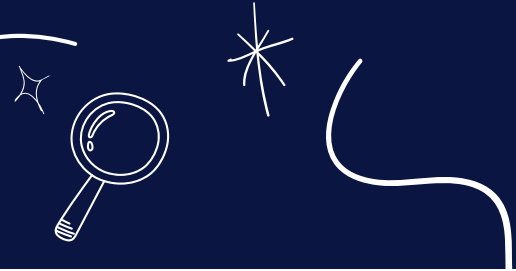
```
public class Volume
{
    double volume(double r) {
        return (4 / 3.0) * (22 / 7.0) * r * r * r;
    }

    double volume(double h, double r) {
        return (22 / 7.0) * r * r * h;
    }

    double volume(double l, double b, double h) {
        return l * b * h;
    }

    public static void main(String args[]) {
        KboatVolume obj = new KboatVolume();
        System.out.println("Sphere Volume = " +
            obj.volume(6));
        System.out.println("Cylinder Volume = " +
            obj.volume(5, 3.5));
        System.out.println("Cuboid Volume = " +
            obj.volume(7.5, 3.5, 2));
    }
}
```

2018 COMPUTER APPLICATION PAPER



Question 8

Write a menu driven program to display the pattern as per user's choice.

Pattern 1

ABCDE

ABCD

ABC

AB

A

Pattern 2

B

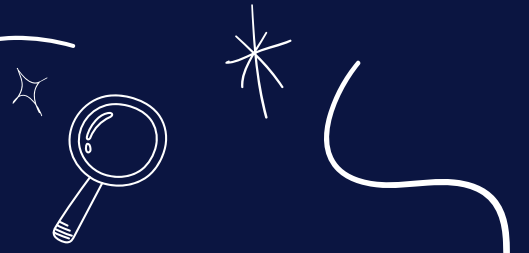
LL

UUU

EEEE

For an incorrect option, an appropriate error message should be displayed.

2018 COMPUTER APPLICATION PAPER



```
import java.util.Scanner;

public class MenuPattern
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter 1 for pattern 1");
        System.out.println("Enter 2 for Pattern 2");
        System.out.print("Enter your choice: ");
        int choice = in.nextInt();
        switch (choice) {
            case 1:
                for (int i = 69; i >= 65; i--) {
                    for (int j = 65; j <= i; j++) {
                        System.out.print((char)j);
                    }
                    System.out.println();
                }
                break;

            case 2:
                String word = "BLUE";
                int len = word.length();
                for(int i = 0; i < len; i++) {
                    for(int j = 0; j <= i; j++) {
                        System.out.print(word.charAt(i));
                    }
                    System.out.println();
                }
                break;

            default:
                System.out.println("Incorrect choice");
                break;
        }
    }
}
```


2018 COMPUTER APPLICATION PAPER

Question 9

Write a program to accept name and total marks of N number of students in two single subscript array name[] and totalmarks[]. Calculate and print:

1. The average of the total marks obtained by N number of students.
2. [average = (sum of total marks of all the students)/N]
3. Deviation of each student's total marks with the average.
4. [deviation = total marks of a student – average]

Answer

```
import java.util.Scanner;
```

```
public class SDAMarks  
{
```

```
    public static void main(String args[]) {  
        Scanner in = new Scanner(System.in);  
        System.out.print("Enter number of students: ");  
        int n = in.nextInt();  
  
        String name[] = new String[n];  
        int totalmarks[] = new int[n];  
        int grandTotal = 0;  
  
        for (int i = 0; i < n; i++) {  
            in.nextLine();  
            System.out.print("Enter name of student " + (i+1) + ": ");  
            name[i] = in.nextLine();  
            System.out.print("Enter total marks of student " + (i+1) + ": ");  
            totalmarks[i] = in.nextInt();  
            grandTotal += totalmarks[i];  
        }
```

```
        double avg = grandTotal / (double)n;  
        System.out.println("Average = " + avg);
```

```
        for (int i = 0; i < n; i++) {  
            System.out.println("Deviation for " + name[i] + " = "  
                + (totalmarks[i] - avg));  
        }
```

```
    }  
}
```

2019 COMPUTER APPLICATION PAPER

SECTION A

Question 1

(a) Name any two basic principles of Object-oriented Programming.

Answer

1. Encapsulation
2. Inheritance

(b) Write a difference between unary and binary operator.

Answer

unary operators operate on a single operand whereas binary operators operate on two operands.

(c) Name the keyword which:

1. indicates that a method has no return type.
2. makes the variable as a class variable

Answer

1. void
2. static

(d) Write the memory capacity (storage size) of short and float data type in bytes.

Answer

1. short — 2 bytes
2. float — 4 bytes

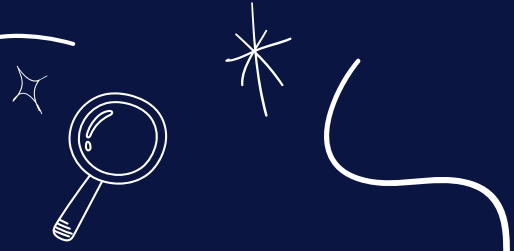
(e) Identify and name the following tokens:

1. **public**
2. **'a'**
3. **==**
4. **{ }**

Answer

1. Keyword
2. Literal
3. Operator
4. Separator

2019 COMPUTER APPLICATION PAPER



Question 2

(a) Differentiate between if else if and switch-case statements

Answer

if else if	switch-case
• if else if can test for any Boolean expression like less than, greater than, equal to, not equal to, etc. • if else if can use different expression involving unrelated variables in its different condition expressions.	• switch-case can only test if the expression is equal to any of its case constants. • switch-case statement tests the same expression against a set of constant values.

(b) Give the output of the following code:

```
String P = "20", Q = "19";  
int a = Integer.parseInt(P);  
int b = Integer.valueOf(Q);  
System.out.println(a+""+b);
```

Answer

Output of the above code is:

2019

Explanation

Both Integer.parseInt() and Integer.valueOf() methods convert a string into integer. Integer.parseInt() returns an int value that gets assigned to a. Integer.valueOf() returns an Integer object (i.e. the wrapper class of int). Due to auto-boxing, Integer object is converted to int and gets assigned to b. So a becomes 20 and b becomes 19. 2019 is printed as the output.

(c) What are the various types of errors in Java?

Answer

There are 3 types of errors in Java:

1. Syntax Error
2. Runtime Error
3. Logical Error

2019 COMPUTER APPLICATION PAPER

(d) State the data type and value of res after the following is executed:

```
char ch = '9';  
res= Character.isDigit(ch);
```

Answer

Data type of res is boolean as return type of method Character.isDigit() is boolean. Its value is true as '9' is a digit.

(e) What is the difference between the linear search and the binary search technique?

Answer

Linear search	Binary search
- Linear search works on sorted and unsorted arrays. - In Linear search, each element of the array is checked against the target value until the element is found or end of the array is reached. - Linear Search is slower.	- Binary search works on only sorted arrays. - In Binary search, array is successively divided into 2 halves and the target element is searched either in the first half or in the second half. - Binary Search is faster.

Question 3

(a) Write a Java expression for the following:

$|x^2 + 2xy|$

Answer

`Math.abs(x * x + 2 * x * y)`

(b) Write the return data type of the following functions:

1. `startsWith()`

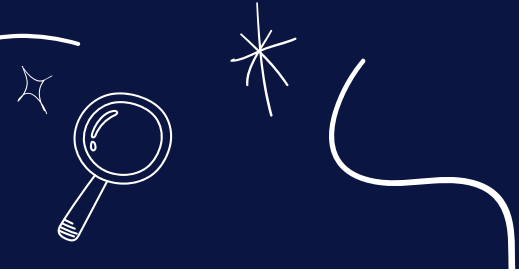
2. `random()`

Answer

1. boolean

2. double

2019 COMPUTER APPLICATION PAPER



(c) If the value of basic=1500, what will be the value of tax after the following statement is executed?

tax = basic > 1200 ? 200 :100;

Answer

Value of tax will be 200. As basic is 1500, the condition of ternary operator — basic > 1200 is true. 200 is returned as the result of ternary operator and it gets assigned to tax.

(d) Give the output of following code and mention how many times the loop will execute?

```
int i;  
for( i=5; i>=1; i--)  
{  
    if(i%2 == 1)  
        continue;  
    System.out.print(i+" ");  
}
```

Answer

Output of the above code is:

4 2

Loop executes 5 times.

(e) State a difference between call by value and call by reference.

Answer

In call by value, actual parameters are copied to formal parameters. Any changes to formal parameters are not reflected onto the actual parameters. In call by reference, formal parameters refer to actual parameters. The changes to formal parameters are reflected onto the actual parameters.

(f) Give the output of the following:

Math.sqrt(Math.max(9,16))

Answer

The output is 4.0.

First Math.max(9,16) is evaluated. It returns 16, the greater of its arguments. Math.sqrt(Math.max(9,16)) becomes Math.sqrt(16). It returns the square root of 16 which is 4.0.

2019 COMPUTER APPLICATION PAPER

(g) Write the output for the following:

```
String s1 = "phoenix"; String s2 ="island";  
System.out.println(s1.substring(0).concat(s2.substring(2)));  
System.out.println(s2.toUpperCase());
```

Answer

The output of above code is:

phoenixland

ISLAND

Explanation

s1.substring(0) results in phoenix. s2.substring(2) results in land.
concat method joins both of them resulting in phoenixland.
s2.toUpperCase() converts island to uppercase.

(h) Evaluate the following expression if the value of x=2, y=3 and z=1.

$v = x + --z + y++ + y$

Answer

$v = x + --z + y++ + y$

$\Rightarrow v = 2 + 0 + 3 + 4$

$\Rightarrow v = 9$

(i) String x[] = {"Artificial intelligence", "IOT", "Machine learning", "Big data"};

Give the output of the following statements:

1. System.out.println(x[3]);

2. System.out.println(x.length);

Answer

1. Big data

2. 4

(j) What is meant by a package? Give an example.

Answer

A package is a named collection of Java classes that are grouped on the basis of their functionality. For example, java.util.

2019 COMPUTER APPLICATION PAPER

SECTION B

Question 4

Design a class name ShowRoom with the following description:
Instance variables / Data members:

String name — To store the name of the customer
long mobno — To store the mobile number of the customer
double cost — To store the cost of the items purchased
double dis — To store the discount amount
double amount — To store the amount to be paid after discount

Member methods:

ShowRoom() — default constructor to initialize data members
void input() — To input customer name, mobile number, cost
void calculate() — To calculate discount on the cost of purchased items, based on following criteria

Cost.	Discount (in percentage)
Less than or equal to ₹10000.	5%
More than ₹10000 and less than or equal to ₹20000.	10%
More than ₹20000 and less than or equal to ₹35000.	15%
More than ₹35000.	20%

void display() — To display customer name, mobile number, amount to be paid after discount.

Write a main method to create an object of the class and call the above member methods.

2019 COMPUTER APPLICATION PAPER

```
import java.util.Scanner;

public class ShowRoom
{
    private String name;
    private long mobno;
    private double cost;
    private double dis;
    private double amount;

    public ShowRoom()
    {
        name = "";
        mobno = 0;
        cost = 0.0;
        dis = 0.0;
        amount = 0.0;
    }

    public void input() {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter customer name: ");
        name = in.nextLine();
        System.out.print("Enter customer mobile no: ");
        mobno = in.nextLong();
        System.out.print("Enter cost: ");
        cost = in.nextDouble();
    }

    public void calculate() {
        int disPercent = 0;
        if (cost <= 10000)
            disPercent = 5;
        else if (cost <= 20000)
            disPercent = 10;
        else if (cost <= 35000)
            disPercent = 15;
        elsedisPercent = 20;

        dis = cost * disPercent / 100.0;
        amount = cost - dis;
    }

    public void display() {
        System.out.println("Customer Name: " + name);
        System.out.println("Mobile Number: " + mobno);
        System.out.println("Amout after discount: " + amount);
    }

    public static void main(String args[]) {
        ShowRoom obj = new ShowRoom();
        obj.input();
        obj.calculate();
        obj.display();
    }
}
```

2019 COMPUTER APPLICATION PAPER

Question 5

Using the switch-case statement, write a menu driven program to do the following:

(a) To generate and print Letters from A to Z and their Unicode

Letters. Unicode

A. 65

B. 66

.

.

.

Z 90

(b) Display the following pattern using iteration (looping) statement:

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

Answer

```
import java.util.Scanner;
```

```
public class CK
```

```
{
```

```
    public static void main(String args[]) {
```

```
        Scanner in = new Scanner(System.in);
```

```
        System.out.println("Enter 1 for letters and Unicode");
```

```
        System.out.println("Enter 2 to display the pattern");
```

```
        System.out.print("Enter your choice: ");
```

```
        int ch = in.nextInt();
```

```
        switch(ch) {
```

```
            case 1:
```

```
                System.out.println("Letters\tUnicode");
```

```
                for (int i = 65; i <= 90; i++)
```

```
                    System.out.println((char)i + "\t" + i);
```

```
                break;
```

```
            case 2:
```

```
                for (int i = 1; i <= 5; i++) {
```

```
                    for (int j = 1; j <= i; j++)
```

```
                        System.out.print(j + " ");
```

```
                    System.out.println();
```

```
                }
```

```
                break;
```

```
            default:
```

```
                System.out.println("Wrong choice");
```

```
                break;
```

```
        }
```

```
    }
```

```
}
```

2019 COMPUTER APPLICATION PAPER

Question 6

Write a program to input 15 integer elements in an array and sort them in ascending order using the bubble sort technique.

Answer

```
import java.util.Scanner;

public class BubbleSort
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        int n = 15;
        int arr[] = new int[n];

        System.out.println("Enter the elements of the array:");
        for (int i = 0; i < n; i++) {
            arr[i] = in.nextInt();
        }

        //Bubble Sort
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int t = arr[j];
                    arr[j] = arr[j+1];
                    arr[j+1] = t;
                }
            }
        }

        System.out.println("Sorted Array:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```


2019 COMPUTER APPLICATION PAPER

Question 7

Design a class to overload a function series() as follows:

(a) void series (int x, int n) – To display the sum of the series given below:

$x^1 + x^2 + x^3 + \dots + x^n$ terms

(b) void series (int p) – To display the following series:

0, 7, 26, 63 p terms

(c) void series () – To display the sum of the series given below:

$\frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{10}$

Answer

```
public class OverloadSeries
{
    void series(int x, int n) {
        long sum = 0;
        for (int i = 1; i <= n; i++) {
            sum += Math.pow(x, i);
        }
        System.out.println("Sum = " + sum);
    }

    void series(int p) {
        for (int i = 1; i <= p; i++) {
            int term = (int)(Math.pow(i, 3) - 1);
            System.out.print(term + " ");
        }
        System.out.println();
    }

    void series() {
        double sum = 0.0;
        for (int i = 2; i <= 10; i++) {
            sum += 1.0 / i;
        }
        System.out.println("Sum = " + sum);
    }
}
```

2019 COMPUTER APPLICATION PAPER

Question 8

Write a program to input a sentence and convert it into uppercase and count and display the total number of words starting with a letter 'A'.

Example:

Sample Input: ADVANCEMENT AND APPLICATION OF INFORMATION TECHNOLOGY ARE EVER CHANGING.

Sample Output: Total number of words starting with letter 'A' = 4

Answer

```
import java.util.Scanner;

public class WordsWithLetterA
{
    public static void main(String args[]) {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter a string: ");
        String str = in.nextLine();
        str = " " + str; //Add space in the beginning of string
        int len = str.length();
        str = str.toUpperCase();
        for (int i = 0; i < len - 1; i++) {
            if (str.charAt(i) == ' ' && str.charAt(i + 1) == 'A')
                c++;
        }
        System.out.println("Total number of words starting with letter 'A' = " + c);
    }
}
```

Question 9

A tech number has even number of digits. If the number is split in two equal halves, then the square of sum of these halves is equal to the number itself. Write a program to generate and print all four digits tech numbers.

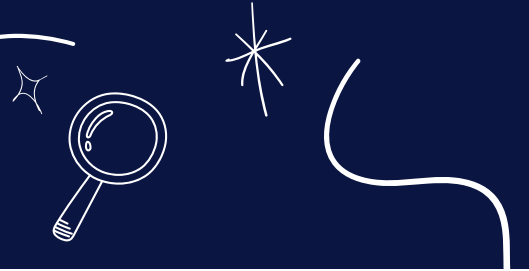
Example:

Consider the number 3025

Square of sum of the halves of 3025 = $(30 + 25)^2$
 $(55)^2$

= 3025 is a tech number.

2019 COMPUTER APPLICATION PAPER



Answer

```
public class TechNumbers
{
    public static void main(String args[]) {
        for (int i = 1000; i <= 9999; i++) {
            int secondHalf = i % 100;
            int firstHalf = i / 100;
            int sum = firstHalf + secondHalf;
            if (i == sum * sum)
                System.out.println(i);
        }
    }
}
```

2022 COMPUTER APPLICATION PAPER

SECTION A

Question 1

(i) Return data type of `isLetter(char)` is

1. **Boolean**
2. **boolean**
3. **bool**
4. **char**

Answer

`boolean`

(II) Method that converts a character to uppercase is

1. **toUpper()**
2. **ToUpperCase()**
3. **toUpperCase()**
4. **toUpperCase(char)**

Answer

`toUpperCase(char)`

(iii) Give output of the following String methods:
`"SUCESS".indexOf('S') + "SUCCESS".lastIndexOf('S')`

1. **0**
2. **5**
3. **6**
4. **-5**

Answer

6

(iv) Corresponding wrapper class of float data type is

1. **Float**
2. **float**
3. **Float**
4. **Floating**

Answer

`Float`

(V) class is used to convert a primitive data type to its corresponding object.

1. **String**
2. **Wrapper**
3. **System**
4. **Math**

Answer

`Wrapper`

2022 COMPUTER APPLICATION PAPER

SECTION A

Question 1

(i) Return data type of `isLetter(char)` is

1. **Boolean**
2. **boolean**
3. **bool**
4. **char**

Answer

`boolean`

(II) Method that converts a character to uppercase is

1. **toUpper()**
2. **ToUpperCase()**
3. **toUpperCase()**
4. **toUpperCase(char)**

Answer

`toUpperCase(char)`

(iii) Give output of the following String methods:
`"SUCESS".indexOf('S') + "SUCCESS".lastIndexOf('S')`

1. **0**
2. **5**
3. **6**
4. **-5**

Answer

6

(iv) Corresponding wrapper class of float data type is

1. **Float**
2. **float**
3. **Float**
4. **Floating**

Answer

`Float`

(V) class is used to convert a primitive data type to its corresponding object.

1. **String**
2. **Wrapper**
3. **System**
4. **Math**

Answer

`Wrapper`

2022 COMPUTER APPLICATION PAPER



(VI) Give the output of the following code:

```
System.out.println("Good".concat("Day"));
```

1. GoodDay
2. Good Day
3. Goodday
4. goodDay

Answer

GoodDay

(VII) A single dimensional array contains N elements. What will be the last subscript?

1. N
2. N - 1
3. N - 2
4. N + 1

Answer

N - 1

(VIII) The access modifier that gives least accessibility is:

1. private
2. public
3. protected
4. package

Answer

private

(IX) Give the output of the following code :

```
String A = "56.0", B = "94.0";
```

```
double C = Double.parseDouble(A);
```

```
double D = Double.parseDouble(B);
```

```
System.out.println((C+D));
```

1. 100
2. 150.0
3. 100.0
4. 150

Answer

150.0

(X) What will be the output of the following code?

```
System.out.println("Lucknow".substring(0,4));
```

1. Lucknow
2. Luckn
3. Luck
4. luck

Answer

Luck

2022 COMPUTER APPLICATION PAPER

SECTION B

Question 2

Define a class to perform binary search on a list of integers given below, to search for an element input by the user, if it is found display the element along with its position, otherwise display the message "Search element not found".

2, 5, 7, 10, 15, 20, 29, 30, 46, 50

```
import java.util.Scanner;

public class BinarySearch
{
    public static void main(String args[]) {

        Scanner in = new Scanner(System.in);
        int arr[] = {2, 5, 7, 10, 15, 20, 29, 30, 46, 50};

        System.out.print("Enter number to search: ");
        int n = in.nextInt();

        int l = 0, h = arr.length - 1, index = -1;
        while (l <= h) {
            int m = (l + h) / 2;
            if (arr[m] < n)
                l = m + 1;
            else if (arr[m] > n)
                h = m - 1;
            else {
                index = m;
                break;
            }
        }

        if (index == -1) {
            System.out.println("Search element not found");
        }
        else {
            System.out.println(n + " found at position " + index);
        }
    }
}
```

2022 COMPUTER APPLICATION PAPER

Question 3

Define a class to declare a character array of size ten. Accept the characters into the array and display the characters with highest and lowest ASCII (American Standard Code for Information Interchange) value. EXAMPLE :

INPUT:

'R', 'z', 'q', 'A', 'N', 'p', 'm', 'U', 'Q', 'F'

OUTPUT :

Character with highest ASCII value = z

Character with lowest ASCII value = A

```
import java.util.Scanner;
```

```
public class ASCIIVal  
{
```

```
    public static void main(String[] args)  
    {
```

```
        Scanner in = new Scanner(System.in);  
        char ch[] = new char[10];  
        int len = ch.length;
```

```
        System.out.println("Enter 10 characters:");  
        for (int i = 0; i < len; i++)  
        {  
            ch[i] = in.nextLine().charAt(0);  
        }
```

```
        char h = ch[0];  
        char l = ch[0];
```

```
        for (int i = 1; i < len; i++)  
        {  
            if (ch[i] > h)  
            {  
                h = ch[i];  
            }  
  
            if (ch[i] < l)  
            {  
                l = ch[i];  
            }  
        }
```

```
        System.out.println("Character with highest ASCII value: " + h);  
        System.out.println("Character with lowest ASCII value: " + l);  
    }
```

```
}
```

2022 COMPUTER APPLICATION PAPER

Question 4

Define a class to declare an array of size twenty of double datatype, accept the elements into the array and perform the following :

**Calculate and print the product of all the elements.
Print the square of each element of the array.**

```
import java.util.Scanner;

public class SDADouble
{
    public static void main(String[] args)
    {
        Scanner in = new Scanner(System.in);
        double arr[] = new double[20];
        int l = arr.length;
        double p = 1.0;

        System.out.println("Enter 20 numbers:");
        for (int i = 0; i < l; i++)
        {
            arr[i] = in.nextDouble();
        }

        for (int i = 0; i < l; i++)
        {
            p *= arr[i];
        }
        System.out.println("Product = " + p);

        System.out.println("Square of array elements :");
        for (int i = 0; i < l; i++)
        {
            double sq = Math.pow(arr[i], 2);
            System.out.println(sq + " ");
        }
    }
}
```

2022 COMPUTER APPLICATION PAPER

Question 5

Define a class to accept a string, and print the characters with the uppercase and lowercase reversed, but all the other characters should remain the same as before.

EXAMPLE:

INPUT : WelCoMe_2022

OUTPUT : wELcOmE_2022

```
import java.util.Scanner;

public class ChangeCase
{
    public static void main(String args[])
    {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter a string:");
        String str = in.nextLine();
        int len = str.length();
        String rev = "";

        for (int i = 0; i < len; i++)
        {
            char ch = str.charAt(i);
            if (Character.isLetter(ch))
            {
                if (Character.isUpperCase(ch))
                {
                    rev += Character.toLowerCase(ch);
                }
                else
                {
                    rev += Character.toUpperCase(ch);
                }
            }
            else
            {
                rev += ch;
            }
        }

        System.out.println(rev);
    }
}
```

2022 COMPUTER APPLICATION PAPER

Question 6

Define a class to declare an array to accept and store ten words. Display only those words which begin with the letter 'A' or 'a' and also end with the letter 'A' or 'a'.

EXAMPLE :

Input : Hari, Anita, Akash, Amrita, Alina, Devi Rishab, John, Farha, AMITHA

Output: Anita

Amrita

Alina

AMITHA

```
import java.util.Scanner;
```

```
public class KboatWords  
{
```

```
    public static void main(String args[])  
    {  
        Scanner in = new Scanner(System.in);  
        String names[] = new String[10];  
        int l = names.length;  
        System.out.println("Enter 10 names : ");
```

```
        for (int i = 0; i < l; i++)  
        {  
            names[i] = in.nextLine();  
        }
```

```
        System.out.println("Names that begin and end with letter A are:");
```

```
        for(int i = 0; i < l; i++)  
        {  
            String str = names[i];  
            int len = str.length();  
            char begin = Character.toUpperCase(str.charAt(0));  
            char end = Character.toUpperCase(str.charAt(len - 1));  
            if (begin == 'A' && end == 'A') {  
                System.out.println(str);  
            }  
        }
```

```
    }  
}
```


2022 COMPUTER APPLICATION PAPER

Question 7

Define a class to accept two strings of same length and form a new word in such a way that, the first character of the first word is followed by the first character of the second word and so on.

Example :

Input string 1 – BALL

Input string 2 – WORD

OUTPUT : BWAOLRLD

```
import java.util.Scanner;
```

```
public class KboatStringMerge  
{
```

```
    public static void main(String args[])  
    {
```

```
        Scanner in = new Scanner(System.in);  
        System.out.println("Enter String 1: ");  
        String s1 = in.nextLine();  
        System.out.println("Enter String 2: ");  
        String s2 = in.nextLine();  
        String str = "";  
        int len = s1.length();
```

```
        if(s2.length() == len)  
        {
```

```
            for (int i = 0; i < len; i++)  
            {  
                char ch1 = s1.charAt(i);  
                char ch2 = s2.charAt(i);  
                str = str + ch1 + ch2;
```

```
            }  
            System.out.println(str);
```

```
        }  
        else
```

```
        {  
            System.out.println("Strings should be of same length");  
        }
```

```
    }  
}
```

2022 COMPUTER APPLICATION PAPER

Question 7

Define a class to accept two strings of same length and form a new word in such a way that, the first character of the first word is followed by the first character of the second word and so on.

Example :

Input string 1 – BALL

Input string 2 – WORD

OUTPUT : BWAOLRLD

```
import java.util.Scanner;
```

```
public class KboatStringMerge  
{
```

```
    public static void main(String args[])  
    {
```

```
        Scanner in = new Scanner(System.in);  
        System.out.println("Enter String 1: ");  
        String s1 = in.nextLine();  
        System.out.println("Enter String 2: ");  
        String s2 = in.nextLine();  
        String str = "";  
        int len = s1.length();
```

```
        if(s2.length() == len)  
        {
```

```
            for (int i = 0; i < len; i++)  
            {  
                char ch1 = s1.charAt(i);  
                char ch2 = s2.charAt(i);  
                str = str + ch1 + ch2;
```

```
            }  
            System.out.println(str);
```

```
        }  
        else
```

```
        {  
            System.out.println("Strings should be of same length");  
        }
```

```
    }  
}
```

2023 COMPUTER APPLICATION PAPER

SECTION A

Question 1

(I) A mechanism where one class acquires the properties of another class:

1. Polymorphism
2. Inheritance
3. Encapsulation
4. Abstraction

Answer

Inheritance

(ii) Identify the type of operator &&:

1. ternary
2. unary
3. logical
4. relational

Answer

logical

(iii) The Scanner class method used to accept words with space:

1. next()
2. nextLine()
3. Next()
4. nextString()

Answer

nextLine()

(iv) The keyword used to call package in the program:

1. extends
2. export
3. import
4. package

Answer

import

(V) What value will `Math.sqrt(Math.ceil(15.3))` return?

1. 16.0
2. 16
3. 4.0
4. 5.0

Answer

4.0

2023 COMPUTER APPLICATION PAPER

(Vi) The absence of which statement leads to fall through situation in switch case statement?

1. **continue**
2. **break**
3. **return**
4. **System.exit(0)**

Answer

break

(vii) State the type of loop in the given program segment:

```
for (int i = 5; i != 0; i -= 2)
```

```
    System.out.println(i);
```

1. **finite**
2. **infinite**
3. **null**
4. **fixed**

Answer

infinite

(Viii) Write a method prototype name check() which takes an integer argument and returns a char:

1. **char check()**
2. **void check (int x)**
3. **check (int x)**
4. **char check (int x)**

Answer

char check (int x)

(Ix) The number of values that a method can return is:

1. **1**
2. **2**
3. **3**
4. **4**

Answer

1

(X) Predict the output of the following code snippet:

```
String P = "20", Q ="22";
```

```
int a = Integer.parseInt(P);
```

```
int b = Integer.valueOf(Q);
```

```
System.out.println(a + " " + b);
```

1. **20**
2. **20 22**
3. **2220**
4. **22**

Answer

20 22

2023 COMPUTER APPLICATION PAPER

(XI) The String class method to join two strings is:

1. `concat(String)`
2. `<string>.joint(string)`
3. `concat(char)`
4. `Concat()`

Answer

`concat(String)`

(XII) The output of the function "COMPOSITION".substring(3, 6):

1. `POSI`
2. `POS`
3. `MPO`
4. `MPOS`

Answer

`POS`

(XIII) `int x = (int)32.8;` is an example of typecasting.

1. `implicit`
2. `automatic`
3. `explicit`
4. `coercion`

Answer

`explicit`

(XIV) The code obtained after compilation is known as:

1. `source code`
2. `object code`
3. `machine code`
4. `java byte code`

Answer

`java byte code`

(XV) Missing a semicolon in a statement is what type of error?

1. `Logical`
2. `Syntax`
3. `Runtime`
4. `No error`

Answer

`Syntax`

2023 COMPUTER APPLICATION PAPER

(XVI) Consider the following program segment and select the output of the same when $n = 10$:

```
switch(n)
{
    case 10 : System.out.println(n*2);
    case 4 : System.out.println(n*4); break;
    default : System.out.println(n);
}
```

1. 20
2. 40
3. 10
4. 4
5. 20, 40
6. 10, 10

Answer

20
40

(XVII) A method which does not modify the value of variables is termed as:

1. Impure method
2. Pure method
3. Primitive method
4. User defined method

Answer

Pure method

(XVIII) When an object of a Wrapper class is converted to its corresponding primitive data type, it is called as

1. Boxing
2. Explicit type conversion
3. Unboxing
4. Implicit type conversion

Answer

Unboxing

(XIX) The number of bits occupied by the value 'a' are:

1. 1 bit
2. 2 bits
3. 4 bits
4. 16 bits

Answer

16 bits

2023 COMPUTER APPLICATION PAPER



(XX) Method which is a part of a class rather than an instance of the class is termed as:

1. **Static method**
2. **Non static method**
3. **Wrapper class**
4. **String method**

Answer

Static method

Question 2

(I) Write the Java expression for $(a + b)^x$

Answer

`Math.pow(a + b, x)`

(II) Evaluate the expression when the value of $x = 4$:

`x *= --x + x++ + x`

Answer

The given expression is evaluated as follows:

`x *= --x + x++ + x` ($x = 4$)

`x *= 3 + x++ + x` ($x = 3$)

`x *= 3 + 3 + x` ($x = 4$)

`x *= 3 + 3 + 4` ($x = 4$)

`x *= 10` ($x = 4$)

`x = x * 10` ($x = 4$)

`x = 4 * 10`

`x = 40`

(III) Convert the following do...while loop to for loop:

```
int x = 10;
```

```
do
```

```
{
```

```
  x—;
```

```
  System.out.print(x);
```

```
}while (x>=1);
```

Answer

```
for(int x = 9; x >= 0; x--)
```

```
{
```

```
  System.out.print(x);
```

```
}
```

Convert the following do...while loop to for loop:

```
int x = 10;
```

```
do
```

```
{
```

```
  x—;
```

```
  System.out.print(x);
```

```
}while (x>=1);
```

Answer

```
for(int x = 9; x >= 0; x--)
```

```
{
```

```
  System.out.print(x);
```

```
}
```

2023 COMPUTER APPLICATION PAPER



(IV) Give the output of the following Character class methods:

(a) Character.toUpperCase('a')

(b) Character.isLetterOrDigit('#')

Answer

(a) Character.toUpperCase('a')

Output

A

(b) Character.isLetterOrDigit('#')

Output

false

(V) Rewrite the following code using the if-else statement:

```
int m = 400;
```

```
double ch = (m > 300) ? (m / 10.0) * 2 : (m / 20.0) - 2;
```

Answer

```
int m = 400;
```

```
double ch = 0.0;
```

```
if(m > 300)
```

```
    ch = (m / 10.0) * 2;
```

```
else ch = (m / 20.0) - 2;
```

(VI) Give the output of the following program segment:

```
int n = 4279; int d;
```

```
while(n > 0)
```

```
{ d = n % 10;
```

```
System.out.println(d);
```

```
n = n / 100;
```

```
}
```

Answer

Output

9

2

(VII) Give the output of the following String class methods:

(a) "COMMENCEMENT".lastIndexOf('M')

(b) "devote".compareTo("DEVOTE")

Answer

(a) "COMMENCEMENT".lastIndexOf('M')

Output

8

(VIII) Consider the given array and answer the questions given below:

```
int x[ ] = {4, 7, 9, 66, 72, 0, 16};
```

(a) What is the length of the array?

(b) What is the value in x[4]?

Answer

(a) 7

(b) 72

2023 COMPUTER APPLICATION PAPER



(IX) Name the following:

(a) What is an instance of the class called?

(b) The method which has same name as that of the class name.

Answer

(a) Object.

(b) Constructor.

(X) Write the value of n after execution:

```
char ch ='d';
```

```
int n = ch + 5;
```

Answer

The value of n is 105.

SECTION B

Question 3

Design a class with the following specifications:

Class name: Student

Member variables:

name — name of student

age — age of student

mks — marks obtained

stream — stream allocated

(Declare the variables using appropriate data types)

Member methods:

void accept() — Accept name, age and marks using methods of Scanner class.

void allocation() — Allocate the stream as per following criteria:

mks.

>= 300.

>= 200 and < 300.

>= 75 and < 200.

< 75.

Stream

Science and Computer

Commerce and Computer

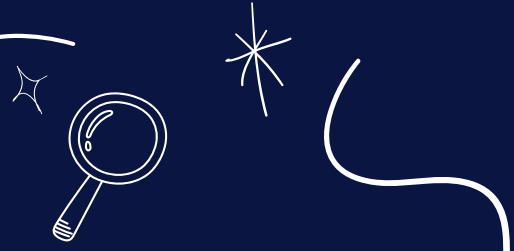
Arts and Animation

Try Again

void print() – Display student name, age, mks and stream allocated.

Call all the above methods in main method using an object.

2023 COMPUTER APPLICATION PAPER



```
import java.util.Scanner;

public class Student
{
    private String name;
    private int age;
    private double mks;
    private String stream;

    public void accept()
    {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter student name: ");
        name = in.nextLine();
        System.out.print("Enter age: ");
        age = in.nextInt();
        System.out.print("Enter marks: ");
        mks = in.nextDouble();
    }

    public void allocation()
    {
        if (mks < 75)
            stream = "Try again";
        else if (mks < 200)
            stream = "Arts and Animation";
        else if (mks < 300)
            stream = "Commerce and Computer";
        else stream = "Science and Computer";
    }

    public void print()
    {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Marks: " + mks);
        System.out.println("Stream allocated: " + stream);
    }

    public static void main(String args[]) {
        Student obj = new Student();
        obj.accept();
        obj.allocation();
        obj.print();
    }
}
```

2023 COMPUTER APPLICATION PAPER



Question 4

Define a class to accept 10 characters from a user. Using bubble sort technique arrange them in ascending order. Display the sorted array and original array.

Answer

```
import java.util.Scanner;
```

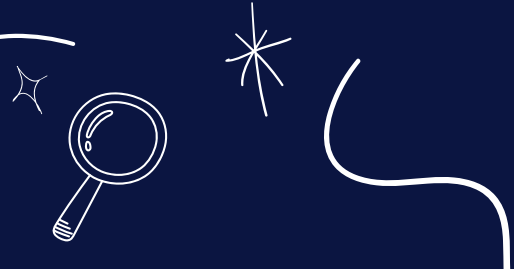
```
public class CharBubbleSort
{
    public static void main(String args[])
    {
        Scanner in = new Scanner(System.in);
        char ch[] = new char[10];
        System.out.println("Enter 10 characters:");
        for (int i = 0; i < ch.length; i++) {
            ch[i] = in.nextLine().charAt(0);
        }

        System.out.println("Original Array");
        for (int i = 0; i < ch.length; i++) {
            System.out.print(ch[i] + " ");
        }

        //Bubble Sort
        for (int i = 0; i < ch.length - 1; i++) {
            for (int j = 0; j < ch.length - 1 - i; j++) {
                if (ch[j] > (ch[j + 1])) {
                    char t = ch[j];
                    ch[j] = ch[j + 1];
                    ch[j + 1] = t;
                }
            }
        }

        System.out.println("\nSorted Array");
        for (int i = 0; i < ch.length; i++) {
            System.out.print(ch[i] + " ");
        }
    }
}
```

2023 COMPUTER APPLICATION PAPER



Question 5

Define a class to overload the function print as follows:

void print() - to print the following format

1 1 1 1

2 2 2 2

3 3 3 3

4 4 4 4

5 5 5 5

void print(int n) - To check whether the number is a lead number.

A lead number is the one whose sum of even digits are equal to sum of odd digits.

e.g. 3669

odd digits sum = $3 + 9 = 12$

even digits sum = $6 + 6 = 12$

3669 is a lead number.

2023 COMPUTER APPLICATION PAPER



```
import java.util.Scanner;

public class MethodOverload
{
    public void print()
    {
        for(int i = 1; i <= 5; i++)
        {
            for(int j = 1; j <= 4; j++)
            {
                System.out.print(i + " ");
            }
            System.out.println();
        }
    }

    public void print(int n)
    {
        int d = 0;
        int evenSum = 0;
        int oddSum = 0;
        while( n != 0)
        {
            d = n % 10;
            if (d % 2 == 0)
                evenSum += d;
            else oddSum += d;
            n = n / 10;
        }

        if(evenSum == oddSum)
            System.out.println("Lead number");
        else System.out.println("Not a lead number");
    }

    public static void main(String args[])
    {
        KboatMethodOverload obj = new KboatMethodOverload();
        Scanner in = new Scanner(System.in);

        System.out.println("Pattern: ");
        obj.print();

        System.out.print("Enter a number: ");
        int num = in.nextInt();
        obj.print(num);
    }
}
```

2023 COMPUTER APPLICATION PAPER



Question 6

Define a class to accept a String and print the number of digits, alphabets and special characters in the string.

Example:

S = "KAPILDEV@83"

Output:

Number of digits – 2

Number of Alphabets – 8

Number of Special characters – 1

import java.util.Scanner;

Answer

```
public class tCount
{
    public static void main(String args[])
    {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter a string:");
        String str = in.nextLine();

        int len = str.length();

        int ac = 0;
        int sc = 0;
        int dc = 0;
        char ch;

        for (int i = 0; i < len; i++) {
            ch = str.charAt(i);
            if (Character.isLetter(ch))
                ac++;
            else if (Character.isDigit(ch))
                dc++;
            else if (!Character.isWhitespace(ch))
                sc++;
        }

        System.out.println("No. of Digits = " + dc);
        System.out.println("No. of Alphabets = " + ac);
        System.out.println("No. of Special Characters = " + sc);
    }
}
```

2023 COMPUTER APPLICATION PAPER



Question 7

Define a class to accept values into an array of double data type of size 20. Accept a double value from user and search in the array using linear search method. If value is found display message "Found" with its position where it is present in the array. Otherwise display message "not found".

Answer

```
import java.util.Scanner;

public class LinearSearch
{
    public static void main(String args[])
    {
        Scanner in = new Scanner(System.in);

        double arr[] = new double[20];
        int l = arr.length;
        int i = 0;

        System.out.println("Enter array elements: ");
        for (i = 0; i < l; i++)
        {
            arr[i] = in.nextDouble();
        }

        System.out.print("Enter the number to search: ");
        double n = in.nextDouble();

        for (i = 0; i < l; i++)
        {
            if (arr[i] == n)
            {
                break;
            }
        }

        if (i == l)
        {
            System.out.println("Not found");
        }
        else
        {
            System.out.println(n + " found at index " + i);
        }
    }
}
```

2023 COMPUTER APPLICATION PAPER



Question 8

Define a class to accept values in integer array of size 10. Find sum of one digit number and sum of two digit numbers entered. Display them separately.

Example:

Input: a[] = {2, 12, 4, 9, 18, 25, 3, 32, 20, 1}

Output:

Sum of one digit numbers : $2 + 4 + 9 + 3 + 1 = 19$

Sum of two digit numbers : $12 + 18 + 25 + 32 + 20 = 107$

Answer

```
import java.util.Scanner;
```

```
public class DigitSum
{
```

```
    public static void main(String args[])
    {
```

```
        Scanner in = new Scanner(System.in);
        int oneSum = 0, twoSum = 0, d = 0;
        int arr[] = new int[10];
        System.out.println("Enter 10 numbers");
        int l = arr.length;
```

```
        for (int i = 0; i < l; i++)
        {
            arr[i] = in.nextInt();
        }
```

```
        for (int i = 0; i < l; i++)
        {
            if(arr[i] >= 0 && arr[i] < 10 )
                oneSum += arr[i];
            else if(arr[i] >= 10 && arr[i] < 100 )
                twoSum += arr[i];
        }
```

```
        System.out.println("Sum of 1 digit numbers = "+ oneSum);
        System.out.println("Sum of 2 digit numbers = "+ twoSum);
```

```
    }
}
```